

Spring Security

参考文档

Ben Alex, Luke Taylor

2.0.x

序言
I. 入门
1. 介绍
1.1. Spring Security是什么?
1.2. 历史
1.3. 发行版本号
1.4. 获得源代码
2. Security命名空间配置
2.1. 介绍
2.1.1. 命名空间的设计
2.2. 开始使用安全命名空间配置
2.2.1. 配置web.xml
2.2.2. 最小 <http>配置
2.2.2.1. auto-config包含了什么?
2.2.2.2. 表单和基本登录选项
2.2.3. 使用其他认证提供器
2.2.3.1. 添加一个密码编码器
2.3. 高级web特性
2.3.1. Remember-Me认证
2.3.2. 添加HTTP/HTTPS信道安全
2.3.3. 同步Session控制
2.3.4. OpenID登录
2.3.5. 添加你自己的filter
2.3.5.1. 设置自定义 AuthenticationEntryPoint
2.3.6. 防止Session固定攻击
2.4. 保护方法
2.4.1. <global-method-security>元素
2.4.1.1. 使用protect-pointcut添加安全切点
2.4.2. intercept-methods Bean 渲染器
2.5. 默认的AccessDecisionManager
2.5.1. 自定义AccessDecisionManager
2.6. 默认验证管理器
3. 示例程序
3.1. Tutorial示例
3.2. Contacts
3.3. LDAP例子
3.4. CAS例子
3.5. Pre-Authentication例子
4. Spring Security社区
4.1. 任务跟踪

4.2. 成为参与者
4.3. 更多信息
II. 总体结构
5. 技术概述
5.1. 运行环境
5.2. 共享组件
5.2.1. SecurityContextHolder, SecurityContext 和 Authentication对象
5.2.2. UserDetailsService
5.2.3. GrantedAuthority
5.2.4. 小结
5.3. 验证
5.3.1. ExceptionTranslationFilter
5.3.2. AuthenticationEntryPoint
5.3.3. AuthenticationProvider
5.3.4. 直接设置SecurityContextHolder的内容
5.4. 安全对象
5.4.1. 安全和AOP建议
5.4.2. AbstractSecurityInterceptor
5.4.2.1. 配置属性是什么?
5.4.2.2. RunAsManager
5.4.2.3. AfterInvocationManager
5.4.2.4. 扩展安全对象模型
6. 支持的基础设施
6.1. 国际化
6.2. 过滤器
6.3. 标签库
6.3.1. 配置
6.3.2. 使用
7. 信道安全
7.1. 总述
7.2. 配置
7.3. 总结
III. 认证
8. 通用认证服务
8.1. 机制,供应者和入口
8.2. UserDetails 和相关类型
8.2.1. 内存里认证
8.2.2. JDBC认证
8.2.2.1. 默认用户数据库表结构
8.3. 并行会话处理
8.4. 认证标签库
9. DAO认证提供器
9.1. 综述
9.2. 配置
10. LDAP认证
10.1. 综述
10.2. 在Spring Security里使用LDAP
10.3. 配置LDAP服务器
10.3.1. 使用嵌入测试服务器
10.3.2. 使用绑定认证
10.3.3. 读取授权
10.4. 实现类
10.4.1. LdapAuthenticator实现
10.4.1.1. 常用功能

10.4.1.2. BindAuthenticator
10.4.1.3. PasswordComparisonAuthenticator
10.4.1.4. 活动目录认证
10.4.2. 链接到LDAP服务器
10.4.3. LDAP搜索对象
10.4.3.1. FilterBasedLdapUserSearch
10.4.4. LdapAuthoritiesPopulator
10.4.5. Spring Bean配置
10.4.6. LDAP属性和自定义UserDetails
11. 表单认证机制
11.1. 概述
11.2. 配置
12. 基本认证机制
12.1. 概述
12.2. 配置
13. 摘要式认证
13.1. 概述
13.2. 配置
14. Remember-Me认证
14.1. 概述
14.2. 简单基于散列标记的方法
14.3. 持久化标记方法
14.4. Remember-Me接口和实现
14.4.1. TokenBasedRememberMeServices
14.4.2. PersistentTokenBasedRememberMeServices
15. Java认证和授权服务(JAAS)供应器
15.1. 概述
15.2. 配置
15.2.1. JAAS CallbackHandler
15.2.2. JAAS AuthorityGranter
16. 预认证场景
16.1. 预认证框架类
16.1.1. AbstractPreAuthenticatedProcessingFilter
16.1.2. AbstractPreAuthenticatedAuthenticationDetailsSource
16.1.2.1. J2eeBasedPreAuthenticatedWebAuthenticationDetailsSource
16.1.3. PreAuthenticatedAuthenticationProvider
16.1.4. PreAuthenticatedProcessingFilterEntryPoint
16.2. 具体实现
16.2.1. 请求头认证(Siteminder)
16.2.1.1. Siteminder示例配置
16.2.2. J2EE容器认证
17. 匿名认证
17.1. 概述
17.2. 配置
18. X.509认证
18.1. 概述
18.2. 把X.509认证添加到你的web系统中
18.3. 为tomcat配置SSL
19. CAS认证
19.1. 概述
19.2. CAS是如何工作的
19.3. 配置CAS客户端
20. 替换验证身份
20.1. 概述
20.2.

配置
21. 容器适配器认证
21.1. 概述
21.2. 适配器认证提供器
21.3. Jetty
21.4. JBoss
21.5. Resin
21.6. Tomcat
A. 安全数据库表结构
A.1. User表
A.1.1. 组权限
A.2. 持久登陆(Remember-Me)表
A.3. ACL表
IV. 授权
22. 通用授权概念
22.1. 授权
22.2. 处理预调用
22.2.1. AccessDecisionManager
22.2.1.1. 基于投票的AccessDecisionManager实现
22.3. 处理后决定
22.3.1. ACL-Aware AfterInvocationProviders
22.3.2. ACL-Aware AfterInvocationProviders (老 ACL 模块)
22.4. 授权标签库
23. 安全对象实现
23.1. AOP联盟 (MethodInvocation) 安全拦截器
23.1.1. 精确的 MethodSecurityInterceptor 配置
23.2. AspectJ (JoinPoint) 安全拦截器
23.3. FilterInvocation 安全拦截器
24. 领域对象安全
24.1. 概述
24.2. 关键概念
24.3. 开始
25. acegi到spring security的转换方式
25.1. Spring Security是什么
25.2. 目标
25.3. 步骤
25.4. 总结

Next
序言

Sponsored by SpringSource



序言

Spring Security为基于J2EE的企业应用软件提供了一套全面的安全解决方案。正如你在本手册中看到的那样,我们尝试为您提供一套好用,高可配置的安全系统。

安全问题是一个不断变化的目标,更重要的是寻求一种全面的,系统化的解决方案。 在安全领域我们建议你采取“分层安全”,这样让每一层确保本身尽可能的安全,并为其他层提供额外的安全保障。 每层自身越是“紧密”,你的程序就会越鲁棒越安全。 在底层,你需要处理传输安全和系统认证,减少“中间人攻击”(man-in-the-middle attacks)。 接下来,我们通常会使用防火墙,结合VPN或IP安全来确保只有获得授权的系统才能尝试连接。 在企业环境中,你可能会部署一个DMZ(demilitarized zone,隔离区),将面向公众的服务器与后端数据库,应用服务器隔离开。 在以非授权用户运行进程和文件系统安全最大化上,你的操作系统也将扮演一个关键的角色。 操作系统通常配置了自己的防火墙。 然后你要防止针对系统的拒绝服务和暴力攻击。 入侵检测系统在检测和应对攻击的时候尤其有用。这些系统可以实时屏蔽恶意TCP/IP地址。 在更高层上,你需要配置Java虚拟机,将授予不同java类型权限最小化,然后,你的应用程序要添加针对自身特定问题域的安全配。Spring Security使后者 - 应用程序安全变得更容易。

当然,你需要妥善处理上面提到的每个安全层,以及包含于每个层的管理因素。 这些管理因素具体包括:安全公告检测,补丁,人工诊断,审计,变更管理,工程管理系统,数据备份,灾难回复,性能评测,负载检测,集中日志,应急反应程序等等。

Spring Security关注的重点是在企业应用安全层为您提供服务,你将发现业务问题领域存在着各式各样的需求。银行系统跟电子商务应用就有很大的不同。电子商务系统与企业销售自动化工具又有很大不同。这些客户化需求让应用安全显得有趣,富有挑战性而且物有所值。

请阅读Part I. “入门”部分,以它作为开始。它向你介绍了整个框架和以命名空间为基础系统配置方式,让你可以很快启动并运行系统。要是想更多的了解Spring Security是如何工作和一些你可能需要用到的类,你应该阅读Part II. “总体结构”部分。本指南的其余部分使用了较传统的参考文档方式,请按照自己的需要选择阅读的部分。 我们也推荐你阅读尽可能多的在应用安全中可能出现的一般问题。 Spring Security也不是万能的,它不可能解决所有问题。 重要的一点,应用程序应该从一开始就为安全做好设计。 企图改造它也不是一个好主意。 特别的,如果你在制作一个web应用,你应该知道许多潜在的脆弱性,比如跨域脚本,伪造请求和会话劫持,这些都是你在一开始就应该考虑到的。OWASP网站(http://www.owasp.org/)维护了一个web应用脆弱性前十名的名单,还有很多有用的参考信息。

我们希望你觉得这是一篇很有用的参考指南,并欢迎您提供反馈意见和建议。

最后,欢迎加入Spring Security 社区。



Part I. 入门

本指南的后面部分提供对框架结构和实现类的深入讨论,了解它们,对你进行复杂的定制是十分重要的。在这部分,我们将介绍Spring Security 2.0,简要介绍该项目的历史,然后看看如何开始在程序中使用框架。特别是,我们将看看命名控件配置提供了一个更加简单的方式,在使用传统的spring bean配置时,你不得不实现所有类。

我们也会看看可用的范例程序。它们值得试着运行,实验,在你阅读后面的章节之前 - 你可以在对框架有了更多连接之后再回来看这些例子。

Prev | Home | Next
序言 | Sponsored by SpringSource | Chapter 1. 介绍



Chapter 1. 介绍

1.1. Spring Security是什么?

Spring Security为基于J2EE企业应用软件提供了全面安全服务。 特别是使用领先的J2EE解决方案-spring框架开发的企业软件项目。 如果你没有使用Spring开发企业软件,我们热情的推荐你仔细研究一下。 熟悉Spring-尤其是依赖注入原理-将帮助你更快的掌握Spring Security。

人们使用Spring Security有很多种原因,不过通常吸引他们的是在J2EE Servlet规范或EJB规范中找不到典型企业应用场景的解决方案。提到这些规范,特别要指出的是它们不能在WAR或EAR级别进行移植。这样,如果你更换服务器环境,就要,在新的目标环境进行大量的工作,对你的应用系统进行重新配置安全。使用Spring Security解决了这些问题,也为你提供了很多有用的,完全可定制的其他安全特性。

你可能知道,安全包括两个主要操作。 第一个被称为“认证”,是为用户建立一个他所声明的主体。 主体一般是指用户,设备或可以在你系统中执行行动的其他系统。 “授权”指的一个用户能否在你的应用中执行某个操作。 在到达授权判断之前,身份的主体已经由身份验证过程建立了。 这些概念是通用的,不是Spring Security特有的。

在身份验证层面,Spring Security广泛支持各种身份验证模式。 这些验证模型绝大多数都由第三方提供,或正在开发的有关标准机构提供的,例如Internet Engineering Task Force。 作为补充,Spring Security也提供了自己的一套验证功能。 Spring Security目前支持认证一体化和如下认证技术:

- HTTP BASIC authentication headers (一个基于IEFT RFC的标准)
- HTTP Digest authentication headers (一个基于IEFT RFC的标准)
- HTTP X.509 client certificate exchange (一个基于IEFT RFC的标准)
- LDAP (一个非常常见的跨平台认证需要做法,特别是在大环境)
- Form-based authentication (提供简单用户接口的需求)
- OpenID authentication
- Computer Associates Siteminder
- JA-SIG Central Authentication Service (也被称为CAS,这是一个流行的开源单点登录系统)
- Transparent authentication context propagation for Remote Method Invocation (RMI) and HttpInvoker (一个Spring远程调用协议)
- Automatic "remember-me" authentication (这样你可以设置一段时间,避免在一段时间内还需要重新验证)
- Anonymous authentication (允许任何调用,自动假设一个特定的安全主体)
- Run-as authentication (这在一个会话内使用不同安全身份的时候是非常有用的)
- Java Authentication and Authorization Service (JAAS)
- Container integration with JBoss, Jetty, Resin and Tomcat (这样,你可以继续使用容器管理认证,如果想的话)
- Java Open Source Single Sign On (JOSSO) *
- OpenNMS Network Management Platform *
- AppFuse *
- AndroMDA *
- Mule ESB *
- Direct Web Request (DWR) *
- Grails *

- Tapestry *
- JTrac *
- Jasypt *
- Roller *
- Elastic Plath *
- Atlassian Crowd *
- 你自己的认证系统(向下看)

(* 是指由第三方提供,查看我们的整合网页,获得最新详情的链接。)

许多独立软件供应商(ISVs, independent software vendors)采用Spring Security,是因为它拥有丰富灵活的验证模型。 这样,无论终端用户需要什么,他们都可以快速集成到系统中,不用花很多功夫,也不用让用户改变运行环境。 如果上述的验证机制都没有满足你的需要,Spring Security是一个开放的平台,编写自己的验证机制是十分简单的。 Spring Security的许多企业用户需要整合不遵循任何特定安全标准的“遗留”系统,Spring Security在这类系统上也表现的很好。

有时基本的认证是不够的。 有时你需要根据在主体和应用交互的方式来应用不同的安全措施。 比如,你可能,为了保护密码,不被窃听或受到中间人攻击,希望确保请求只通过HTTPS到达。 或者,你希望确保发起请求的是一个真正的人,而不是机器人或其他自动化程序。 这对保护找回密码不被暴力攻击特别有帮助,或者让别人更难复制你程序中的关键内容。 为了帮助你实现这些目标,Spring Security支持自动“通道安全”,整合jcaptcha一体化进行人类用户检测。

Spring Security不仅提供认证功能,也提供了完备的授权功能。 在授权方面主要有三个领域,授权web请求,授权被调用方法,授权访问单个对象的实例。 为了帮你了解它们之间的区别,对照考虑授在Servlet规范web模式安全,EJB容器管理安全,和文件系统安全方面的授权方式。 Spring Security在所有这些重要领域都提供了完备的能力,我们将在这份参考指南的后面进行探讨。

1.2. 历史

Spring Security开始于2003年年底,那时候叫“spring的acegi安全系统”。 起因是Spring开发者邮件列表中的一个问题,有人提问是否考虑提供一个基于spring的安全实现。 在当时Spring的社区相对较小(尤其是和今天的规模比!),其实Spring本身是从2003年初才作为一个sourceforge的项目出现的。 对这个问题的回应是,这的确是一个值得研究的领域,虽然限于时间问题阻碍了对它的继续研究。

有鉴于此,一个简单的安全实现建立起来了,但没有发布。 几周之后,spring社区的其他成员询问安全问题,代码就被提供给了他们。 随后又有人请求,在2004年一月左右,有20人在使用这些代码。 另外一些人加入到这些先行者中来,并建议在sourceforge上建立一个项目,项目在2004年3月正式建立起来。

在早期,项目本身没有自己的认证模块。 认证过程都是依赖容器管理安全的,而acegi则注重授权。 这在一开始是合适的,但随着越来越多用户要求提供额外的容器支持,基于容器认证的限制就显现出来了。 还有一个有关的问题,向容器的classpath中添加新jar,常常让最终用户感到困惑,又容易出现配置错误。

随后acegi加入了认证服务。大约一年后,acegi成为spring的官方子项目。 经过了两年半在许多生产软件项目中的活跃使用和数以万计的改善和社区的贡献,1.0.0最终版本发布于2006年5月。

acegi在2007年年底,正式成为spring组合项目,被更名为“Spring Security”。

现在,Spring Security成为了一个强大而又活跃的开源社区。 在Spring Security支持论坛上有成千上万的信息。 有一个积极的核心开发团队专职开发,一个积极的社区定期共享补丁并支持他们的同伴。

1.3. 发行版本号

了解spring Security发行版本号是非常有用的。它可以帮助你判断升级到新的版本是否需要花费很大的精力。 我们使用apache便携式运行项目版本指南,可以在以下网址查看`http://apr.apache.org/versioning.html`。 为了方便大家,我们引用页面上的一段介绍:

“版本号是一个包含三个整数的组合:主要版本号.次要版本号.补丁。 基本思路是主要版本是不兼容的,大规模升级API。 次要版本号在源代码和二进制要与老版本保持兼容,补丁则意味着向前向后的完全兼容。”

1.4. 获得源代码

Spring Security是一个开源项目,我们大力推荐你从subversion获得源代码。 这样你可以获得所有的示例,你可以很容易的建立目前最新的项目。 获得项目的源代码对调试也有很大的帮助。 异常堆栈不再是模糊的黑盒问题,你可以直接找到发生问题的那一行,查找发生了什么额外难题。 源代码也是项目的最终文档,常常是最简单的方法,找出这些事情是如何工作的。

要像获得项目最新的源代码,使用如下subversion命令:

```
    svn checkout
http://acegisecurity.svn.sourceforge.net/svnroot/acegisecurity/spring-
security/trunk/
```

你可以获得特定版本的源代码 `http://acegisecurity.svn.sourceforge.net/svnroot/acegisecurity/spring-security/tags/`.



Security命名空间配置

2.1. 介绍

从Spring-2.0开始可以使用命名空间的配置方式。 使用它呢,可以通过附加xml架构,为传统的spring beans应用环境语法做补充。 你可以在spring参考文档得到更多信息。 命名空间元素可以简单的配置单个bean,或使用更强大的,定义一个备用配置语法,这可以更加紧密的匹配问题域,隐藏用户背后的复杂性。 简单元素可能隐藏事实,多种bean和处理步骤添加到应用环境中。 比如,把下面的security命名元素添加到应用环境中,将会为测试用途,在应用内部启动一个内嵌LDAP服务器:

```
  <security:ldap-server />
```

这比配置一个Apache目录服务器bean要简单得多。 最常见的替代配置需求都可以使用ldap-server元素的属性进行配置,这样用户就不用担心他们需要设置什么,不用担心bean里的各种属性。 [1]。使用一个良好的XML编辑器来编辑应用环境文件,应该提供可用的属性和元素信息。 我们推荐你尝试一下 SpringSource工具套件 因为它具有处理spring组合命名空间的特殊功能。

要开始在你的应用环境里使用security命名空间,你所需要的就是把架构声明添加到你的应用环境文件里:

```
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:security="http://www.springframework.org/schema/security"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-2.0.xsd
              http://www.springframework.org/schema/security
http://www.springframework.org/schema/security/spring-security-2.0.2.xsd">
    ...
</beans>
```

在许多例子里,你会看到(在示例中)应用,我们通常使用"security"作为默认的命名空间,而不是"beans",这意味着我们可以省略所有security命名空间元素的前缀,使上下文更容易阅读。 如果你把应用上下文分割成单独的文件,让你的安全配置都放到其中一个文件里,这样更容易使用这种配置方法。 你的安全应用上下文应该像这样开头

```
<beans:beans xmlns="http://www.springframework.org/schema/security"
  xmlns:beans="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-2.0.xsd
              http://www.springframework.org/schema/security
http://www.springframework.org/schema/security/spring-security-2.0.2.xsd">
    ...
</beans:beans>
```

就在这一章里,我们都将假设使用这种语法。

2.1.1. 命名空间的设计

命名空间被用来设计成,处理框架内最常见的功能,提供一个简化和简洁的语法,使他们在一个应用程序里。 这种设计是基于框架内的大型依赖,可以分割成下面这些部分:

- *Web/HTTP*安全 - 最复杂的部分。设置过滤器和相关的服务bean来应用框架验证机制, 保护URL,渲染登录和错误页面还有更多。

业务类(方法)安全 - 可选的安全服务层。

- *AuthenticationManager* - 通过框架的其它部分,处理认证请求。一个默认的实例会通过命名空间注册到内部。
- *AccessDecisionManager* - 提供访问的决定,适用于web以及方法的安全。一个默认的主体会被注册,但是你也可以选择自定义一个,使用正常的spring bean语法进行声明。
- *AuthenticationProviders* - 验证管理器验证用户的机制。 该命名空间提供几种标准选项,意味着使用传统语法添加自定义bean。
- *UserDetailsService* - 密切相关的认证供应器,但往往也需要由其他bean需要。

下一章中,我们将看到如何把这些放到一起工作。

2.2. 开始使用安全命名空间配置

在本节中,我们来看看如何使用一些框架里的主要配置,建立一个命名空间配置。 我们假设你最初想要尽快的启动运行,为已有的web应用添加认证支持和权限控制,使用一些测试登录。 然后我们看一下如何修改一下,使用数据库或其他安全信息残酷。 在以后的章节里我们将引入更多高级的命名空间配置选项。

2.2.1. 配置web.xml

我们要做的第一件事是把下面的filter声明添加到 `web.xml` 文件中:

```
<filter>
  <filter-name>springSecurityFilterChain</filter-name>
  <filter-class>org.springframework.web.filter.DelegatingFilterProxy</filter-class>
</filter>

<filter-mapping>
  <filter-name>springSecurityFilterChain</filter-name>
  <url-pattern>/*</url-pattern>
</filter-mapping>
```

这是为Spring Security的web机制提供了一个调用钩子。 `DelegatingFilterProxy`是一个Spring Framework的类,它可以代理一个application context中定义的Spring bean所实现的filter。 这种情况下,bean的名字是"springSecurityFilterChain",这是由命名空间创建的用于处理web安全的一个内部的机制。 注意,你不应该自己使用这个bean的名字。 一旦你把这个添加到你的`web.xml`中,你就准备好开始编辑呢的application context文件了。web安全服务是使用`<http>`元素配置的。

2.2.2. 最小 <http>配置

只需要进行如下配置就可以实现安全配置:

```
  <http auto-config='true'>
    <intercept-url pattern="/**" access="ROLE_USER" />
  </http>
```

这表示,我们要保护应用程序中的所有URL,只有拥有 `ROLE_USER`角色的用户才能访问。

> **Note**
>
> 你可以使用多个`<intercept-url>`元素为不同URL的集合定义不同的访问需求,它们会被归入一个有序队列中,每次取出最先匹配的一个元素使用。 所以你必须把期望使用的匹配条件放到最上边。

要是想添加一些用户,你可以直接使用下面的命名空间直接定义一些测试数据:

```
  <authentication-provider>
    <user-service>
```

```
      <user name="jimi" password="jimispassword" authorities="ROLE_USER, ROLE_ADMIN"
/>
      <user name="bob" password="bobspassword" authorities="ROLE_USER" />
    </user-service>
  </authentication-provider>
```

上面的配置定义了两个用户,他们在应用程序中的密码和角色(用在权限控制上)。 也可以从一个标准properties文件中读取这些信息,使用user-service的properties属性。 参考in-memory authentication获得更多信息。 使用<authentication-provider>元素意味着用户信息将被认证管理用作处理认证请求。

现在,你可以启动程序,然后就会进入登录流程了。 试试这个,或者试试工程里的"tutorial"例子。 上述配置实际上把很多服务添加到了程序里,因为我们使用了auto-config属性。 比如,表单登录和"remember-me"服务自动启动了。

如果你熟悉以前的版本,你很可能已经猜到了这里是怎么回事。 <http>元素会创建一个FilterChainProxy和filter使用的bean。 以前常常出现的,因为filter顺序不正确产生的问题,不会再出现了,现在这些过滤器的位置都是预定义好的。

<authentication-provider>元素创建了一个DaoAuthenticationProvider bean,<user-service>元素创建了一个InMemoryDaoImpl。 一个ProviderManager bean通常是由命名空间过程系统创建的,DaoAuthenticationProvider自动注册到它上面。 你可以在命名空间附录中找到关于创建这个bean的更新信息。

2.2.2.1. auto-config包含了什么?

我们在上面用到的auto-config属性,其实是下面这些配置的缩写:

```
  <http>
    <intercept-url pattern="/**" access="ROLE_USER" />
    <form-login />
    <anonymous />
    <http-basic />
    <logout />
    <remember-me />
  </http>
```

这些元素分别与form-login,匿名认证,基本认证,注销处理和remember-me对应。 他们拥有各自的属性,来改变他们的具体行为。

> **auto-config需要一个UserDetailsService**
>
> 使用auto-config的时候如果没配置UserDetailsService就会出现错误(比如,如果你使用了LDAP认证)。 这是因为remember-me服务在auto-config="true"的时候启动了,它的认证机制需要UserDetailsService来实现(参考Remember-me章获得更多信息)。 如果你遇到了一个因为没定义UserDetailsService造成的问题,那就试着去掉auto-config配置(或者是其他你配置上的remember-me)。

2.2.2.2. 表单和基本登录选项

你也许想知道,在需要登录的时候,去哪里找这个登录页面,到现在为止我们都没有提到任何的HTML或JSP文件。 实际上,如果我们没有确切的指定一个页面用来登录,Spring Security会自动生成一个,基于可用的功能,为这个URL使用标准的数据,处理提交的登录,然后发送到默认的目标URL。 然而,命名空间提供了许多支持,让你可以自定义这些选项。比如,如果你想实现自己的登录页面,你可以使用:

```
  <http auto-config='true'>
    <intercept-url pattern="/login.jsp*" filters="none"/>
    <intercept-url pattern="/**" access="ROLE_USER" />
    <form-login login-page='/login.jsp'/>
```

```
  </http>
```

注意,你依旧可以使用auto-config。 这个form-login元素会覆盖默认的设置。 也要注意我们需要添加额外的intercept-url元素,指定用来做登录的页面的URL,这些URL不应该被安全filter处理。 否则,这些请求会被/**部分拦截,它没法访问到登录页面。 如果你想使用基本认证而不是表单登录,可以把配置修改成如下所示:

```
  <http auto-config='true'>
    <intercept-url pattern="/**" access="ROLE_USER" />
    <http-basic />
  </http>
```

基本身份认证会被优先用到,在用户尝试访问一个受保护的资源时,用来提示用户登录。 在这种配置中,表单登录依然是可用的,如果你还想用的话,比如,把一个登录表单内嵌到其他页面里。

2.2.2.2.1. 设置一个默认的提交登陆目标

如果在进行表单登陆之前,没有试图去访问一个被保护的资源,default-target-url就会起作用。 这是用户登陆后会跳转到的URL,默认是"/"。 你也可以把always-use-default-target属性配置成"true",这样用户就会一直跳转到这一页(无论登陆是“跳转过来的”还是用户特定进行登陆)。 如果你的系统一直需要用户从首页进入,就可以使用它了,比如:

```
  <http>
    <intercept-url pattern='/login.htm*' filters='none'/>
    <intercept-url pattern='/**' access='ROLE_USER' />
    <form-login login-page='/login.htm' default-target-url='/home.htm' always-use-
default-target='true' />
  </http>
```

2.2.3. 使用其他认证提供器

现实中,你会需要更大型的用户信息源,而不是写在application context里的几个名字。 多数情况下,你会想把用户信息保存到数据库或者是LDAP服务器里。 LDAP命名空间会在LDAP章里详细讨论,所以我们这里不会讲它。 如果你自定义了一个Spring Security的UserDetailsService实现,在你的application context中名叫"myUserDetailsService",然后你可以使用下面的验证。

```
  <authentication-provider user-service-ref='myUserDetailsService'/>
```

如果你想用数据库,可以使用下面的方式

```
  <authentication-provider>
    <jdbc-user-service data-source-ref="securityDataSource"/>
  </authentication-provider>
```

这里的"securityDataSource"就是 DataSource bean在application context里的名字,它指向了包含着Spring Security用户信息的表。 另外,你可以配置一个Spring Security JdbcDaoImpl bean,使用user-service-ref属性指定:

```
  <authentication-provider user-service-ref='myUserDetailsService'/>

  <beans:bean id="myUserDetailsService"
```

```
class="org.springframework.security.userdetails.jdbc.JdbcDaoImpl">
    <beans:property name="dataSource" ref="dataSource"/>
  </beans:bean>
```

你也可以通过在bean定义中添加<custom-authentication-provider>元素来使用标准AuthenticationProvider bean。 查看Section 2.6, “默认验证管理器”了解更多信息。

2.2.3.1. 添加一个密码编码器

你的密码数据通常要使用一种散列算法进行编码。 使用<password-encoder>元素支持这个功能。 使用SHA加密密码,原始的认证供应器配置,看起来就像这样:

```
<authentication-provider>
  <password-encoder hash="sha"/>
  <user-service>
    <user name="jimi" password="d7e6351eaa13189a5a3641bab846c8e8c69ba39f"
authorities="ROLE_USER, ROLE_ADMIN" />
    <user name="bob" password="4e7421b1b8765d8f9406d87e7cc6aa784c4ab97f"
authorities="ROLE_USER" />
  </user-service>
</authentication-provider>
```

在使用散列密码时,用盐值防止字典攻击是个好主意,Spring Security也支持这个功能。 理想情况下,你可能想为每个用户随机生成一个盐值,不过,你可以使用从UserDetailsService读取出来的UserDetails对象中的属性。 比如,使用username属性,你可以这样用:

```
<password-encoder hash="sha">
  <salt-source user-property="username"/>
</password-encoder>
```

你可以通过password-encoder的ref属性,指定一个自定义的密码编码器bean。 这应该包含application context中一个bean的名字,它应该是Spring Security的PasswordEncoder接口的一个实例。

2.3. 高级web特性

2.3.1. Remember-Me认证

参考Remember-Me章获得remember-me命名空间配置的详细信息。

2.3.2. 添加HTTP / HTTPS信道安全

如果你的同时支持HTTP和HTTPS协议,然后你要求特定的URL只能使用HTTPS,这时可以直接使用<intercept-url>的requires-channel属性:

```
  <http>
    <intercept-url pattern="/secure/**" access="ROLE_USER" requires-
channel="https"/>
    <intercept-url pattern="/**" access="ROLE_USER" requires-channel="any"/>
    ...
  </http>
```

使用了这个配置以后,如果用户通过HTTP尝试访问"/secure/**"匹配的网址,他们会先被重定向到HTTPS网址下。 可用的选项有"http", "https" 或 "any"。 使用"any"意味着使用HTTP或HTTPS都可以。

如果你的程序使用的不是HTTP或HTTPS的标准端口,你可以用下面的方式指定端口对应关系:

```
  <http>
    ...
    <port-mappings>
      <port-mapping http="9080" https="9443"/>
    </port-mappings>
  </http>
```

你可以在Chapter 7. 信道安全找到更详细的讨论。

2.3.3. 同步Session控制

如果你希望限制单个用户只能登录到你的程序一次,Spring Security通过添加下面简单的部分支持这个功能。 首先,你需要把下面的监听器添加到你的web.xml文件里,让Spring Security获得session生存周期事件:

```
<listener>
  <listener-
class>org.springframework.security.ui.session.HttpSessionEventPublisher</listener-
class>
</listener>
```

然后,在你的application context加入如下部分:

```
  <http>
    ...
    <concurrent-session-control max-sessions="1" />
  </http>
```

这将防止一个用户重复登录好几次-第二次登录会让第一次登录失效。 通常我们更想防止第二次登录,这时候我们可以使用

```
  <http>
    ...
    <concurrent-session-control max-sessions="1" exception-if-maximum-
exceeded="true"/>
  </http>
```

第二次登录将被阻止。

2.3.4. OpenID登录

命名空间支持OpenID登录,替代普通的表单登录,或作为一种附加功能,只需要进行简单的修改:

```
  <http>
    <intercept-url pattern="/**" access="ROLE_USER" />
    <openid-login />
  </http>
```

你应该注册一个OpenID供应器(比如myopenid.com),然后把用户信息添加到你的内存<user-service>中:

```
  <user name="http://jimi.hendrix.myopenid.com/" password="notused"
authorities="ROLE_USER" />
```

你应该可以使用myopenid.com网站登录来进行验证了。

2.3.5. 添加你自己的filter

如果你以前使用过Spring Security,你就应该知道这个框架里维护了一个过滤器链,来提供服务。 你也许想把你自己的过滤器添加到链条的特定位置,或者使用一个Spring Security的过滤器,这个过滤器现在还没有在命名空间配置中进行支持(比如CAS)。 或者你想要使用一个特定版本的标准命名空间过滤器,比如`<form-login>`创建的`AuthenticationProcessingFilter`,从而获得一些额外的配置选项的优势,这些可以通过直接配置bean获得。 你如何在命名空间配置里实现这些功能呢? 过滤器链现在已经不能直接看到了。

过滤器顺序在使用命名空间的时候是被严格执行的。 每个Spring Security过滤器都实现了Spring的`Ordered`接口,这些通过命名空间穿件的过滤器在初始化的时候就预先被排好序了。 标准的过滤器在命名空间里都有自己的假名,有关创建过滤器的过滤器,假名和命名空间元素,属性可以在Table 2.1, “标准过滤器假名和顺序”中找到。

Table 2.1. 标准过滤器假名和顺序

假名	过滤器累	命名空间元素或属性
CHANNEL_FILTER	`ChannelProcessingFilter`	`http/intercept-url`
CONCURRENT_SESSION_FILTER	`ConcurrentSessionFilter`	`http/concurrent-session-control`
SESSION_CONTEXT_INTEGRATION_FILTER	`HttpSessionContextIntegrationFilter`	`http`
LOGOUT_FILTER	`LogoutFilter`	`http/logout`
X509_FILTER	`X509PreAuthenticatedProcessigFilter`	`http/x509`
PRE_AUTH_FILTER	`AstractPreAuthenticatedProcessingFilter` Subclasses	N/A
CAS_PROCESSING_FILTER	`CasProcessingFilter`	N/A
AUTHENTICATION_PROCESSING_FILTER	`AuthenticationProcessingFilter`	`http/form-login`
BASIC_PROCESSING_FILTER	`BasicProcessingFilter`	`http/http-basic`
SERVLET_API_SUPPORT_FILTER	`SecurityContextHolderAwareRequestFilter`	`http/@servlet-api-provision`
REMEMBER_ME_FILTER	`RememberMeProcessingFilter`	`http/remember-me`
ANONYMOUS_FILTER	`AnonymousProcessingFilter`	`http/anonymous`
EXCEPTION_TRANSLATION_FILTER	`ExceptionTranslationFilter`	`http`
NTLM_FILTER	`NtlmProcessingFilter`	N/A
FILTER_SECURITY_INTERCEPTOR	`FilterSecurityInterceptor`	`http`
SWITCH_USER_FILTER	`SwitchUserProcessingFilter`	N/A

你可以把你自己的过滤器添加到队列中,使用`custom-filter`元素,使用这些名字中的一个,来指定你的过滤器应该出现的位置:

```
<beans:bean id="myFilter" class="com.mycompany.MySpecialAuthenticationFilter">
  <custom-filter position="AUTHENTICATION_PROCESSING_FILTER"/>
</beans:bean>
```

你还可以使用`after` 或 `before`属性,如果你想把你的过滤器添加到队列中另一个过滤器的前面或后面。 可以分别在`position`属性使用"FIRST" 或 "LAST"来指定你想让你的过滤器出现在队列元素的前面或后面。

避免过滤器位置发生冲突

如果你插入了一个自定义的过滤器,而这个过滤器可能与命名空间自己创建的标准过滤器放在同一个位置上,这样首要的是你不要错误包含命名空间的版本信息。 避免使用auto-config属性,然后删除所有会创建你希望替换的过滤器的元素。

注意,你不能替换那些<http>元素自己使用而创建出的过滤器,比如HttpSessionContextIntegrationFilter, ExceptionTranslationFilter 或FilterSecurityInterceptor。

如果你替换了一个命名空间的过滤器,而这个过滤器需要一个验证入口点(比如,认证过程是通过一个未通过验证的用户访问受保护资源的尝试来触发的),你将也需要添加一个自定义的入口点bean。

2.3.5.1. 设置自定义 AuthenticationEntryPoint

如果你没有通过命名空间,使用表单登陆,OpenID或基本认证,你可能想定义一个认证过滤器,并使用传统bean语法定义一个入口点然后把它链接到命名空间里,就像我们已经看到的那样。 对应的AuthenticationEntryPoint可以使用<http>元素中的entry-point-ref属性来进行设置。

CAS示例程序是一个在命名空间中使用自定义bean的好例子,包含这种语法。 如果你对认证入口点并不熟悉,可以在技术纵览章中找到关于它们的讨论。

2.3.6. 防止Session固定攻击

Session固定攻击是一个潜在危险,当一个恶意攻击者可以创建一个session访问一个网站的时候,然后说服另一个用户登录到同一个会话上(比如,发送给他们一个包含了session标识参数的链接)。 Spring Security通过在用户登录时,创建一个新session来防止这个问题。 如果你不需要保护,或者它与其他一些需求冲突,你可以通过使用<http>中的session-fixation-protection属性来配置它的行为,它有三个选项

- migrateSession - 创建一个新session,把原来session中所有属性复制到新session中。这是默认值。
- none - 什么也不做,继续使用原来的session。
- newSession - 创建一个新的“干净的”session,不会复制session中的数据。

2.4. 保护方法

Spring Security 2.0大幅改善了对你的服务层方法添加安全。 如果你使用Java 5或更高版本,还支持JSR-250的安全注解,这与框架提供的@secured注解相似。 你可以为单个bean提供安全控制,通过使用intercept-methods元素装饰bean声明,或者你可以使用AspectJ方式的切点来控制实体服务层里的多个bean。

2.4.1. <global-method-security>元素

这个元素用来在你的应用程序中启用基于安全的注解(通过在这个元素中设置响应的属性),也可以用来声明将要应用在你的实体application context中的安全切点组。 你应该只定义一个<global-method-security>元素。 下面的声明同时启用Spring Security的@Secured和JSR-250注解:

```
  <global-method-security secured-annotations="enabled" jsr250-
annotations="enabled"/>
```

向一个方法(或一个类或一个接口)添加注解,会限制对这个方法的访问。 Spring Security原生注解支持为方法定义一系列属性。 这些属性将传递给AccessDecisionManager,进行决策。 这个例子来自tutorial sample,如果你想在程序中使用方法安全,那么它是一个很好的开始:

```
  public interface BankService {

    @Secured("IS_AUTHENTICATED_ANONYMOUSLY")
    public Account readAccount(Long id);

    @Secured("IS_AUTHENTICATED_ANONYMOUSLY")
    public Account[] findAccounts();
```

```
    @Secured("ROLE_TELLER")
    public Account post(Account account, double amount);
  }
```

2.4.1.1. 使用protect-pointcut添加安全切点

protect-pointcut是非常强大的,它让你可以用简单的声明对多个bean的进行安全声明。 参考下面的例子:

```
  <global-method-security>
    <protect-pointcut expression="execution(* com.mycompany.*Service.*(..))"
access="ROLE_USER"/>
  </global-method-security>
```

这样会保护application context中的符合条件的bean的所有方法,这些bean要在com.mycompany包下,类名以"Service"结尾。 ROLE_USER的角色才能调用这些方法。 就像URL匹配一样,指定的匹配要放在切点队列的最前面,第一个匹配的表达式才会被用到。

2.4.2. intercept-methods Bean 渲染器

可选语法,让你可以指定安全,为一个特定的bean,使用元素在bean里。

```
<bean:bean id="target" class="com.mycompany.myapp.MyBean">
    <intercept-methods>
        <protect method="set*" access="ROLE_ADMIN" />
        <protect method="get*" access="ROLE_ADMIN,ROLE_USER" />
        <protect method="doSomething" access="ROLE_USER" />
    </intercept-methods>
</bean:bean>
```

这让你配置安全属性,为单独的方法,在bean或简单通配符模式。

2.5. 默认的AccessDecisionManager

这章假设你有一些Spring Security权限控制有关的架构知识。 如果没有,你可以跳过这段,以后再来看,因为这章只是为了自定义的用户设置的,需要在简单基于角色安全的基础上加一些客户化的东西。

当你使用命名空间配置时,默认的AccessDecisionManager实例会自动注册,然后用来为方法调用和web URL访问做验证,这些都是基于你设置的intercept-url和protect-pointcut权限属性内容(和注解中的内容,如果你使用注解控制方法的权限)。

默认的策略是使用一个AffirmativeBased AccessDecisionManager ,以及RoleVoter 和AuthenticatedVoter。

2.5.1. 自定义AccessDecisionManager

如果你需要使用一个更复杂的访问控制策略,把它设置给方法和web安全是很简单的。

对于方法安全,你可以设置global-security里的access-decision-manager-ref属性,用对应 AccessDecisionManager bean在application context里的id:

```
  <global-method-security access-decision-manager-ref="myAccessDecisionManagerBean">
    ...
  </global-method-security>
```

web安全安全的语法也是一样,但是放在http元素里:

```
  <http access-decision-manager-ref="myAccessDecisionManagerBean">
    ...
  </http>
```

2.6. 默认验证管理器

我们大概知道命名空间配置会自动为我们注册一个验证管理器bean。 这是一个Spring Security的ProviderManager类,如果你以前使用过框架,应该对它很熟悉了。 如果你在命名空间中使用了HTTP或方法安全,你不能使用自定义的AuthenticationProvider,但你已经可以对使用的AuthenticationProvider进行完全的控制了,所以这不会是一个问题。

你也许想为ProviderManager注册另外的AuthenticationProvider bean,你可以使用<custom-authentication-provider>元素实现。比如:

```
  <bean id="casAuthenticationProvider"
      class="org.springframework.security.providers.cas.CasAuthenticationProvider">
    <security:custom-authentication-provider />
    ...
  </bean>
```

另一个常见的需求是,上下文中的另一个bean可能需要引用AuthenticationManager。 这里有一个特殊的元素,可以让你为AuthenticationManager注册一个别名,然后你可以application context的其他地方使用这个名字。

```
  <security:authentication-manager alias="authenticationManager"/>

  <bean id="customizedFormLoginFilter"
class="org.springframework.security.ui.webapp.AuthenticationProcessingFilter">
     <security:custom-filter position="AUTHENTICATION_PROCESSING_FILTER"/>
     <property name="authenticationManager" ref="authenticationManager"/>
     ...
  </bean>
```

[1] 你可以在LDAP的章节里,找到更多有关使用的ldap-server的元素。



示例程序

项目中包含了很多web实例程序。 为了不让下载包变得太大,我们只把"tutorial"和"contacts"两个例子放到了zip发布包里。 你可以自己编译部署它们,也可以从Maven中央资源库里获得单独的war文件。 我们建议你使用前一种方法。 你可以按照简介里的介绍获得源代码,使用maven编译它们也很简单。 如果你需要的话,可以在 http://www.springframework.org/spring-security/ 网站上找到更多信息。

3.1. Tutorial示例

这个tutorial示例是带你入门的很好的一个基础例子。 它完全使用了简单命名空间配置。 编译好的应用就放在zip发布包中,已经准备好发布到你的web容器中(`spring-security-samples-tutorial-2.0.x.war`)。 使用了form-based验证机制,与常用的remember-me验证提供器相结合,自动使用cookie记录登录信息。

我们推荐你从tutorial例子开始,因为XML非常小,也很容易看懂。 更重要的是,你很容易就可以把这个XML文件(和它对应的`web.xml`入口)添加到你的程序中。 只有做基本集成成功的时候,我们建议你试着添加方法验证和领域对象安全。

3.2. Contacts

Contacts例子,是一个很高级的例子,它在基本程序安全上附加了领域对象的访问控制列表,演示了更多强大的功能。

要发布它,先把Spring Security发布中的war文件按复制到你的容器的`webapps`目录下。 这个war文件应该叫做`spring-security-samples-contacts-2.0.0.war`(后边的版本号,很大程度上依赖于你使用的发布版本)。

在启动你的容器之后,检测一下程序是不是加载了,访问`http://localhost:8080/contacts`(或是其他你把war发布后,对应于你web容器的URL)。

下一步,点击"Debug"。 你将看到需要登录的提示,这页上会有一些测试用的用户名和密码。 随便使用其中的一个通过认证,就会看到结果页面。 它应该会包含下面这样的一段成功信息:

```
Authentication object is of type: org.springframework.security.providers.UsernamePasswordAuthenticationToken


Authentication object as a String:
  org.springframework.security.providers.UsernamePasswordAuthenticationToken@1f127853:
  Principal: org.springframework.security.userdetails.User@b07ed00:
  Username: rod; Password: [PROTECTED]; Enabled: true; AccountNonExpired: true;
  credentialsNonExpired: true; AccountNonLocked: true;
  Granted Authorities: ROLE_SUPERVISOR, ROLE_USER; Password: [PROTECTED]; Authenticated: true;
  Details: org.springframework.security.ui.WebAuthenticationDetails@0:
  RemoteIpAddress: 127.0.0.1; SessionId: k5qypsawgpwb;
  Granted Authorities: ROLE_SUPERVISOR, ROLE_USER

Authentication object holds the following granted authorities:

ROLE_SUPERVISOR (getAuthority(): ROLE_SUPERVISOR)
ROLE_USER (getAuthority(): ROLE_USER)

SUCCESS! Your web filters appear to be properly configured!
```

一旦你成功的看到了上面的信息,就可以返回例子程序的主页,点击"Manage"了。 然后你就可以尝试这个程序了。 注意,只有当前登录的用户对应的联系信息会显示出来,而且只有`ROLE_SUPERVISOR`权限的用户可以授权删除他们的联系信息。 在这场景后面,`MethodSecurityInterceptor`保护着业务对象。

陈程序允许你修改访问控制列表,分配不同的联系方式。 确保自己好好试用过,看看程序里的上下文XML文件,搞明白它是如何运行的。

3.3. LDAP例子

LDAP例子程序提供了一个基础配置,同时使用命名空间配置和使用传统方式bean的配置方式,这两种配置方式都写在application

context文件里。 这意味着,在这个程序里,其实是配置了两个定义验证提供器。

3.4. CAS例子

CAS示例要求你同时运行CAS服务器和CAS客户端。 它没有包含在发布包里,你应该使用 简介中的描述获得源代码。 你可以在sample/cas目录下找到对应的文件。 这里还有一个Readme.txt文件,解释如何从源代码树中直接运行服务器和客户端,提供完全的SSL支持。 你应该下载CAS服务器web程序(一个war文件)从CAS网站,把它放到samples/cas/server目录下。

3.5. Pre-Authentication例子

这个例子演示了如何从pre-authentication框架绑定bean,从J2EE容器中获得有用的登录信息。 用户名和角色是由容器设置的。

代码在samples/preauth目录下。



Spring Security社区

4.1. 任务跟踪

Spring Security使用JIRA管理bug报告和扩充请求。 如果你发现一个bug,请使用JIRA提交一个报告。 不要把它放到支持论坛上,邮件列表里,或者直接发邮件给项目开发者。 这样做法是特设的,我们更愿意使用更正式的方式管理bug。

如果有可能,最好为你的任务报告提供一个Junit单元测试,演示每一种不正确的行为。 或者,更好的是,提供一个不定来修正这个问题。 一般来说,扩充也也可以提交到任务跟踪系统里,虽然我们只接受提供了对应的单元测试的扩充请求。因为保证项目的测试覆盖率是非常必要的,它需要适当的进行维护。

你可以访问任务跟踪的网址 http://jira.springframework.org/browse/SEC。

4.2. 成为参与者

我们欢迎你加入到Spring Security项目中来。 这里有很多贡献的方式,包括在论坛上阅读别人的帖子发表回复,写新代码,提升旧代码,帮助写文档,开发例子或指南,或简单的提供建议。

4.3. 更多信息

欢迎大家为Spring Security提出问题或评论。 你可以使用Spring社区论坛网址http://forum.springframework.org 同框架的其他用户讨论Spring Security。 记得使用JIRA提交bug,这部分在上面提到过了。 我们也欢迎大家加入Acegisecurity-developer邮件列表,特别是对设计部分的讨论。 交通量是非常轻的。



Part II. 总体结构

和大多数软件一样,Spring Security有一个一定的中央接口,类和抽象概念,贯穿整个框架。 在指南的这个部分,我们将介绍Spring Security,在解释这些中央元素之前,有必要进行成功的计划,执行Spring Security整合。

Prev | Home | Next
Chapter 4. Spring Security社区 | Sponsored by SpringSource | Chapter 5. 技术概述



技术概述

5.1. 运行环境

Spring Security可以运行在标准的Java 1.4运行环境下。 它也支持Java 5.0,不过这部分代码单独打包起来,放到发布的,文件名是"tiger"前缀的JAR文件里。 因为Spring Security的目标是自己容器内管理,所以不需要为你的Java运行环境进行什么特别的配置。 特别是,不需要特别配置一个Java Authentication and Authorization Service (JAAS)政策文件,也不需要把Spring Security放到server的classLoader下。

这些设计确保了发布时的最大轻便性,你可以简单把你的目标文件(JAR或WAR或EAR)从一个系统复制到另一个系统,它会立即正常工作。

5.2. 共享组件

让我们展示一些Spring Security中很重要的共享组件。 被成为"shared"的组件,是指它在框架中占有很重要的位置,框架离开它们就没法运行。 这些java类表达了维持系统的构建代码块,所以理解他们的位置是非常重要的,即使你不需要直接跟他们打交道。

5.2.1. SecurityContextHolder, SecurityContext 和 Authentication对象

最基础的对象就是`SecurityContextHolder`。 我们把当前应用程序的当前安全环境的细节存储到它里边了。 默认情况下,`SecurityContextHolder`使用`ThreadLocal`存储这些信息,这意味着,安全环境在同一个线程执行的方法一直是有效的,即使这个安全环境没有作为一个方法参数传递到那些方法里。 这种情况下使用`ThreadLocal`是非常安全的,只要记得在处理完当前主体的请求以后,把这个线程清除就行了。 当然,Spring Security自动帮你管理这一切了,你就不用担心什么了。

有些程序并不适合使用`ThreadLocal`,因为它们处理线程的特殊方法。 比如,swing客户端也许希望JVM里的usoyou线程都使用同一个安全环境。 为了这种情况,我们而已使用`SecurityContextHolder.MODE_GLOBAL`。 其他程序可能想让一个线程创建的线程也使用相同的安全主体。 这时可以使用`SecurityContextHolder.MODE_INHERITABLETHREADLOCAL`。 想要修改默认的`SecurityContextHolder.MODE_THREADLOCAL`模式,可以使用两种方法。 第一个是设置系统属性。 另一个是调用`SecurityContextHolder`的静态方法。 大多数程序不需要修改默认值,但是如果你需要做修改,先看一下`SecurityContextHolder`的JavaDoc中的详细信息。

我们把安全主体和系统交互的信息都保存在`SecurityContextHolder`中了。 Spring Security使用一个`Authentication`对应来表现这些信息。 虽然你通常不需要自己创建一个`Authentication`对象,很常见的,用户查询`Authentication`对象。 你可以使用下面的代码-在你程序中的任何位置-来获得已认证用户的名字,比如:

```
Object obj = SecurityContextHolder.getContext().getAuthentication().getPrincipal();

if (obj instanceof UserDetails) {
  String username = ((UserDetails)obj).getUsername();
} else {
  String username = obj.toString();
}
```

上面的代码介绍了一定数量的,有趣的,几个关键对象之间的相互关系。 首先,你会注意到`SecurityContextHolder` 和`Authentication`之间的中间对象。 这个`SecurityContextHolder.getContext()`方法会直接返回`SecurityContext`。

5.2.2. UserDetailsService

从上面的代码片段中还可以看出另一件事,就是你可以从`Authentication`对象中获得安全主体。 这个安全主体就是一个对象。 大多数情况下,可以强制转换成`UserDetails`对象。 `UserDetails`是一个Spring Security的核心接口。 它代表一个主体,是扩展的,而且是为特定程序服务的。 想一下`UserDetails`章节,在你自己的用户数据库和如何把Spring

Security需要的数据放到SecurityContextHolder里。 为了让你自己的用户数据库起作用,我们常常把UserDetails转换成你系统提供的类,这样你就可以直接调用业务相关的方法了(比如getEmail(), getEmployeeNumber()等等)。

现在,你可能想知道,我应该什么时候提供这个UserDetails对象呢? 我怎么做呢? 我想你说这个东西是声明式的,我不需要写任何代码,怎么办? 简单的回答是,这里有一个特殊的接口,叫UserDetailsService。 这个接口里的唯一一个方法,接收String类型的用户名参数,返回UserDetails。 大多数验证提供器使用Spring Security代理UserDetailsService,作为验证过程的一部分。 这个UserDetailsService用来建立Authentication对象,保存在SecurityContextHolder里。 好消息是我们提供了好几个UserDetailsService实现,其中一个使用内存里的map,另一个使用JDBC。 虽然,大多数用户倾向于写自己的,使用这些实现常常放到已有的数据访问对象(DAO)上,表示它们的雇员,客户或其他企业应用中的用户。 记住这个优势,无论你用什么UserDetailsService返回的数据都可以通过SecurityContextHolder获得,就像上面的代码片段讲的一样。

5.2.3. GrantedAuthority

除了主体,另一个Authentication提供的重要方法是getAuthorities()。 这个方法提供了GrantedAuthority对象数组。 毫无疑问,GrantedAuthority是赋予到主体的权限。 这些权限通常使用角色表示,比如ROLE_ADMINISTRATOR 或ROLE_HR_SUPERVISOR。 这些角色会在后面,对web验证,方法验证和领域对象验证进行配置。 Spring Security的其他部分用来拦截这些权限,期望他们被表现出现。 GrantedAuthority对象通常使用UserDetailsService读取的。

通常情况下,GrantedAuthority对象是应用程序范围下的授权。 它们不会特意分配给一个特定的领域对象。 因此,你不能设置一个GrantedAuthority,让它有权限展示编号54的Employee对象,因为如果有成千上网的这种授权,你会很快用光内存(或者,至少,导致程序花费大量时间去验证一个用户)。 当然,Spring Security被明确设计成处理常见的需求,但是你最好别因为这个目的使用项目领域模型安全功能。

最后,但不是最不重要的,优势你需要在HTTP请求之间共享SecurityContext. 其他时候,主体会在每个请求里重新验证,虽然大多数情况下它可以存储。 HttpSessionContextIntegrationFilter就是实现在HTTP请求之间存储SecurityContext的。 就像它的类名一样,HttpSession被用来保存这些信息。 你不应该因为安全的问题,直接与HttpSession打交道。 根本不存在这样做的理由-一直使用SecurityContextHolder作替代方式。

5.2.4. 小结

简单回顾一下,Spring Security主要是由一下几部分组成的:

- SecurityContextHolder,提供几种访问SecurityContext的方式。
- SecurityContext,保存Authentication信息,和请求对应的安全信息。
- HttpSessionContextIntegrationFilter,为了在不同请求使用,把SecurityContext保存到 HttpSession里。
- Authentication,展示Spring Security特定的主体。
- GrantedAuthority,反应,在应用程序范围你,赋予主体的权限。
- UserDetails,通过你的应用DAO,提供必要的信息,构建Authentication对象。
- UserDetailsService,创建一个 UserDetails,传递一个 String类型的用户名(或者证书ID或其他)。

现在,你应该对这种重复使用的组件有一些了解了。 让我们贴近看一下验证的过程。

5.3. 验证

就像这篇指南开头提到的那样,Spring Security可在很多不同的验证环境下使用。 虽然我们推荐人们使用Spring Security,不与已存在的容器管理认证系统整合,但它也是支持的-使用你自己的属性验证系统进行整合。 让我们先看看Spring Security完全依靠自己,管理web安全,这里会演示最复杂和最常见的情况。

讨论一个典型的web应用验证过程:

- 你访问首页,点击一个链接。
- 向服务器发送一个请求,服务器判断你是否在访问一个受保护的资源。
- 如果你还没有进行过认证,服务器发回一个响应,提示你必须进行认证。 响应可能是HTTP响应代码,或者是重新定向到一个特定的web页面。
- 依据验证机制,你的浏览器将重定向到特定的web页面,这样你可以添加表单, 或者浏览器使用其他方式校验你的身份(比如,一个基本校验对话框,cookie,或者X509证书,或者其他)。
- 浏览器会发回一个响应给服务器。 这将是HTTP POST包含你填写的表单内容,或者是HTTP头部,包含你的验证

信息。

- 下一步,服务器会判断当前的证书是否是有效的, 如果他们是有效的,下一步会执行。 如果他们是非法的,通常你的浏览器会再尝试一次(所以你返回的步骤二)。
- 你发送的原始请求,会导致重新尝试验证过程。 有希望的是,你会通过验证,得到猪狗的授权,访问被保护的资源。 如果你有足够的权限,请求会成功。否则,你会收到一个HTTP错误代码403,意思是访问被拒绝。

Spring Security使用鲜明的类负责上面提到的每个步骤。 主要的部分是(为了使用他们)是ExceptionTranslationFilter, 一个AuthenticationEntryPoint, 一个验证机制,一个AuthenticationProvider。

5.3.1. ExceptionTranslationFilter

ExceptionTranslationFilter是一个Spring Security过滤器,用来检测是否抛出了Spring Security异常。 这些异常会被AbstractSecurityInterceptor抛出,它主要用来提供验证服务。 我们会在下一节讨论AbstractSecurityInterceptor,但是现在,我们只需要知道,它是用来生成Java异常,和知道跟HTTP没啥关系,或者如何验证一个主体。 而ExceptionTranslationFilter提供这些服务,使用特点那个的响应,返回错误代码403(如果主题被验证了,但是权限不足-在上边的步骤七),或者启动一个AuthenticationEntryPoint(如果主体没有认证,然后我们需要进入步骤三)。

5.3.2. AuthenticationEntryPoint

AuthenticationEntryPoint对应中上面列表中的步骤三。 如你所想的,每个web应用程序都有默认的验证策略(好的,这可以在Spring Security里配置一切,但是让我们现在保持简单)。 每个主要验证系统会有它自己的AuthenticationEntryPoint实现, 会执行动作,如同步骤三里的描述一样。

在你的浏览器决定提交你的认证证书之后(使用HTTP表单发送或者是HTTP头),服务器部分需要有一些东西来“收集”这些验证信息。 现在我们到了上述的第六步。 在Spring Security里,我们需要一个特定的名字,来描述从用户代码(通常是浏览器)收集验证信息的功能,这个名字就是“验证机制”。 在从用户代码哪里收集了验证细节之后,一个"Authentication 请求"对象会被AuthenticationProvider建立。

5.3.3. AuthenticationProvider

Spring Security认证过程的最后一个角色是AuthenticationProvider。 非常简单,这是跟获得Authentication请求对象相关的,决定它是否是有效的。 这个供应器或者抛出一个异常,或者返回一个完整的Authentication对象。 还记得我们的好朋友UserDetails和UserDetailsService吗? 如果不记得,回头看看前面的章节,刷新你的记忆。 大多数AuthenticationProvider都会要求UserDetailsService提供一个UserDetails对象。 像上面提到的那样,大多数程序会提供他们自己的UserDetailsService虽然一些可以使用Spring Security提供的JDBC和内存实现。 由此产生的UserDetails对象-特别是UserDetails中包含的GrantedAuthority[]-将被用来组装Authentication对象。

在验证机制重新获得了组装好的Authentication对象以后,他会认为请求有效,把Authentication放到SecurityContextHolder里,然后导致原始请求重审(第七步)。 另一方面,如果AuthenticationProvider驳回了请求,验证机制会让用户代码重试(第二步)。

5.3.4. 直接设置SecurityContextHolder的内容

虽然这表述了一个典型的验证流程,但是好消息是Spring Security不在意你如何把一个Authentication放到SecurityContextHolder里的。 唯一关键的需求是SecurityContextHolder包含Authentication,用来表现一个主体,在AbstractSecurityInterceptor之前验证请求的。

你可以(很多用户都这样做)写一个自己的过滤器或MVC控制器来提供验证系统的交互,这些都不是基于Spring Security的。 比如,你也许使用容器管理验证,从ThreadLocal或JNDI里获得当前用户信息。 或者,你的公司可能有一个遗留系统,它是一个企业标准,你不能控制它。 这种情况下,很容易让Spring Security工作,也能提供验证能力。 你所需要的就是写一个过滤器(或等价物)从指定位置读取第三方用户信息,把它放到SecurityContextHolder里。 实现这些很简单,这种整合是完全被支持的方法。

5.4. 安全对象

Spring Security使用词组"secure object"来指代任何需要保护的对象(就像一个认证决定)。 最常见例子就是方法调用和web请求。

5.4.1. 安全和AOP建议

如果你熟悉AOP的话,就会知道有几种不同的拦截方式:之前,之后,抛异常和环绕。 其中环绕是非常有用的,因为advisor可以决定是否执行这个方法,是否修改返回的结果,是否抛出异常。 Spring Security为方法调用提供了一个环绕advice,就像web请求一样。 我们使用Spring的标准AOP支持制作了一个处理方法调用的环绕advice,我们使用标准filter建立了对web请求的环绕advice。

对那些不熟悉AOP的人,需要理解的关键问题是Spring Security可以帮助你保护方法的调用,就像保护web请求一样。 大多数人对保护服务层里的安全方法非常按兴趣。 这是因为在目前这一代J2EE程序里,服务器放了更多业务相关的逻辑(需要澄清,作者不建议这种设计方法,作为替代的,而是应该使用DTO,集会,门面和透明持久模式压缩领域对象,但是使用贫血领域对象是当前的主流思路,所以我们还是会在这里讨论它)。 如果你只是需要保护服务层的方法调用,Spring标准AOP平台(一般被称作AOP联盟)就够了。 如果你想直接保护领域对象,你会发现AspectJ非常值得考虑。

可以选择使用AspectJ还是Spring AOP处理方法验证,或者你可以选择使用filter处理web请求验证。 你可以不选,选择其中一个,选择两个,或者三个都选。 主流的应用是处理一些web请求验证,再结合一些在服务层里的Spring AOP方法调用验证。

5.4.2. AbstractSecurityInterceptor

Spring Security支持的每个安全对象类型都有它自己的类型,它们都是`AbstractSecurityInterceptor`的子类。 很重要的是,如果主体是已经通过了验证,在`AbstractSecurityInterceptor`被调用的时候,`SecurityContextHolder`将会包含一个有效的`Authentication`。

`AbstractSecurityInterceptor`提供了一套一致的工作流程,来处理对安全对象的请求,通常是:

- 查找当前请求里分配的"配置属性"。
- 把安全对象,当前的`Authentication`和配置属性,提交给`AccessDecisionManager`,来进行以此认证决定。
- 有可能在调用的过程中,对 `Authentication` 进行修改。
- 允许安全对象进行处理(假设访问被允许了)。
- 在调用返回的时候执行配置的`AfterInvocationManager`。

5.4.2.1. 配置属性是什么?

一个"配置属性"可以看做是一个字符串,它对于`AbstractSecurityInterceptor`使用的类是有特殊含义的。它们可能是简单的角色名称或拥有更复杂的含义,这就与`AccessDecisionManager`实现的先进程度有关了。
`AbstractSecurityInterceptor`和配置在一起的`ObjectDefinitionSource`用来为一个安全对象搜索属性。 通常这个属性对用户是不可见的。 配置属性将以注解的方式设置在受保护方法上,或者作为受保护URL的访问属性(使用命名空间里的`<intercept-url>`语法)。

5.4.2.2. RunAsManager

假设`AccessDecisionManager`决定允许执行这个请求,`AbstractSecurityInterceptor`会正常执行这个请求。 话虽如此,罕见情况下,用户可能需要把`SecurityContext`的`Authentication`换成另一个`Authentication`,通过访问`RunAsManager`。 这也许在,有原因,不常见的情况下有用,比如,服务层方法需要调用远程系统,表现不同的身份。 因为Spring Security自动传播安全身份,从一个服务器到另一个(假设你使用了配置好的RMI或者HttpInvoker远程调用协议客户端),就可以用到它了。

5.4.2.3. AfterInvocationManager

按照下面安全对象执行和返回的方式-可能意味着完全的方法调用或过滤器链的执行。 这种状态下`AbstractSecurityInterceptor`对有可能修改返回对象感兴趣。 你可能想让它发生,因为验证决定不能“关于如何在”一个安全对象调用。 高可插拔性,`AbstractSecurityInterceptor`通过控制`AfterInvocationManager`,实际上在需要的时候,修改对象。 这里类实际上可能替换对象,或者抛出异常,或者什么也不做。

`AbstractSecurityInterceptor`和相关对象展示在 Figure 5.1, “关键"secure object"模型”中。

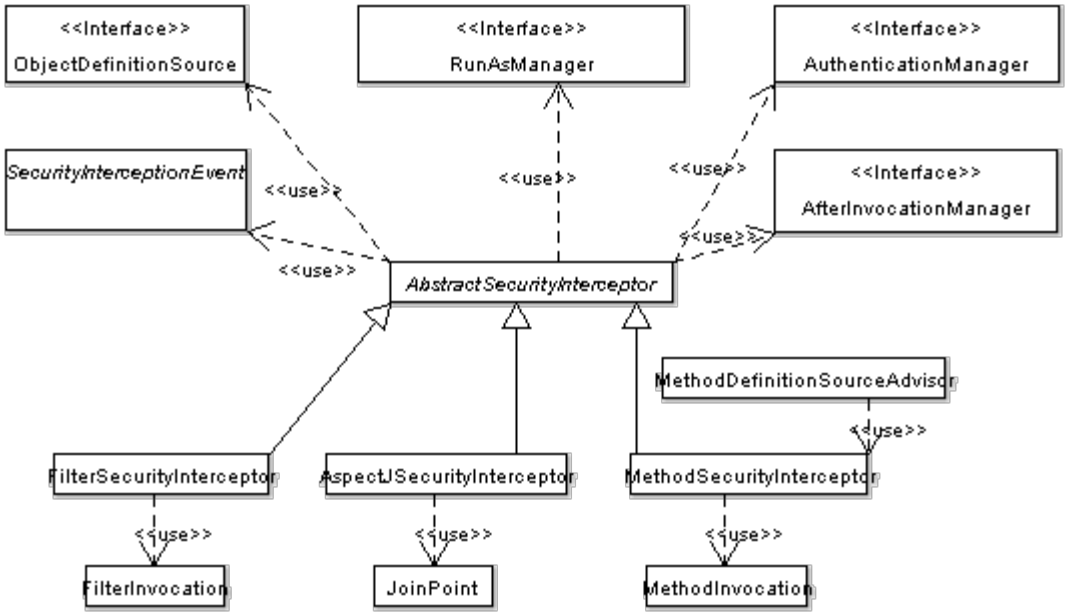


Figure 5.1. 关键**"secure object"**模型

5.4.2.4. 扩展安全对象模型

只有开发者才会关心使用全心的方法,进行拦截和验证请求,将直接使用安全方法。 比如,可能新建一个安全方法,控制对消息系统的权限。 安全需要的任何事情,也可以提供一种拦截的方法(好像AOP的环绕advice语法那样)有可能在安全对象里处理。 这样说的话,大多数Spring应用简单拥有三种当前支持的安全类型(AOP联盟的`MethodInvocation`,AspectJ `JoinPoint`和web请求`FilterInterceptor`)完全透明的。



支持的基础设施

这章里讨论一些Spring Security中用到的基础设施。 如果这个功能跟安全没有直接关系,但是还是包含在Spring Security中了,我们就在这章里讨论它。

6.1. 国际化

Spring Security支持异常信息的国际化,最终用户会很喜欢这点。 如果你的程序是为英语用户设计的,你不需要做任何事,因为默认情况下所有的Spring Security消息都是英文的。 如果你需要支持其他语言,你所需要做的就是看看这节内容。

所有的异常信息都是可以本地化的,包括验证失败和访问被拒绝(验证失败)信息。 开发者和系统发布者关注的异常和日志信息(包括不正确的属性,接口联系分隔,使用错误的构造器,开始验证册书,日志调试等级)等等,不能进行本地化,而是使用英文直接硬编码到了Spring Security代码里。

打开`spring-security-core-xx.jar`,你可以找到`org.springframework.security`包,里面包含了一个`messages.properties`文件。 你的`ApplicationContext`应用应该引用这个文件,因为Spring Security实现了Spring的`MessageSourceAware`接口,希望这些消息会在application context启动的时候注入到程序中。 通常,你需要做的就是,在你的application context 里注册一个bean,来引用这些信息,比如像下面这样:

```
<bean id="messageSource"
class="org.springframework.context.support.ReloadableResourceBundleMessageSource">
  <property name="basename" value="org/springframework/security/messages"/>
</bean>
```

这个`messages.properties`使用了标准的资源束命名方式,为Spring Security提供了默认的语言支持。 默认的语言是英文。 如果你没有注册消息源,Spring Security依然可以正常工作,但是使用的是硬编码的英文信息版本。

如果你想自定义`messages.properties`文件,或支持其他语言,你应该复制这个文件,改成对应的文件名,并把它注册到上面的bean定义中。 文件里的关键信息并不多,本地化应该不是一个很大的任务。 如果你对这个文件进行了本地化,请考虑把你的工作与社区分享,记录一个JIRA任务,把你的本地化版本`messages.properties`以附件形式发送上来。

围绕本地化进行讨论,就要提到Spring里的`ThreadLocal`,`org.springframework.context.i18n.LocaleContextHolder`。 你应该通过`LocaleContextHolder`,来表现每个用户想要的`Locale`. Spring Security尝试从信息源中定位信息,使用从`ThreadLocal`里中获得的`Locale`。 请参考Spring文档获得使用`LocaleContextHolder`的更多信息,可以使用一些帮助类进行自动设置(比如`AcceptHeaderLocaleResolver`, `CookieLocaleResolver`, `FixedLocaleResolver`, `SessionLocaleResolver`等等)。

6.2. 过滤器

Spring Security用到了很多过滤器,参考指南的后续部分会一一提到。 如果你使用了命名空间配置,你就不用经常去明确指定过滤器bean。 有几种可能情况,你希望对安全过滤器链进行完全控制,或许因为你使用的功能没法使用命名空间进行支持,或者你使用了自己自定义版本的类。

这种情况下,你可以选择向你的web应用成立里添加哪些过滤器,这里你可以使用Spring的`DelegatingFilterProxy`或`FilterChainProxy`。我们会在下面介绍它们两个。

在使用`DelegatingFilterProxy`的时候,你会看到web.xml里这样的内容:

```
    <filter>
        <filter-name>myFilter</filter-name>
        <filter-class>org.springframework.web.filter.DelegatingFilterProxy</filter-
class>
    </filter>

    <filter-mapping>
```

```
    <filter-name>myFilter</filter-name>
    <url-pattern>/*</url-pattern>
  </filter-mapping>
```

注意这个过滤器其实是一个DelegatingFilterProxy,这个过滤器里没有实现过滤器的任何逻辑。 DelegatingFilterProxy做的事情是代理Filter的方法,从application context里获得bean。 这让bean可以获得spring web application context的生命周期支持,使配置较为轻便。 bean必须实现javax.servlet.Filter接口,它必须和filter-name里定义的名称是一样的。

在生命周期的问题上,要考虑在IoC容器里而不是在servlet容器里管理Filter。 具体来说,到底是哪个容器应该调用Filter的“启动”与“关闭”方法。 需要指出的是Filter的初始化和销毁很容易受servlet容器的影响,如果一个Filter依赖于较早初始化Filter的配置,那么可能会引发一些问题。 Spring IoC容器,从另一方面讲,拥有更强大的生命周期/IoC接口(比如InitializingBean, DisposableBean, BeanNameAware, ApplicationContextAware和很多其他的),拥有更容易理解的接口协议,可预见的方法调用顺序,支持自动绑定,更可以选择不用实现Spring的接口(比如通过Spring XML中的destroy-method属性)。 介于这些原因,只要有可能的话,我们推荐使用Spring生命周期服务,代替servlet容器的生命周期。 可以参考DelegatingFilterProxy的Javadoc获得更多信息。

最好别用DelegatingFilterProxy,我们强烈推荐你使用FilterChainProxy代替它。 虽然DelegatingFilterProxy是一个非常有用的类,问题是在需要使用几个过滤器的时候,需要在web.xml中定义<filter>和<filter-mapping>入口的代码数量太多了。 为了解决这个问题,Spring Security提供了一个FilterChainProxy类。 它绑定了一个DelegatingFilterProxy(好像上面的例子那样),但是使用的类是org.springframework.security.util.FilterChainProxy。 过滤器链要声明在application context里,使用下面的代码:

```
<bean id="filterChainProxy"
class="org.springframework.security.util.FilterChainProxy">
  <sec:filter-chain-map path-type="ant">
     <sec:filter-chain pattern="/webServices/**"

filters="httpSessionContextIntegrationFilterWithASCFalse,basicProcessingFilter,exceptionTr

     <sec:filter-chain pattern="/**"

filters="httpSessionContextIntegrationFilterWithASCTrue,authenticationProcessingFilter,exc

  </sec:filter-chain-map>
</bean>
```

你可能注意到FilterSecurityInterceptor声明的不同方式。 它同时支持正则表达式和ant路径,并且只使用第一个出现的匹配URI。 在运行阶段FilterChainProxy会定位当前web请求匹配的第一个URI模式,由filters属性指定的过滤器bean列表将开始处理请求。 过滤器会按照定义的顺序依次执行,所以你可以对处理特定URL的过滤器链进行完全的控制。

你可能注意到了,我们在过滤器链里声明了两个HttpSessionContextIntegrationFilter(ASC是allowSessionCreation的简写,是HttpSessionContextIntegrationFilter的一个属性)。 因为web服务从来不会在请求里带上jsessionid,为每个用户代理都创建一个HttpSession完全是一种浪费。 如果你需要构建一个高等级最高可扩展性的系统,我们推荐你使用上面的配置方法。 对于小一点儿的项目,使用一个HttpSessionContextIntegrationFilter(让它的allowSessionCreation默认为true)就足够了。

在有关声明周期的问题上,如果这些方法被FilterChainProxy自己调用,FilterChainProxy会始终根据下一层的Filter代理init(FilterConfig)和destroy()方法。 这时,FilterChainProxy会保证初始化和销毁操作只会在Filter上调用一次,而不管它们被FilterInvocationDefinitionSource声明了多少次。 你可以完全控制是否调用这些方法,通过代理DelegatingFilterProxy的targetFilterLifecycle初始化参数,。 像上面讨论的那样,默认的servlet容器生命周期调用不会被DelegatingFilterProxy代理。

在使用命名空间配置的时候,你可以通过同样的方式使用filters = "none"属性,来建立FilterChainProxy。 这将从安全过滤器链中完全忽略请求的模式。 注意,任何匹配这个路径的请求,不会要求认证,不会使用验证服务,可以自由的访问。

web.xml <filter-mapping>

定义在 里的过滤器的顺序是非常重要的。 不论你实际使用的是哪个过滤器, 的顺序应该像下面这样:

- ChannelProcessingFilter,因为它可能需要重定向到其他协议。
- ConcurrentSessionFilter,因为它不使用SecurityContextHolder功能,但是需要更新 SessionRegistry 来从主体中放映正在进行的请求。
- HttpSessionContextIntegrationFilter,这样 SecurityContext可以在web请求的开始阶段通过SecurityContextHolder建立,然后 SecurityContext的任何修改都会在web请求结束的时候(为下一个web请求做准备)复制到 HttpSession中。
- 验证执行机制 - AuthenticationProcessingFilter, CasProcessingFilter, BasicProcessingFilter, HttpRequestIntegrationFilter, JbossIntegrationFilter 等等 - 这样 SecurityContextHolder 可以被修改,并包含一个合法的 Authentication 请求标志。
- SecurityContextHolderAwareRequestFilter,如果,你使用它,把一个Spring Security提醒HttpServletRequestWrapper安装到你的servlet容器里。
- RememberMeProcessingFilter,这样如果之前的验证执行机制没有更新 SecurityContextHolder,这个请求提供了一个可以使用的remember-me服务的cookie,一个对应的已保存的 Authentication对象会被创建出来。
- AnonymousProcessingFilter,这样如果之前的验证执行机制没有更新 SecurityContextHolder,会创建一个匿名 Authentication对象。
- ExceptionTranslationFilter,用来捕捉 Spring Security异常,这样,可能返回一个HTTP错误响应,或者执行一个对应的 AuthenticationEntryPoint。
- FilterSecurityInterceptor,保护web URI。

上面所有的过滤器,都使用了 DelegatingFilterProxy 或 FilterChainProxy。推荐使用单独的 DelegatingFilterProxy 为每个程序代理一个单独的 FilterChainProxy,通过这个 FilterChainProxy 定义Spring Security的所有过滤器。

如果你使用了SiteMesh,一定要确保Spring Security过滤器在SiteMesh的过滤器之前被调用。 这才可以保证为SiteMesh渲染器使用的 SecurityContextHolder 先被组装起来。

6.3. 标签库

Spring Security捆绑了很多JSP标签库,提供了各种不同的服务。

6.3.1. 配置

所有的taglib类都包含在核心spring-security-xx.jar文件里, security.tld文件放在JAR的META-INF目录下。 这意味着,对于JSP 1.2+的web容器,你可以直接把JAR放到WAR的WEB-INF/lib目录下,然后就可以用了。 如果你使用的是JSP 1.1 容器,你需要在你的web.xml文件里声明JSP taglib,还要把security.tld放到WEB-INF/lib目录下。 把下面的片段添加到web.xml里:

```
<taglib>
  <taglib-uri>http://www.springframework.org/security/tags</taglib-uri>
  <taglib-location>/WEB-INF/security.tld</taglib-location>
</taglib>
```

6.3.2. 使用

现在你已经配置好了标签库,可以转到单独的参考指南章节,阅读如何使用他们的详细信息。 注意当使用标签时,你应该在你的JSP里包含taglib引用:

```
  <%@ taglib prefix='security' uri='http://www.springframework.org/security/tags' %>
```

Chapter 5. 技术概述 | Sponsored by SpringSource | Chapter 7. 信道安全



信道安全

7.1. 总述

为了在你的程序中协调认证与验证的要求,Spring Security也可以让未认证的web请求携带一些属性。 这些属性可能包括特定的传输类型,拥有特定的 `HttpSession` 属性设置,或其他。 最常见的需求是,让你的web请求使用特定的传输协议发送,比如HTTPS。

威胁传输安全的一个重要因素是会话劫持。你的web容器通过用户代理发送cookie或URL重写,使用`jsessionid`,来管理`HttpSession`。 如果`jsessionid`使用HTTP发送,会话标识符很可能在用户通过验证过后,被其他人拦截,再去模拟用户访问。 这是因为大多数web容器,为给定的用户维护相同的session标识符,即使他们从HTTP转换到HTTPS页面也不会改变。

如果会话劫持在你的特定程序里被认为是一个特别值得关注的威胁,唯一的解决方式就是在每个请求里都使用HTTPS。这意味着`jsessionid`再也不会通过不安全的通道。 你会需要确认你的`web.xml`定义的`<welcome-file>`指向HTTPS位置。 Spring Security在后面提供一个方法来帮助实现这些功能。

7.2. 配置

信道安全已在安全命名空间获得支持, 使用`<intercept-url>`元素里的`requires-channel`属性,就能很简单实现(也推荐这样做)。

为了明确配置信道安全你需要在你的application context中定义下面的过滤器:

```
<bean id="channelProcessingFilter"
class="org.springframework.security.securechannel.ChannelProcessingFilter">
  <property name="channelDecisionManager" ref="channelDecisionManager"/>
  <property name="filterInvocationDefinitionSource">
    <security:filter-invocation-definition-source path-type="regex">
      <security:intercept-url pattern="\A/secure/.*\Z"
access="REQUIRES_SECURE_CHANNEL"/>
      <security:intercept-url pattern="\A/acegilogin.jsp.*\Z"
access="REQUIRES_SECURE_CHANNEL"/>
      <security:intercept-url pattern="\A/j_spring_security_check.*\Z"
access="REQUIRES_SECURE_CHANNEL"/>
      <security:intercept-url pattern="\A/.*\Z" access="ANY_CHANNEL"/>
    </security:filter-invocation-definition-source>
  </property>
</bean>

<bean id="channelDecisionManager"
class="org.springframework.security.securechannel.ChannelDecisionManagerImpl">
  <property name="channelProcessors">
    <list>
    <ref bean="secureChannelProcessor"/>
    <ref bean="insecureChannelProcessor"/>
    </list>
  </property>
</bean>

<bean id="secureChannelProcessor"
class="org.springframework.security.securechannel.SecureChannelProcessor"/>
<bean id="insecureChannelProcessor"
class="org.springframework.security.securechannel.InsecureChannelProcessor"/>
```

和 `FilterSecurityInterceptor` 一样,`ChannelProcessingFilter`也支持Apache Ant样式的路径。

这个 `ChannelProcessingFilter` 通过过滤所有的web请求,决定应用配置属性。 它将代理`ChannelDecisionManager`。

默认实现ChannelDecisionManagerImpl,应该涵盖了大多数情况。 它直接调用配置的ChannelProcessor实例。 ANY_CHANNEL属性可以用来覆盖这种行为,忽略指定的URL。 否则,一个ChannelProcessor会重新检查请求,如果请求不符合(比如,它发送数据使用了错误的传输协议),就会执行一个重定向,抛出一个异常,或执行需要的任何一个操作。

Spring Security里包含两个具体ChannelProcessor实现: SecureChannelProcessor 确保对应配置REQUIRES_SECURE_CHANNEL属性的请求使用HTTPS发送数据。 InsecureChannelProcessor 确保对应配置REQUIRES_INSECURE_CHANNEL属性的请求使用HTTP发送数据。 如果没有使用对应的传输协议,这两个实现都代理ChannelEntryPoint。 包含在Spring Security里的这两个ChannelEntryPoint实现,只是把请求简单重定向到合适的HTTP或HTTPS。 合适的默认分配给ChannelProcessor实现,为它们相应的配置属性关键字和它们代表的ChannelProcessor,不过,你可以使用application context重写它们。

注意,重定向使用的是绝对地址(比如http://www.company.com:8080/app/page),不是相对地址(比如/app/page)。 在测试过程中,发现了IE6 sp1的一个bug,在重定向时,如果只改变了端口,它就不能响应成功。 因此,绝对URL就是用来在PortResolverImpl中,解决这个bug检测逻辑,这是默认情况下被Spring Security bean使用的。 请参考PortResolverImpl的JavaDocs获得更多信息。

你应该注意,在登录过程中,推荐使用信道安全,保持用户名和密码的安全。 如果你没有决定在基于表单的登录里使用ChannelProcessingFilter。 请确认你的登录页,设置了REQUIRES_SECURE_CHANNEL,并让AuthenticationProcessingFilterEntryPoint.forceHttps的值是true。

7.3. 总结

一旦配置好,使用信道安全过滤器就非常简单了。 简单请求页面,不限制协议(比如HTTP或HTTPS)或端口(比如80,8080,443,8443等等)。 很明显,我们还需要配置初始请求(可能通过web.xml的<welcome-file>或一个众所周知的主页URL),但是一旦这个发生了,过滤器就会按照你的application context定义的那样执行重定向。

你也可以添加自己的ChannelProcessor实现ChannelDecisionManagerImpl。 比如,你可能在使用"输入图片中的内容"检测真人用户的过程中,要设置一个HttpSession属性。 你的ChannelProcessor会响应REQUIRES_HUMAN_USER配置属性,然后重定向到合适的入口点,如果HttpSession属性还没有设置,启动真人用户的验证过程。

为了决定是否进行一个安全检测,依靠ChannelProcessor或AccessDecisionVoter,前一个被设计成处理未认证的请求,而后一个被设计成处理已认证的请求。 因此,后一个可以访问验证主体的被授权权限。 另外,ChannelProcessor检测到问题,就会进行HTTP/HTTPS的重定向,来满足它的需求。 AccessDecisionVoter检测到一个问题,会导致一个AccessDeniedException(取决于AccessDecisionManager)。



Part III. 认证

我们已经在技术概述章中介绍了Spring Security的认证架构。 在参考指南的这部分,我们将单独讨论认证机制和他们对应的`AuthenticationProvider`。 我们也会看看如何更普遍的配置认证,包括如何你有多个认证入口,需要链接在一起。

包括一些例外情况,我们会讨论Spring Security bean配置的全部细节,而不是像命名空间语法那种简化形式。 你应该重新看一下使用命名空间进行配置的介绍,如果它们符合你的需求还要看一下他们的选项。 你越是不断的使用框架,就越需要自定义内部行为,你可能会希望跟多了解,单独服务是如何实现的,哪个类需要找到进行扩展,等等。 这部分更集中于提供这类信息。 我们建议你从Javadoc以及它自己的源代码获得信息[2]。

---

[2] 从项目的网站可以找到Javadoc API和可以浏览的源代码交叉引用。



通用认证服务

8.1. 机制,供应者和入口

如果你使用Spring Security提供的认证方式,你通常需要配置一个web过滤器,AuthenticationProvider 和AuthenticationEntryPoint。 在这章里,我们将要研究一个应用例子,它要支持基于表单的认证(比如,把一个非常好的HTML页面展现给用户,让他们进行登录)和基本认证(比如一个web服务或类似可访问的被保护资源)。

在web.xml里,这个程序需要单独的Spring Security过滤器来使用FilterChainProxy。 几乎所有Spring Security程序都有这样一个入口,看起来像这样:

```
<filter>
    <filter-name>filterChainProxy</filter-name>
    <filter-class>org.springframework.web.filter.DelegatingFilterProxy</filter-
class>
</filter>

<filter-mapping>
  <filter-name>filterChainProxy</filter-name>
  <url-pattern>/*</url-pattern>
</filter-mapping>
```

上面的声明会让所有web请求,都通过叫做FilterChainProxy的bean,通常它是Spring Security的FilterChainProxy的一个实例。 就像在这个参考指南的过滤器章节解释的那样,FilterChainProxy是一个通用类,让web请求可以基于URL模式通过不同的过滤器。 那些被委派的过滤器都管理在application context中,这样它们可以从依赖注入获得更大的益处。 让我们看一下FilterChainProxy bean定义,在application context的形式就像这样:

```
<bean id="filterChainProxy"
        class="org.springframework.security.util.FilterChainProxy">
  <security:filter-chain-map path-type="ant">
    <security:filter-chain pattern="/**"
filters="httpSessionContextIntegrationFilter,logoutFilter,authenticationProcessingFilter,b

  </security:filter-chain-map>
</bean>
```

安全命名空间里的filter-chain-map的语法,让你可以使用filter-chain子元素序列,为过滤器链定义一个URL映射。它们使用pattern属性定义了一系列 URL,使用filters属性定义了过滤器链。 这里需要重点注意的是,过滤器系列会按照定义的顺序执行 - 而且每个过滤器,都与在application context中定义的其他bean的id相对应。 所以,我们看到一些特定的bean出现在application context里,他们可能叫作httpSessionContextIntegrationFilter,logoutFilter等等。 过滤器的顺序,在参考文档的过滤器章节讨论 - 不过上面的例子没有问题。

在我们的例子中,我们使用了AuthenticationProcessingFilter,BasicProcessingFilter。 他们是,响应基于表单认证和基本HTTP header认证的“认证机制”(我们在参考文档的前面部分讨论过认证机制的角色)。 如果你没有使用表单或基本认证,就不需要定义这几个bean。 你应该使用,你想要的认证环境需要的过滤器,比如DigestProcessingFilter或CasProcessingFilter。 参考有关这部分参考文档的单独章节,学习如何配置每个认证机制。

让我们回忆一下,HttpSessionContextIntegrationFilter保存了在HTTP session调用中的SecurityContext内容。这意味着认证机制只在主体初始化尝试认证的时候使用一次。 剩下的时间里,认证机制就待在那里,静静的让请求通过,进入过滤器链的下一个过滤器里。 这是一个很现实的需求,认证过程很少需要在每个调用的时候起作用(BASIC认证是一个例外),但是,在初始化认证完成之后,如果主体帐号发生了取消或禁用或其他改变(比如,增加或减少授权GrantedAuthority[]),怎么办? 让我们看看现在是如何处理的。

安全对象的主要认证提供器在前面介绍过,是AbstractSecurityInterceptor。 这个类需要访问AuthenticationManager。 它也可以配置成,一个Authentication对象在每次安全对象调用的时候,是否需要重新认证。 默认情况下,如果Authentication.isAuthenticated()返回true,它就只从SecurityContextHolder中取得已经认证的Authentication。 这样执行很有效率,但是没办法处理即时认证的有效。 对这种情况,你就要把AbstractSecurityInterceptor.alwaysReauthenticate属性设置成true。

你可能会问自己,“AuthenticationManager是什么? ”。 我们之前还没有讲过它,但是我们讨论过AuthenticationProvider的概念。 非常简单,一个AuthenticationManager用来从一个AuthenticationProvider链传递请求,获得响应。 这跟我们以前讨论到的过滤器链有点儿相似,虽然这里有很多区别。 在Spring Security里,这里只有一个AuthenticationManager实现,所以让我们看看,对于这章中的例子,它是如何配置的:

```
<bean id="authenticationManager"
        class="org.springframework.security.providers.ProviderManager">
<property name="providers">
<list>
  <ref local="daoAuthenticationProvider"/>
  <ref local="anonymousAuthenticationProvider"/>
  <ref local="rememberMeAuthenticationProvider"/>
</list>
</property>
</bean>
```

还要提到一点,你的认证机制(通常是过滤器)也要注入AuthenticationManager的引用。 所以AbstractSecurityInterceptor作为一个认证机制,也要使用上面的ProviderManager轮询一系列的AuthenticationProvider。

在我们例子里有三个提供器。 他们按照顺序进行排列(这里使用的是List而不是Set),每个提供器都尝试进行认证,或跳过验证简单返回null。 如果所有实现都返回null,ProviderManager就会抛出一个对应的异常。 如果你对链装提供器感兴趣,请参考ProviderManager的JavaDocs。

这些提供器有时在不同认证机制之间是可互换的,其他时候,它们依赖于特定的认证机制。 比如DaoAuthenticationProvider只需要基于字符串的用户名和密码。 好多认证机制会产生基于字符串的用户名和密码的集合,包括(但不限于)表单和基本验证。 同样的,一些认证机制创建只能被对应AuthenticationProvider使用的认证请求对象。 比如,一一对应的JA-SIG CAS,他使用服务票据体型,只能被CasAuthenticationProvider认证。 另一个一一对应的例子是LDAP认证机制,它只会被LdapAuthenticationProvider处理。 每个类特定关系的细节在JavaDocs里,这个参考指南的认证特定方式的章节里都有详细说明。 你不需要特别注意这些实现细节,因为如果你忘记注册对应的提供器,你就会在尝试认证的时候收到一个ProviderNotFoundException。

在正确配置完FilterChainProxy里的认证机制后,也确定对应的AuthenticationProvider注册到了ProviderManager中,你最后一步是配置一个AuthenticationEntryPoint。 回忆一下我们以前讨论过的ExceptionTranslationFilter的角色,它是在认证开始的使用,用来在基于HTTP请求,返回一个HTTP头或HTTP重定向。 继续我们以前的例子:

```
<bean id="exceptionTranslationFilter"
        class="org.springframework.security.ui.ExceptionTranslationFilter">
  <property name="authenticationEntryPoint"
ref="authenticationProcessingFilterEntryPoint"/>
  <property name="accessDeniedHandler">
    <bean class="org.springframework.security.ui.AccessDeniedHandlerImpl">
      <property name="errorPage" value="/accessDenied.jsp"/>
    </bean>
  </property>
</bean>

<bean id="authenticationProcessingFilterEntryPoint"

class="org.springframework.security.ui.webapp.AuthenticationProcessingFilterEntryPoint">

  <property name="loginFormUrl" value="/login.jsp"/>
  <property name="forceHttps"><< value="false"/>
</bean>
```

注意ExceptionTranslationFilter需要两个协作者。 第一个是AccessDeniedHandlerImpl,使用RequestDispatcher转向到显示特定访问拒绝错误页面。 我们使用请求转向,这样SecurityContextHolder还能包含

着主体的信息,这对呈现给用户是很有用的(老版本中,我们让servlet容器处理403错误信息,这会丢失很多有用的内容信息)。 AccessDeniedHandlerImpl也会把HTTP header设置成403,这是一个正规的错误代码,描述拒绝访问。 至于AuthentionEntryPoint,这里我们设置当一个未认证的主体尝试执行一个安全操作时,会执行什么动作。 因为在我们的例子里,我们将使用基于表单的认证,我们指定AuthenticationProcessinFilterEntryPoint和登录用页面的URL。 你的程序通常只会有一个入口点,大多数的认证方法定义它们自己的AuthenticationEntryPoint。 每个认证方式对应哪个入口点,在参考指南的认证特定方式章节里讨论。

8.2. UserDetails 和相关类型

就像参考指南第一部分提到的,大多数认证提供器使用UserDetails 和 UserDetailsService 接口。 后一个接口只有一个单独的方法:

```
  UserDetails loadUserByUsername(String username) throws UsernameNotFoundException,
DataAccessException;
```

返回的UserDetails是一个接口,提供了getter方法,保障非空的基本认证信息,比如用户名,密码,授予的权限,和这个用户是否被禁用了。 大多数认证提供器会使用UserDetailsService,即使用户名和密码其实没有在认证过程中用到。 通常,这种供应器会使用返回的UserDetails对象,只是使用其中的GrantedAuthority[]信息,因为一些系统(像LDAP或X509或CAS等等)已经对整数的有效性进行过验证。

一个由Spring Security提供的UserDetails单独的具体实现,是User类。 Spring Security用户将需要决定什么时候编写他们的UserDetailsService,返回具体的UserDetails类。 大多数情况下User会直接使用,或被继承,虽然特定的环境(比如ORM)可能需要用户根据脚手架编写他们自己的UserDetails。 这并不是不常见的情况,用户应该毫不犹豫的返回他们正常的领域对象,展示系统的一个用户。 尤其是通常给UserDetails提供附加的主体相关的属性(比如他们的电话号码和邮件地址),这样他们可以很容易在web视图里使用。

UserDetailsService是非常容易实现的,用户应该很容易使用他们自己选择持久化策略,获得认证信息。 这里,Spring Security确实包含了很多有用的实现,我们会在下面看到。

8.2.1. 内存里认证

虽然很容易创建一个自定义的UserDetailsService实现,从选择的持久化引擎里获得确切信息,不过很多应用不需要搞得这么复杂。 特别是在你进行一个快速原型或者仅仅开始集成Spring Security的时候,当你不是真的需要在配置数据库或写UserDetailsService实现上面花费时间的时候。 为了这种情况,一个简单的选择是使用安全命名空间里的user-service元素,:

```
    <user-service id="userDetailsService">
      <user name="jimi" password="jimispassword" authorities="ROLE_USER, ROLE_ADMIN"
/>
      <user name="bob" password="bobspassword" authorities="ROLE_USER" />
    </user-service>
```

它也支持外部属性文件:

```
    <user-service id="userDetailsService" properties="users.properties"/>
```

属性文件,应该包含下面格式的内容

```
  username=password,grantedAuthority[,grantedAuthority][,enabled|disabled]
```

比如

```
  jimi=jimispassword,ROLE_USER,ROLE_ADMIN,enabled
  bob=bobspassword,ROLE_USER,enabled
```

8.2.2. JDBC认证

Spring Security也包含一个UserDetailsService可以从JDBC数据源里获得认证信息。 内部使用了Spring JDBC,避免了仅仅为了保存用户信息而实现ORM完全功能的复杂性。 如果你的程序使用了ORM工具,你估计会喜欢自己写一个UserDetailsService来重用你已经写好的映射文件。 回到JdbcDaoImpl,一个配置的例子如下所示:

```
<bean id="dataSource"
class="org.springframework.jdbc.datasource.DriverManagerDataSource">
  <property name="driverClassName" value="org.hsqldb.jdbcDriver"/>
  <property name="url" value="jdbc:hsqldb:hsql://localhost:9001"/>
  <property name="username" value="sa"/>
  <property name="password" value=""/>
</bean>

<bean id="userDetailsService"
class="org.springframework.security.userdetails.jdbc.JdbcDaoImpl">
  <property name="dataSource" ref="dataSource"/>
</bean>
```

通过修改上面的DriverManagerDataSource,你可使用不同的关系型数据库管理系统。 你也可以使用通常的Spring选项,从JNDI获得的公共数据源。

8.2.2.1. 默认用户数据库表结构

不论什么什么数据库,或者,不管DataSource是如何获得的,必须用到一个标准的表结构,表结构定义的内容就在这里。一个HSQL数据库的DDL内容应该像这样:

```
  CREATE TABLE users (
  username VARCHAR(50) NOT NULL PRIMARY KEY,
  password VARCHAR(50) NOT NULL,
  enabled BIT NOT NULL
  );

  CREATE TABLE authorities (
  username VARCHAR(50) NOT NULL,
  authority VARCHAR(50) NOT NULL
  );

  ALTER TABLE authorities ADD CONSTRAINT fk_authorities_users foreign key (username)
REFERENCES users(username);
```

如果默认的表结构不满足你的需要,JdbcDaoImpl提供了属性,可以自定义SQL语句。 请通过JavaDocs查找详细信息,不过请注意,这个类不是为了复杂的自定义子类设计的。 如果你有一个复杂的表结构或想要返回一个自定义UserDetails实现,你最好重写你自己的UserDetailsService。 这个Spring Security提供的基本实现,是为了典型情况,而不是覆盖所有可能出现的需求。

8.3. 并行会话处理

Spring Security可以限制一个主体并行认证到同一系统的次数。 很多ISV利用这点来加强授权公里,网管也喜欢这个功能,因为它可以防止人们共享登录名。 你可以,比如,禁止用户"Batman"从两个不同的会话登录到web应用里。

要使用并行会话支持,你需要把下面内容添加到web.xml里:

```
<listener>
    <listener-
class>org.springframework.security.ui.session.HttpSessionEventPublisher</listener-
class>
</listener>
```

另外,你需要把org.springframework.security.concurrent.ConcurrentSessionFilter添加到你的FilterChainProxy里。 ConcurrentSessionFilter需要两个属性,sessionRegistry,这个通常指向一

个SessionRegistryImpl实例,和expiredUrl,在session过期的时候就指向这个页面。

web.xml里的HttpSessionEventPublisher会在 HttpSession 创建或销毁的时候,发送一个ApplicationEvent给Spring的 ApplicationContext。 这是关键,它让SessionRegistryImpl 知道什么时候session结束了。

你还需要装配 ConcurrentSessionControllerImpl ,然后从你的 ProviderManager bean里引用它:

```
<bean id="authenticationManager"
    class="org.springframework.security.providers.ProviderManager">
  <property name="providers">
    <!-- your providers go here -->
  </property>
  <property name="sessionController" ref="concurrentSessionController"/>
</bean>

<bean id="concurrentSessionController"
    class="org.springframework.security.concurrent.ConcurrentSessionControllerImpl">
  <property name="maximumSessions" value="1"/>
  <property name="sessionRegistry">
    <bean class="org.springframework.security.concurrent.SessionRegistryImpl"/>
  <property>
</bean>
```

8.4. 认证标签库

AuthenticationTag只是用来把当前的Authentication对象的一个属性,输出到网页上。

下面的JSP片段,展示了如何使用 AuthenticationTag:

```
<security:authentication property="principal.username"/>
```

这个标签会把主体的名称显示出来。 这里我们假设Authentication.getPrincipal()是一个UserDetails对象,通常是由Spring Security的标准AuthenticationProvider实现的。



DAO认证提供器

9.1. 综述

Spring Security包含了一个产品级别的AuthenticationProvider实现,叫做DaoAuthenticationProvider。 这个认证提供器兼容所有生成UsernamePasswordAuthenticationToken的验证机制,它可能是框架里最常用到的提供器。 与其他认证提供器一样,DaoAuthenticationProvider通过一个UserDetailsService来获得用户名,密码和GrantedAuthority[]。 与其他认证提供器不同的是,这个认证提供器需要获得一个密码,它会根据认证请求对象里的密码来判断认证是否成功。

9.2. 配置

你需要把DaoAuthenticationProvider加入你的ProviderManager列表(如我们在参考指南这章开始讲到的),并确保合适的认证机制配置为UsernamePasswordAuthenticationToken,其他对提供器自己的配置基本和下面一样简单:

```
<bean id="daoAuthenticationProvider"
    class="org.springframework.security.providers.dao.DaoAuthenticationProvider">
  <property name="userDetailsService" ref="inMemoryDaoImpl"/>
  <property name="saltSource" ref bean="saltSource"/>
  <property name="passwordEncoder" ref="passwordEncoder"/>
</bean>
```

PasswordEncoder和SaltSource是可选的。 PasswordEncoder为从配置好的UserDetailsService中返回的UserDetails对象里的密码,提供编码和反编码的功能。 SaltSource使用盐值生成密码,这可以提升认证资源密码的安全性。 Spring Security支持MD5,SHA和纯文本编码的PasswordEncoder实现。 Spring Security提供了两种SaltSource实现:SystemWideSaltSource对所有密码都使用相同的盐值进行编码,ReflectionSaltSource,使用返回的UserDetails对象的属性来获得盐值。 请参考JavaDocs获得这些选项的更多信息。

除了上述的属性,DaoAuthenticationProvider还可以为UserDetails对象提供缓存。 UserCache接口可以让DaoAuthenticationProvider把一个UserDetails对象放到缓存里,在以后的认证进程中,如果需要同样的用户名就会重新获得它。 默认情况下,DaoAuthenticationProvider使用NullUserCache,这意味着不使用缓存。 Spring Security也提供了一个可用的缓存实现,EhCacheBasedUserCache,如下配置:

```
<bean id="daoAuthenticationProvider"
    class="org.springframework.security.providers.dao.DaoAuthenticationProvider">
  <property name="userDetailsService" ref="userDetailsService"/>
  <property name="userCache" ref="userCache"/>
</bean>

<bean id="cacheManager"
class="org.springframework.cache.ehcache.EhCacheManagerFactoryBean">
  <property name="configLocation" value="classpath:/ehcache-failsafe.xml"/>
</bean>

<bean id="userCacheBackend"
class="org.springframework.cache.ehcache.EhCacheFactoryBean">
  <property name="cacheManager" ref="cacheManager"/>
  <property name="cacheName" value="userCache"/>
</bean>

<bean id="userCache"
class="org.springframework.security.providers.dao.cache.EhCacheBasedUserCache">
  <property name="cache" ref="userCacheBackend"/>
</bean>
```

所有Spring Security的EH-CACHE实现(包括EhCacheBasedUserCache)要求一个EH-CACHE的Cache对象。 这个Cache对象可以从任何地方获得,不过我们推荐你使用Spring的工厂类,请参考Spring文档获得更多的细节,如何优化缓存存储位置,内存使用,剔除策略,超时等。

Note

大部分情况,你的程序都是有状态的web应用程序,你不需要使用缓存,因为用户的认证信息会被保存在HttpSession里。

我们在设计的时候,决定不支持为DaoAuthenticationProvider加锁,因为这样会加大UserDetailsService接口的复杂度。 比如,一个方法可能需要在认证请求失败的时候进行累加计数。 这些功能都可以利用下面讨论到的应用程序事件发布功能,来轻松实现。

DaoAuthenticationProvider返回一个Authentication对象,包含有principal属性集合。 它的内容可能是String(基本就是用户名)或者是UserDetails对象(是从UserDetailsService获得的)。 默认返回的是UserDetails,它允许添加系统特定的潜在属性,比如用户的全名,邮件地址等。 如果使用容器适配器,或者如果你的程序就是要操作String(就像之前在Spring Security 0.6里发布的情况一样),你应该把你程序里的DaoAuthenticationProvider.forcePrincipalAsString属性设置成true。



LDAP认证

10.1. 综述

LDAP通常被公司用作用户信息的中心资源库,同时也被当作一种认证服务。 它也可以为应用用户储存角色信息。

这里有很多如何对LDAP服务器进行配置的场景,所以Spring Security的LDAP提供器也是完全可配置的。 它使用为验证和角色检测提供了单独的策略接口,并提供了默认的实现,这些都是可配置成处理绝大多数情况。

你还是应该熟悉一下LDAP,在你在Spring Security使用它之前。 下面的链接提供了很好的概念介绍,也是一个使用免费的LDAP服务器建立一个目录http://www.zytrax.com/books/ldap/的指南。 我们也应该熟悉一下通过JNDI API使用java访问LDAP。 我们没有在LDAP提供器里使用任何第三方LDAP库(Mozilla, JLDAP等等),但是还是用到了Spring LDAP,所以如果你希望自己进行自定义,对这个工程熟悉一下也是有好处的。

10.2. 在Spring Security里使用LDAP

Spring Security的LDAP认证可以粗略分成以下几部分。

- 从登录名中获得唯一的“辨别名称”或DN。 这就意味着要对目录执行搜索,除非预先知道了用户名和DN之前的明确映射关系。
- 验证这个用户,进行绑定用户,或调用远程“比较”操作,比对用户的密码和DN在目录入口中的密码属性。
- 为这个用户读取权限队列。

例外情况是,当LDAP目录只是用来检索用户信息和进行本地验证的时候,这也许不可能的,因为目录的属性,比如对用户密码属性,常常被设置成只读权限。

让我们看看下面的一些配置场景。 要是想得到所有可用的配置选项,请参考安全命名空间结构(使用你的XML编辑器应该就可以看到所有有效信息)。

10.3. 配置LDAP服务器

你需要做的第一件事是配置服务器,它里面应该存放着认证信息。 这可以使用安全命名空间里的`<ldap-server>`元素实现。 使用`url`属性指向一个外部LDAP服务器:

```
<ldap-server url="ldap://springframework.org:389/dc=springframework,dc=org" />
```

10.3.1. 使用嵌入测试服务器

这个`<ldap-server>`元素也可以用来创建一个嵌入服务器,这在测试和演示的时候特别有用。 在这种情况,你不需要使用`url`属性:

```
<ldap-server root="dc=springframework,dc=org"/>
```

这里我们指定目录的根DIT应该是“dc=springframework,dc=org”,这是默认的。 使用这种方式,命名空间解析器会建立一个嵌入Apache目录服务器,然后检索classpath下的LDIF文件,尝试从它里边把数据加载到服务器里。 你可以通过`ldif`属性自定义这些行为,这样可以定义具体要加载哪个LDIF资源:

```
<ldap-server ldif="classpath:users.ldif" />
```

这就让启动和运行LDAP变得更轻松了,因为使用一个外部服务器还是不大方便。 它也避免链接到Apache目录服务器的复杂bean配置。 如果使用普通Spring bean配置方法会变的更加混乱。 你必须把必要的Apache目录依赖的jar放到你的程序中。 这些都可以从LDAP示例程序中获得。

10.3.2. 使用绑定认证

这是一个非常常见的LDAP认证场景。

```
    <ldap-authentication-provider user-dn-pattern="uid={0},ou=people"/>
```

这个很简单的例子可以根据用户登录名提供的模式为用户获得DN,然后尝试和用户的登录密码进行绑定。 如果所有用户都保存到一个目录的单独节点下就没有问题。 如果你想配置一个LDAP搜索过滤器来定位用户,你可以使用如下配置:

```
    <ldap-authentication-provider user-search-filter="(uid={0})" user-search-
base="ou=people"/>
```

如果使用了上面的服务器定义,它会在DN`ou=people,dc=springframework,dc=org`下执行搜索,使用`user-search-filter`里的值作为过滤条件。 然后把用户登录名作为过滤名称的一个参数。 如果没有提供`user-search-base`,搜索将从根开始。

10.3.3. 读取授权

如果从LDAP目录的组里读取权限信息呢,这是通过下面的属性控制的。

- `group-search-base`。定义目录树部分,哪个组应该执行搜索。
- `group-role-attribute`。这个属性包含了组入口中定义的权限名称。默认是 `cn`
- `group-search-filter`。这个过滤器用来搜索组的关系。 默认是`uniqueMember={0}`,对应于`groupOfUniqueMembers`LDAP类。 在这情况下,取代参数是用户的辨别名称。 如果你想对登录名搜索,可以使用`{1}`这个参数。

因此,如果我们使用下面进行配置

```
    <ldap-authentication-provider user-dn-pattern="uid={0},ou=people" group-search-
base="ou=groups" />
```

并以用户“ben”的身份通过认证,在读取权限信息的子流程里,要在目录入口`ou=groups,dc=springframework,dc=org`下执行搜索,查找包含`uniqueMember`属性值为`ou=groups,dc=springframework,dc=org`的入口。 默认,权限名都要以`ROLE_`作为前缀。 你可以使用`role-prefix`属性修改它。 如果你不想使用任何前缀,可以使用`role-prefix="none"`。 要想得到更多读取权限的信息,可以查看`DefaultLdapAuthoritiesPopulator`类的Javadoc。

10.4. 实现类

我们上面使用到的命名空间选项很容易使用,也比使用spring bean更准确。 也有可能你需要知道如何配置在你的application context里配置Spring Security LDAP目录。 比如,你可能想自定义一些类的行为。 如果你想使用命名空间配置,你可以跳过这节,直接进入下一段。

最主要的LDAP提供器类是`org.springframework.security.providers.ldap.LdapAuthenticationProvider`。 这个bean自己没做什么事情,而是代理了其他两个bean的工作,一个是`LdapAuthenticator`,一个是`LdapAuthoritiesPopulator`,用来处理用户认证和检索用户的`GrantedAuthority`属性集合。

10.4.1. LdapAuthenticator实现

验证者还负责检索所有需要的用户属性。 这是因为对于属性的授权可能依赖于使用的验证类型 比如,如果对某个用户进行绑定,它也许必须通过用户自己的授权才能进行读取。

当前Spring Security提供两种验证策略:

- 直接去LDAP服务器验证("绑定"验证)。
- 比较密码,将用户提供的密码与资源库中保存的进行比较。 这可以通过检索密码属性的值并在本地检测,或者执行LDAP"比较"操作,提供用来比较的密码是从服务器获得的,绝对不会检索真实密码的值。

10.4.1.1. 常用功能

在认证一个用户之前(使用任何一个策略),辨别名称(DN)必须从系统提供的登录名中获得。 这可以通过,简单的模式匹配(设置`setUserDnPatterns`数组属性)或者设置`userSearch`属性。 为了实现DN模式匹配方法,一个标准的java模式格式被用到了,登录名将被参数`{0}`替代。 这个模式应该和DN有关系,并绑定到配置好的`SpringSecurityContextSource`(看看链接到LDAP服务器那节,获得更多信息)。 比如,如果你使用了LDAP服务的URL是`ldap://monkeymachine.co.uk/dc=springframework,dc=org`,并有一个模式`uid={0},ou=greatapes`,然后登录名"gorilla"会映射到DN`uid=gorilla,ou=greatapes,dc=springframework,dc=org`。 每个配置好的DN模式将尝试进行定位,直到有一个匹配上。 使用搜索获得信息,看看下面的安全对象那节。 两种方式也可以结合在一起使用 - 模式会先被检测一下,然后如果没有找到匹配的DN,就会使用搜索。

10.4.1.2. BindAuthenticator

这个类 `org.springframework.security.providers.ldap.authenticator.BindAuthenticator` 实现了绑定认证策略。 它只是尝试对用户进行绑定。

10.4.1.3. PasswordComparisonAuthenticator

这个类 `org.springframework.security.providers.ldap.authenticator.PasswordComparisonAuthenticator` 实现了密码比较认证策略。

10.4.1.4. 活动目录认证

除了标准LDAP认证以外(绑定到一个DN),活动目录对于用户认证提供了自己的非标准语法。

10.4.2. 链接到LDAP服务器

上面讨论的bean必须连接到服务器。 它们都必须使用`SpringSecurityContextSource`,这个是Spring LDAP的一个扩展。 除非你有特定的需求,你通常只需要配置一个`DefaultSpringSecurityContextSource` bean,这个可以使用你的LDAP服务器的URL进行配置,可选项还有管理员用户的用户名和密码,这将默认用在绑定服务器的时候(而不是匿名绑定)。 参考Spring LDAP的`AbstractContextSource`类的Javadoc获得更多信息。

10.4.3. LDAP搜索对象

通常,比简单DN匹配越来越复杂的策略需要在目录里定位一个用户入口。 这可以使用`LdapUserSearch`的一个示例,它可以提供认证者实现,比如让他们定位一个用户。 提供的实现是`FilterBasedLdapUserSearch`。

10.4.3.1. FilterBasedLdapUserSearch

这个bean使用一个LDAP过滤器,来匹配目录里的用户对象。 这个过程在javadoc里进行过解释,在对应的搜索方法,JDK DirContext class。 就如那里解释的,搜索过滤条件可以通过方法指定。 对于这个类,唯一合法的参数是`{0}`,它会代替用户的登录名。

10.4.4. LdapAuthoritiesPopulator

在成功认证用户之后,`LdapAuthenticationProvider`会调用配置好的`LdapAuthoritiesPopulator` bean,尝试读取用户的授权集合。 这个`DefaultLdapAuthoritiesPopulator`是一个实现类,它将通过搜索目录读取授权,查找用户成员所在的组(典型的这会是目录中的`groupOfNames`或`groupOfUniqueNames`入口)。 查看这个类的Javadoc获得它如何工作的更多信息。

10.4.5. Spring Bean配置

典型的配置方法,使用到像我们这在里讨论的这些bean,就像这样:

```
<bean id="contextSource"
        class="org.springframework.security.ldap.DefaultSpringSecurityContextSource">
  <constructor-arg value="ldap://monkeymachine:389/dc=springframework,dc=org"/>
  <property name="userDn" value="cn=manager,dc=springframework,dc=org"/>
  <property name="password" value="password"/>
</bean>

<bean id="ldapAuthProvider"

class="org.springframework.security.providers.ldap.LdapAuthenticationProvider">
  <constructor-arg>
    <bean
class="org.springframework.security.providers.ldap.authenticator.BindAuthenticator">
      <constructor-arg ref="contextSource"/>
      <property name="userDnPatterns">
        <list><value>uid={0},ou=people</value></list>
      </property>
    </bean>
  </constructor-arg>
  <constructor-arg>
    <bean
class="org.springframework.security.ldap.populator.DefaultLdapAuthoritiesPopulator">
      <constructor-arg ref="contextSource"/>
      <constructor-arg value="ou=groups"/>
      <property name="groupRoleAttribute" value="ou"/>
    </bean>
  </constructor-arg>
</bean>
```

这里建立了一个提供器,访问LDAP服务,URL是 `ldap://monkeymachine:389/dc=springframework,dc=org`。 认证会被执行,尝试绑定这个DN `uid=<user-login-name>,ou=people,dc=springframework,dc=org`。 在成功认证之后,会通过查找下面的DN `ou=groups,dc=springframework,dc=org` 使用默认的过滤条件 `(member=<user's-DN>)`,将角色分配给用户。 角色名会通过每个匹配的“ou”属性获得。

要配置用户的搜索对象,使用过滤条件 `(uid=<user-login-name>)` 替代DN匹配(或附加到它上面),你需要配置下面的bean

```
<bean id="userSearch"
    class="org.springframework.security.ldap.search.FilterBasedLdapUserSearch">
  <constructor-arg index="0" value=""/>
  <constructor-arg index="1" value="(uid={0})"/>
  <constructor-arg index="2" ref="contextSource" />
</bean>
```

并使用它,设置认证者的`userSearch`属性。 这个认证者会调用搜索对象,在尝试绑定到用户之前获得正确的用户DN。

10.4.6. LDAP属性和自定义UserDetails

使用`LdapAuthenticationProvider`进行认证的结果,和使用普通Spring Security认证一样,都要使用标准`UserDetailsService`接口。 它会创建一个`UserDetails`对象,并保存到返回的`Authentication`对象里。 在使用`UserDetailsService`时,常见的需求是可以自定义这个实现,添加额外的属性。 在使用LDAP的时候,这些基本都来自用户入口的属性。 `UserDetails`对象的创建结果被提供者的`UserDetailsContextMapper`策略控制,它负责在用户对象和LDAP环境数据之间进行映射:

```
public interface UserDetailsContextMapper {
    UserDetails mapUserFromContext(DirContextOperations ctx, String username,
GrantedAuthority[] authority);

    void mapUserToContext(UserDetails user, DirContextAdapter ctx);
}
```

只有第一个方法与认证有关。 如果你提供这个接口的实现,你可以精确控制如何创建UserDetails对象。 第一个参数是Spring LDAP的`DirContextOperations`实例,他给你访问加载的LDAP属性的通道。 `username`参数是用来认证的名字,最后一个参数是从用户加载的授权列表。]

环境数据加载的方式不同,视乎你采用的认证方法。 使用`BindAuthenticatior`,从绑定操作返回的环境会用来读取属性,否则数据会通过标准的环境,从配置好的`ContextSource`获得(当测试配置好定位用户,这会从搜索对象中获得数据)。



表单认证机制

11.1. 概述

HTTP表单认证,使用AuthenticationProcessingFilter来处理一个登录表单。 这是系统认证最终用户的最常见的一种方法。 基于表单认证与DAO和JAAS认证提供器是完全兼容的。

11.2. 配置

登录表单包含j_username 和 j_password 输入框,然后将数据发送到过滤器监听的一个URL(默认是/j_spring_security_check)。 你应该把AuthenticationProcessingFilter添加到你的application context中:

```
<bean id="authenticationProcessingFilter"
    class="org.springframework.security.ui.webapp.AuthenticationProcessingFilter">
  <property name="authenticationManager" ref="authenticationManager"/>
  <property name="authenticationFailureUrl" value="/login.jsp?login_error=1"/>
  <property name="defaultTargetUrl" value="/"/>
  <property name="filterProcessesUrl" value="/j_spring_security_check"/>
</bean>
```

配置好的AuthenticationManager会处理每个认证请求。 如果认证失败,浏览器会重定向到authenticationFailureUrl。 AuthenticationException会被放到HttpSession中,属性名是AbstractProcessingFilter.SPRING_SECURITY_LAST_EXCEPTION_KEY,在错误页里为用户提供一个出错原因。

如果认证成功了,得到的Authentication对象会被放到SecurityContextHolder中。

一旦SecurityContextHolder更新了, 浏览器需要重定向到目标URL,这个URL通常保存在HttpSession里,属性名是AbstractProcessingFilter.SPRING_SECURITY_TARGET_URL_KEY。 这个属性由ExceptionTranslationFilter在抛出AuthenticationException异常的时候自动设置,这样在登录完成之后,用户可以跳转到最初尝试访问的位置。 如果因为一些原因,HttpSession没办法分析目标URL,浏览器将重定向到defaultTargetUrl属性的值。



基本认证机制

12.1. 概述

Spring Security提供一个BasicProcessingFilter,它可以处理HTTP头部中的基本认证证书。 它可以用来像对待普通用户代理一样(比如IE和Navigator)认证由Spring远程协议的调用(比如Hessian和Burlap)。 HTTP基本认证的执行标准定义在RFC 1945,11章,BasicProcessingFilter符合这个RFC。 基本认证是一个极具吸引力的认证方法,因为它在用户代理发布很广泛,实现也特别简单(只需要对username:password进行Base64编码,再放到HTTP头部里)。

12.2. 配置

要实现HTTP基本认证,要先在过滤器链里定义BasicProcessingFilter。 还要在application context里定义BasicProcessingFilter和协作的类:

```
        <bean id="basicProcessingFilter"
class="org.springframework.security.ui.basicauth.BasicProcessingFilter">
        <property name="authenticationManager"><ref
bean="authenticationManager"/></property>
        <property name="authenticationEntryPoint"><ref
bean="authenticationEntryPoint"/></property>
        </bean>

        <bean id="authenticationEntryPoint"

class="org.springframework.security.ui.basicauth.BasicProcessingFilterEntryPoint">
        <property name="realmName"><value>Name Of Your Realm</value></property>
        </bean>
```

配置好的AuthenticationManager会处理每个认证请求。 如果认证失败,配置好的AuthenticationEntryPoint会用来重试认证过程。 通常你会使用BasicProcessingFilterEntryPoint,它会返回一个401响应,使用对应的头部重试HTTP基本验证。 如果验证成功,就把得到的Authentication对象放到SecurityContextHolder里。

如果认证事件成功,或者因为HTTP头部没有包含支持的认证请求所以没有进行认证,过滤器链会像通常一样继续下去。唯一打断过滤器的情况是在认证失败并调用AuthenticationEntryPoint的时候,向上面段落里讨论的那样。



摘要式认证

13.1. 概述

Spring Security提供了一个 DigestProcessingFilter,它可以处理HTTP头部中的摘要认证证书。 摘要认证在尝试着解决许多基本认证的缺陷,特别是保证不会通过纯文本发送证书。 许多用户支持摘要式认证,包括FireFox和IE。HTTP摘要式认证的执行标准定义在RFC 2617,它是对RFC 2069这个早期摘要式认证标准的更新。 Spring Security DigestProcessingFilter会保证"auth"的安全质量(qop),它订明在RFC 2617中,并与RFC 2069提供了兼容。 如果你需要使用没有加密的HTTP(比如没有TLS/HTTP),还希望认证达到最大的安全性的时候,摘要式认证便具有很高吸引力。 事实上,摘要式认证是WebDAV协议的强制性要求,写在RFC 2518的17.1章,所以我们应该期望看到更多的代替基本认证。

摘要式认证,是表单认证,基本认证和摘要式认证中最安全的选择,不过更安全也意味着更复杂的用户代理实现。 摘要式认证的中心是一个"nonce"。 这是由服务器生成的一个值。 Spring Security的nonce采用下面的格式:

```
    base64(expirationTime + ":" + md5Hex(expirationTime + ":" + key))

    expirationTime:   The date and time when the nonce expires, expressed in
milliseconds
    key:              A private key to prevent modification of the nonce token
```

这个DigestProcessingFilterEntryPoint有一个属性,通过指定一个key来生成nonce标志,通过nonceValiditySeconds属性来决定过期时间(默认300,等于5分钟)。 只要nonce是有效的,摘要就会通过串联字符串计算出来,包括用户名,密码,nonce,请求的URI,一个客户端生成nonce(仅仅是一个随机值,用户代理每个请求生成一个),realm名称等等,然后执行一次MD5散列。 服务器和用户代理都要执行这个摘要计算,如果他们包含的值不同(比如密码),就会生成不同的散列码。 在Spring Security的实现中,如果服务器生成的nonce已经过期(但是摘要还是有效),DigestProcessingFilterEntryPoint会发送一个"stale=true"头信息。 这告诉用户代理,这里不再需要打扰用户(像是密码和用户其他都是正确的),只是简单尝试使用一个新nonce。

DigestProcessingFilterEntryPoint的 nonceValiditySeconds参数,会作为一个适当的值依附在你的程序上。 对安全要求很高的用户应该注意,一个被拦截的认证头部可以用来假冒主体,直到nonce达到expirationTime。 在选择合适的配置的时候,这是一个必须考虑到的关键性条件,但是在对安全性要求很高的程序里,第一次请求都会首先运行在TLS/HTTPS之上。

因为摘要式认证需要更复杂的实现,这里常常有用户代理的问题。 比如,IE不能在同一个会话的请求进程里阻止"透明"标志。 因此Spring Security把所有状态信息都概括到"nonce"标记里。 在我们的测试中,Spring Security在FireFox和IE里都可以工作,正确的处理nonce超时等等。

13.2. 配置

现在我们重新看一下理论,让我们看看如何使用它。 为了实现HTTP摘要认证,必须在过滤器链里定义DigestProcessingFilter。 application context还需要定义DigestProcessingFilter和它需要的合作伙伴:

```
<bean id="digestProcessingFilter"
    class="org.springframework.security.ui.digestauth.DigestProcessingFilter">
  <property name="userDetailsService" ref="jdbcDaoImpl"/>
  <property name="authenticationEntryPoint" ref="digestProcessingFilterEntryPoint"/>
  <property name="userCache" ref="userCache"/>
</bean>

<bean id="digestProcessingFilterEntryPoint"

class="org.springframework.security.ui.digestauth.DigestProcessingFilterEntryPoint">
```

```
  <property name="realmName" value="Contacts Realm via Digest Authentication"/>
  <property name="key" value="acegi"/>
  <property name="nonceValiditySeconds" value="10"/>
</bean>
```

需要配置一个UserDetailsService,因为 DigestProcessingFilter必须直接访问用户的纯文本密码。 如果你在DAO中使用编码过的密码,摘要式认证就没法工作。 DAO合作者,与UserCache一起,通常使用DaoAuthenticationProvider直接共享。 这个authenticationEntryPoint属性必须是DigestProcessingFilterEntryPoint,这样DigestProcessingFilter可以在进行摘要计算时获得正确的realmName和key。

像BasicAuthenticationFilter一样,如果认证成功,会把Authentication请求标记放到SecurityContextHolder中。 如果认证事件成功,或者认证不需要执行,因为HTTP头部没有包含摘要认证请求,过滤器链会正常继续。 过滤器链中断的唯一情况是,如果认证失败,就会像上面讨论的那样调用AuthenticationEntryPoint。

摘要式认证的RFC要求附加功能范围,来更好的提升安全性。 比如,nonce可以在每次请求的时候变换。 但是,Spring Security的设计思路是最小复杂性的实现(毫无疑问,用户代理会出现不兼容),也避免保存服务器端的状态。 如果你想研究这些功能的更多细节,我们推荐你看一下RFC 2617。 像我们知道的那样,Spring Security实现类遵守了RFC的最低标准。



Remember-Me认证

14.1. 概述

记住我(remember-me)或持久登录(persistent-login)认证,指的是网站可以在不同会话之间记忆验证的身份。 通常情况是发送一个cookie给浏览器,在以后的session里检测cookie,进行自动登录。 Spring Security为remember-me实现提供了必要的调用钩子,并提供了两个remember-me的具体实现。 其中一个使用散列来保护基于cookie标记的安全性,另一个使用了数据库或其他持久化存储机制来保存生成的标记。

注意,两个实现方式,都需要`UserDetailsService`。 如果你使用了认证提供器,没有使用`UserDetailsService`(比如LDAP供应器),那它就没法工作,除非你在application context里设置了一个`UserDetailsService`。

14.2. 简单基于散列标记的方法

这种方法使用散列来完成remember-me策略。 本质上,在成功进行认证的之后,把一个cookie发送给浏览器,使用的cookie组成结构如下:

```
    base64(username + ":" + expirationTime + ":" + md5Hex(username + ":" +
expirationTime + ":" password + ":" + key))

    username:          As identifiable to the UserDetailsService
    password:          That matches the one in the retrieved UserDetails
    expirationTime:    The date and time when the remember-me token expires,
expressed in milliseconds
    key:               A private key to prevent modification of the remember-me token
```

这个remember-me标记只适用于指定范围,提供用户名,密码和关键字都不会改变。 值得注意,这里有一个潜在的安全问题,来自任何一个用户代理的remember-me标记,直到标记过期都是可用的。 这个问题和摘要式认证相同。 如果一个用户发现标记已经设置了,他们可以轻易修改他们的密码,并且立即注销所有的remember-me标记。 如果需要更好的安全性,你应该使用下一章描述的方法。 或者不应该使用remember-me服务。

如果你还记得在命名空间配置中讨论的主题,你只要添加`<remember-me>`元素就可以使用remember-me认证:

```
  <http>
    ...
    <remember-me key="myAppKey"/>
  </http>
```

如果你使用了auto-config设,它也会自动启用。 这个`UserDetailsService`会自动选上。 如果你在application context中配置了多个,你需要使用`user-service-ref`属性指定应该使用哪一个,这里的值要放上你的`UserDetailsService` bean的名字。

14.3. 持久化标记方法

这个方法是基于这篇文章 http://jaspan.com/improved_persistent_login_cookie_best_practice 进行了一些小修改[3]。 要用在命名空间配置里使用这个方法,你应该提供一个datasource引用:

```
  <http>
    ...
    <remember-me data-source-ref="someDataSource"/>
  </http>
```

```
```

数据应该包含一个 persistent_logins 表,可以使用下面的SQl创建(或等价物):

```
    create table persistent_logins (username varchar(64) not null, series
varchar(64) primary key, token varchar(64) not null, last_used timestamp not null)
```

14.4. Remember-Me接口和实现

Remember-me认证不能和基本认证一起使用,因为基本认证往往不使用HttpSession。 Remember-me使用在AuthenticationProcessingFilter中,通过在它的超类AbstractProcessingFilter里实现的一个调用钩子。 这个钩子会在合适的时候调用一个具体的RememberMeServices。 这个接口看起来像这样:

```
  Authentication autoLogin(HttpServletRequest request, HttpServletResponse
response);
  void loginFail(HttpServletRequest request, HttpServletResponse response);
  void loginSuccess(HttpServletRequest request, HttpServletResponse response,
Authentication successfulAuthentication);
```

请参考JavaDocs获得有关这些方法的完整讨论,不过注意在这里,AbstractProcessingFilter只调用loginFail() 和loginSuccess()方法。 当SecurityContextHolder没有包含Authentication的时候,RememberMeProcessingFilter才去调用autoLogin()。 因此,这个接口通过使用完整的认证相关事件的提醒提供了下面remember-me实现,然后在可能包含一个cookie希望被记得的申请web请求中调用这个实现。 这个设计允许任何数目的remember-me实现策略。 我们在下面看看上面介绍过的两个Spring Security提供的实现。

14.4.1. TokenBasedRememberMeServices

这个实现支持在Section 14.2, “简单基于散列标记的方法”里描述的简单方法。 TokenBasedRememberMeServices被RememberMeAuthenticationProvider执行的时候生成一个RememberMeAuthenticationToken。 认证提供器和TokenBasedRememberMeServices之间共享一个key。 另外TokenBasedRememberMeServices需要一个UserDetailsService,用它来获得用户名和密码,进行比较,然后生成RememberMeAuthenticationToken来包含正确的GrantedAuthority[]。 如果用户请求注销,让cookie失效,就应该使用系统提供的一系列注销命令。 TokenBasedRememberMeServices也实现Spring Security的LogoutHandler接口,这样可以使用LogoutFilter自动清除cookie。

这些bean要求在application context里启用remember-me服务,像下面一样:

```
<bean id="rememberMeProcessingFilter"
    class="org.springframework.security.ui.rememberme.RememberMeProcessingFilter">
  <property name="rememberMeServices" ref="rememberMeServices"/>
  <property name="authenticationManager" ref="theAuthenticationManager" />
</bean>

<bean id="rememberMeServices"
class="org.springframework.security.ui.rememberme.TokenBasedRememberMeServices">
  <property name="userDetailsService" ref="myUserDetailsService"/>
  <property name="key" value="springRocks"/>
</bean>

<bean id="rememberMeAuthenticationProvider"

class="org.springframework.security.providers.rememberme.RememberMeAuthenticationProvider'

  <property name="key" value="springRocks"/>
</bean>
```

不要忘记把你的RememberMeServices实现添加到AuthenticationProcessingFilter.setRememberMeServices()属性

中,包括把RememberMeAuthenticationProvider添加到你的AuthenticationManager.setProviders()队列中,把RememberMeProcessingFilter添加到你的FilterChainProxy中(要放到AuthenticationProcessingFilter后面)。

14.4.2. PersistentTokenBasedRememberMeServices

这个类可以像TokenBasedRememberMeServices一样使用,但是它还需要配置一个PersistentTokenRepository来保存标记。 这里有两个标准实现。

- InMemoryTokenRepositoryImpl最好是只用来测试。
- JdbcTokenRepositoryImpl把标记保存到数据库里。

数据库表结构在 Section 14.3, “持久化标记方法”.

---

[3] 基本上,为了防止暴露有效登录名,用户名没有包含在cookie里。在这个文章的评论里有一个相关的讨论。



Java认证和授权服务(JAAS)供应器

15.1. 概述

Spring Security提供一个包,可以代理Java认证和授权服务(JAAS)的认证请求。 这个包的细节在下面讨论。

JAAS的核心是登录配置文件。 想要了解更多JAAS登录配置文件的信息,可以查询Sun公司的JAAS参考文档。 我们希望你对JAAS有一个基本了解,也了解它的登录配置语法,这才能更好的理解这章的内容。

15.2. 配置

这个 `JaasAuthenticationProvider` 通过JAAS认证用户的主体和证书。

让我们假设我们有一个JAAS登录配置文件,`/WEB-INF/login.conf`,里边的内容如下:

```
JAASTest {
    sample.SampleLoginModule required;
};
```

就像所有的Spring Security bean一样,这个`JaasAuthenticationProvider` 要配置在application context里。 下面的定义是与上面的JAAS登录配置文件对应的:

```
<bean id="jaasAuthenticationProvider"

class="org.springframework.security.providers.jaas.JaasAuthenticationProvider">
  <property name="loginConfig" value="/WEB-INF/login.conf"/>
  <property name="loginContextName" value="JAASTest"/>
  <property name="callbackHandlers">
    <list>
      <bean
class="org.springframework.security.providers.jaas.JaasNameCallbackHandler"/>
      <bean
class="org.springframework.security.providers.jaas.JaasPasswordCallbackHandler"/>
    </list>
  </property>
  <property name="authorityGranters">
    <list>
      <bean
class="org.springframework.security.providers.jaas.TestAuthorityGranter"/>
    </list>
  </property>
</bean>
```

这个 `CallbackHandler`和`AuthorityGranter`会在下面进行讨论。

15.2.1. JAAS CallbackHandler

大多数JAAS的登录模块需要设置一系列的回调方法。 这些回调方法通常用来获得用户的用户名和密码。

在Spring Security发布的时候,Spring Security负责用户交互(通过认证机制)。 因此,现在认证请求使用JAAS代理,Spring Security的认证机制将组装一个`Authentication`对象,它包含了所有JAAS`LoginModule`需要的信息。

因此,Spring Security的JAAS包提供两个默认的回调处理器,`JaasNameCallbackHandler` 和`JaasPasswordCallbackHandler`。 他们两个都实现了`JaasAuthenticationCallbackHandler`。 大多数情况下,这些回调函数可以直接使用,不用了解它们的内部机制。

为了需要完全控制回调行为,内部`JaasAutheticationProvider`使用一个`InternalCallbackHandler`封装这
`JaasAuthenticationCallbackHandler` `InternalCallbackHandler`

个 。 这个 才是实际实现了JAAS通常的CallbackHandler接口。 任何时候JAASLoginModule被使用的时候,它传递一个application context里配置的InternalCallbackHandler列表。 如果这个LoginModule需要回调InternalCallbackHandler,回调会传递封装好的JaasAuthenticationCallbackHandler。

15.2.2. JAAS AuthorityGranter

JAAS工作在主体上。 任何“角色”在JAAS里都是作为主体表现的。 另一方面Spring Security使用Authentication对象。 每个Authentication对象包含单独的主体和多个GrantedAuthority[]。 为了方便映射不同的概念,Spring Security的JAAS包包含了AuthorityGranter接口。

一个 AuthorityGranter 负责检查JAAS主体,返回一个String。 JaasAuthenticationProvider会创建一个JaasGrantedAuthority(实现了Spring Security的GrantedAuthority接口),包含了AuthorityGranter返回的字符串和AuthorityGranter传递的JAAS主体。 JaasAuthenticationProvider获得JAAS主体,通过首先成功认证用户的证书,使用JAAS的LoginModule,然后调用LoginContext.getSubject().getPrincipals(),使用返回的每个主体,传递到每个AuthorityGranter里,最后定义在JaasAuthenticationProvider.setAuthorityGranters(List)属性里。

Spring Security没有包含任何产品型的AuthorityGranter,因为每个JAAS主体都有特殊实现的意义。 但是,这里的单元测试里有一个TestAuthorityGranter,演示了一个简单的AuthorityGranter实现。



预认证场景

有的情况下,你需要使用Spring Security进行认证,但是用户已经在访问系统之前,在一些外部系统中认证过了。 我们把这种情况叫做“预认证”场景。 例子包括X.509,Siteminder和应用所在的J2EE容器进行认证。 在使用预认证的使用,Spring Security必须

- 定义使用请求的用户
- 从用户里获得权限

细节信息要依靠外部认证机制。 一个用户可能,在X.509的情况下由认证信息确定,或在Siteminder的情况下使用HTTP请求头。 对于容器认证,需要调用获得HTTP请求的`getUserPrincipal()`方法来确认用户。 一些情况下,外部机制可能为用户提供角色/权限信息,其他情况就需要通过单独信息源获得,比如`UserDetailsService`。

16.1. 预认证框架类

因为大多数预认证机制都遵循相同的模式,所以Spring Security提供了一系列的类,它们作为内部框架实现预认证认证提供器。 这样就避免了重复实现,让新实现很容易添加到结构中,不需要一切从脚手架开始写起。 你不需要知道这些类,如果你想使用一些东西,比如X.509认证,因为它已经是命名空间配置里的一个选项了,可以很简单的使用,启动它。 如果你需要使用精确的bean配置,或计划编写你自己的实现,这时了解这些提供的实现是如何工作就很有用了。 你会在org.springframework.security.ui.preauth包下找到web相关的类,后台类都在org.springframework.security.providers.preauth包里。 我们这里只提供一个纲要,你应该从对应的Javadoc和源代码里获得更多信息。

16.1.1. AbstractPreAuthenticatedProcessingFilter

这个类会检测安全环境的当前内容,如果是空的,它会从HTTP请求里获得信息,提交给`AuthenticationManager`。 子类重写了以下方法来获得信息:

```
  protected abstract Object getPreAuthenticatedPrincipal(HttpServletRequest
request);

  protected abstract Object getPreAuthenticatedCredentials(HttpServletRequest
request);
```

在调用之后,过滤器会创建一个包含了返回数据的`PreAuthenticatedAuthenticationToken`,然后提交它进行认证。 通过这里的“authentication”,我们其实只是可能进行读取用户的权限,不过下面就是标准的Spring Security认证结构了。

16.1.2. AbstractPreAuthenticatedAuthenticationDetailsSource

就像其他的Spring Security认证过滤器一样,预认证过滤器有一个`authenticationDetailsSource`属性,默认会创建一个`WebAuthenticationDetails`对象来保存额外的信息,比如在`Authentication`对象的`details`属性里的会话标识,原始IP地址。 用户角色信息可以从预认证机制中获得,数据也保存在这个属性里。`AbstractPreAuthenticatedAuthenticationDetailsSource`的子类,使用实现了`GrantedAuthoritiesContainer`接口的扩展信息,因此可以使用认证提供其器来读取权限,明确定位用户。 下面我们看一个具体的例子。

16.1.2.1. J2eeBasedPreAuthenticatedWebAuthenticationDetailsSource

如果过滤器配置了`authenticationDetailsSource`的实例,通过调用`isUserInRole(String role)`方法为每个预先决定的“可映射角色”集合获得认证信息。 这个类从`MappableAttributesRetriever`里获得这些信息。 可能的实现方法,包含了在application context中进行硬编码,或者从`web.xml`的`<security-role>`中读取角色信息。 预认证例子程序使用了后一种方式。

这儿有一个额外的步骤,使用一个`Attributes2GrantedAuthoritiesMapper`把角色(或属性)映射到Spring

Security的GrantedAuthority。 它默认只会为名称添加一个ROLE_前缀,但是你可以对这些行为进行完全控制。

16.1.3. PreAuthenticatedAuthenticationProvider

预认证提供器除了从用户中读取UserDetails以外,还要一些其他事情。 它通过调用一个AuthenticationUserDetailsService来做这些事情。 后者就是一个标准的UserDetailsService,但要需要的参数是一个Authentication对象,而不是用户名:

```
  public interface AuthenticationUserDetailsService {
    UserDetails loadUserDetails(Authentication token) throws
UsernameNotFoundException;
  }
```

这个接口可能也含有其他用户,但是预认证允许访问权限,打包在Authentication对象里,像上一节所见的。 这个PreAuthenticatedGrantedAuthoritiesUserDetailsService就是用来作这个的。 或者,它可能调用标准UserDetailsService,使用UserDetailsByNameServiceWrapper这个实现。

16.1.4. PreAuthenticatedProcessingFilterEntryPoint

这个 AuthenticationEntryPoint 在 技术概述 那章讨论过。 通常它用来为未认证用户(当他们想访问被保护资源的时候)启动认证过程,但是在预认证情况下这不会发生。 如果你不使用预认证结合其他认证机制的话,你只要配置ExceptionTranslationFilter的一个实例。 如果用户的访问被拒绝了,它就会调用,AbstractPreAuthenticatedProcessingFilter结果返回的一个空的认证。 调用的时候,它总会返回一个403禁用响应代码。

16.2. 具体实现

X.509认证写在它自己的章里。 这里,我们看一些支持其他预认证的场景。

16.2.1. 请求头认证(Siteminder)

一个外部认证系统可以通过在HTTP请求里设置特殊的头信息,给应用提供信息。 一个众所周知的例子就是Siteminder,它在头部传递用户名,叫做SM_USER。 这个机制被RequestHeaderPreAuthenticatedProcessingFilter支持,直接从头部得到用户名。 默认使用SM_USER作为头部名。看一下Javadoc获得更多信息。

> **Tip**
>
> 注意使用这种系统时,框架不需要作任何认证检测,极端重要的是,要把外部系统配置好,保护系统的所有访问。 如果攻击者可以从原始请求中获得请求头,不通过检测,他们可能潜在修改任何他们想要的用户名。

16.2.1.1. Siteminder示例配置

使用这个过滤器的典型配置应该像这样:

```
    <bean id="siteminderFilter"

class="org.springframework.security.ui.preauth.header.RequestHeaderPreAuthenticatedProcess

    <security:custom-filter position="PRE_AUTH_FILTER" />
    <property name="principalRequestHeader" value="SM_USER"/>
    <property name="authenticationManager" ref="authenticationManager" />
  </bean>

  <bean id="preauthAuthProvider"

class="org.springframework.security.providers.preauth.PreAuthenticatedAuthenticationProvic

    <security:custom-authentication-provider />
    <property name="preAuthenticatedUserDetailsService">
```

```
      <bean id="userDetailsServiceWrapper"

class="org.springframework.security.userdetails.UserDetailsByNameServiceWrapper">
        <property name="userDetailsService" ref="userDetailsService"/>
      </bean>
    </property>
    </bean>

    <security:authentication-manager alias="authenticationManager" />
```

我们假设使用安全命名空间的配置方式,`custom-filter`,使用了`authentication-manager`和`custom-authentication-provider`三个元素(你可以从命名空间章节里了解它们的更多信息)。 你应该走出传统的配置方式。 我们也假设了你在配置里添加了一个`UserDetailsService`(名叫“userDetailsService”),来读取用户的角色信息。

16.2.2. J2EE容器认证

这个`J2eePreAuthenticatedProcessingFilter`类会从`HttpServletRequest`的`userPrincipal`属性里获得准确的用户名。 这个过滤器的用法常常结合使用J2EE角色,像上面描述的Section 16.1.2.1, “J2eeBasedPreAuthenticatedWebAuthenticationDetailsSource”。



匿名认证

17.1. 概述

有时,特别是在web请求URI安全的情况里,为每个安全对象的调用都分配一个配置属性更方便。 不同的是,有时默认需要一个ROLE_SOMETHING更好一些,这个规则只允许一些特定例外,比如登录,注销,系统的首页。 也有需要匿名认证的其他情况,比如审核拦截器查询SecurityContextHolder来确定哪个主体可以使用对应的操作。 如果它们能确定SecurityContextHolder里总是包含一个Authentication对象,而不可能是null,这些类就可以得到更多的鲁棒性。

17.2. 配置

Spring Security提供三个类来一起提供匿名认证功能。 AnonymousAuthenticationToken实现了Authentication,保存着GrantedAuthority[],用来处理匿名主体。 有一个对应的需要链入ProviderManager的AnonymousAuthenticationProvider,可以从中获得AnonymousAuthenticationTokens。 最后是AnonymousProcessingFilter,需要串链到普通认证机制后面,如果还没有存在的Authentication的话,它会自动向SecurityContextHolder添加一个AnonymousAuthenticationToken。 过滤器和认证提供器的配置如下:

```
<bean id="anonymousProcessingFilter"

class="org.springframework.security.providers.anonymous.AnonymousProcessingFilter">
  <property name="key" value="foobar"/>
  <property name="userAttribute" value="anonymousUser,ROLE_ANONYMOUS"/>
</bean>

<bean id="anonymousAuthenticationProvider"

class="org.springframework.security.providers.anonymous.AnonymousAuthenticationProvider">

  <property name="key" value="foobar"/>
</bean>
```

这个 key 会在过滤器和认证提供器之间共享,这样创建的标记可以在以后用到。 userAttribute表达式的格式是usernameInTheAuthenticationToken,grantedAuthority[,grantedAuthority]。 这和InMemoryDaoImpl中userMap属性的语法一样。

如上面所讲的,匿名认证的好处是,可以对所有的URL模式都进行安全配置。 比如:

```
<bean id="filterInvocationInterceptor"
    class="org.springframework.security.intercept.web.FilterSecurityInterceptor">
  <property name="authenticationManager" ref="authenticationManager"/>
  <property name="accessDecisionManager" ref="httpRequestAccessDecisionManager"/>
  <property name="objectDefinitionSource">
    <security:filter-invocation-definition-source>
      <security:intercept-url pattern='/index.jsp'
access='ROLE_ANONYMOUS,ROLE_USER'/>
      <security:intercept-url pattern='/hello.htm'
access='ROLE_ANONYMOUS,ROLE_USER'/>
      <security:intercept-url pattern='/logoff.jsp'
access='ROLE_ANONYMOUS,ROLE_USER'/>
      <security:intercept-url pattern='/login.jsp'
access='ROLE_ANONYMOUS,ROLE_USER'/>
      <security:intercept-url pattern='/**' access='ROLE_USER'/>
    </security:filter-invocation-definition-source>
```

```
  </property>
</bean>
```

简略对匿名认证的讨论,就是AuthenticationTrustResolver接口,它对应着AuthenticationTrustResolverImpl实现。这个接口提供了一个isAnonymous(Authentication)方法,允许感兴趣的类评估认证的特殊状态类型。 在处理AccessDeniedException异常的时候,ExceptionTranslationFilter使用这个接口。 如果抛出了一个AccessDeniedException异常,而且认证是匿名类型,那么不会抛出403(禁止)响应,这个过滤器会展开AuthenticationEntryPoint,这样主体可以正确验证。 这是一个必要的区别,否则主体会一直被认为“需要认证”,没有机会通过表单,摘要,或其他普通的认证机制登录。

Prev | Home | Next
Chapter 16. 预认证场景 | Sponsored by SpringSource | Chapter 18. X.509认证



X.509认证

18.1. 概述

X.509证书认证最常见的使用方法是使用SSL验证服务器的身份,通常情况是在浏览器使用SSL。 浏览器使用一个它维护的可信任的证书权限列表,自动检测服务器发出的证书(比如数字签名)。

你也可以使用SSL进行“mutual authentication”相互认证;服务器会从客户端请求一个合法的证书,作为SSL握手协议的一部分。 服务器将验证客户端,通过检测它被签在一个可接受的权限里的认证。 如果已经提供了一个有效的证书,就可以从程序的servlet API里获得。 Spring Security X.509模块确认证书,使用过滤器,传递一个配置好的X.509认证提供器,允许任何复杂的特定程序检测使用。 它也可以为应用程序的用户映射证书,并使用标准的Spring Security基础设施读取用户的已授予权限集合。

你应该很熟悉使用证书,在使用Spring Security之前为你的servlet容器启动客户端认证。 大多数工作都是创建和安装合适的证书和密匙。 比如,如果你使用tomcat,可以阅读这里的教程http://tomcat.apache.org/tomcat-6.0-doc/ssl-howto.html。 在你把它用在Spring Security里之前,先知道它是怎么工作,是很重要的。

18.2. 把X.509认证添加到你的web系统中

启用X.509客户端认证非常直观。 只需要把`<x509/>`元素添加到你的http安全命名空间配置里。

```
<http>
 ...
    <x509 subject-principal-regex="CN=(.*?)," user-service-ref="userService"/>
 ...
</http>
```

这个元素有两个可选属性:

- `subject-principal-regex`。这是一个正则表达式,用来从证书主体名称里获得用户名。默认值已经写在上面了。这个用户名会传递给`UserDetailsService`来获得用户的认证信息。
- `user-service-ref`。这是X.509需要用到的一个`UserDetailsService`的bean的id。如果你的application context里只定义了一个bean,就不需要使用它。

`subject-principal-regex`应该包含一个单独的组。 比如默认的表达式"CN=(.*?),"匹配普通的名字字段。 所以,如果证书主题名是"CN=Jimi Hendrix, OU=...",就会得到一个名叫"Jimi Hendrix"的用户。 这个匹配是大小写不敏感的。 所以"emailAddress=(.?),"也会匹配"EMAILADDRESS=jimi@hendrix.org,CN=...",得到一个"jimi@hendrix.org"用户名。 如果客户端给出一个证书,并成功获得了一个合法用户名,然后在安全环境里应该有一个有效的`Authentication`对象。 如果没有找到证书,或没有找到对应的用户,安全环境会保持为空。 这说明你可以很简单的和其他选项一起使用X.509认证,比如基于表单登录。

18.3. 为tomcat配置SSL

在Spring Security项目的`samples/certificate`目录下,有几个已经生成好的证书。 如果你不想自己去生成,就可以使用们启用SSL做测试,。 `server.jks`文件包含了服务器证书,私匙和签发证书颁发机构证书。 这里还有一些客户端证书文件,提供给例子程序的用户。 你可以把他们安装到你的浏览器,启动SSL客户端认证。

要运行支持SSL的tomcat,把 `server.jks` 文件放到tomcat的 `conf` 目录下,然后把下面的连接器添加到 `server.xml` 文件中

```
<Connector port="8443" protocol="HTTP/1.1" SSLEnabled="true" scheme="https"
secure="true"
           clientAuth="true" sslProtocol="TLS"
```

```
            keystoreFile="${catalina.home}/conf/server.jks"
            keystoreType="JKS" keystorePass="password"
            truststoreFile="${catalina.home}/conf/server.jks"
            truststoreType="JKS" truststorePass="password"
/>
```

clientAuth 也可以设置成 *want*,如果你希望客户端没有提供证书的时候SSL链接也能成功。 客户端不提供证书的话,就不能访问Spring Security的任何安全对象,除非你使用了非X.509认证机制,比如表单认证。



Chapter 19. CAS认证

19.1. 概述

JA-SIG开发了一个企业级的单点登录系统,叫做CAS。 与其他项目不同,JA-SIG的中心认证服务是开源的,广泛使用的,简单理解的,不依赖平台的,而且支持代理能力。 Spring Security完全支持CAS,提供一个简单的整合方式,把使用Spring Security的单应用发布,转换成使用企业级CAS服务器的多应用发布安全

你可以从`http://www.ja-sig.org/products/cas/`找到CAS的更多信息。 你还需要访问这个网站下载CAS服务器文件。

19.2. CAS是如何工作的

虽然CAS网站包含了CAS的架构文档,我们这里还是说一下使用Spring Security环境的一般性概述,。 Spring Security 2.0支持CAS 3。 在写文档的时候,CAS服务器的版本是3.2。

你要在公司内部安装CAS服务器。 CAS服务器就是一个WAR文件,所以安装服务器没有什么难的。 在WAR文件里,你需要自定义登录和其他单点登录展示给用户的页面。

发布CAS 3.2的时候,你也需要指定一个CAS的`deployerConfigContext.xml`里包含的`AuthenticationHandler`。`AuthenticationHandler`有一个简单的方法,返回布尔值,判断给出的证书集合是否有效。 你的`AuthenticationHandler`实现会需要链接到后台认证资源类型里,像是LDAP服务器或数据库。 CAS自己也包含非常多`AuthenticationHandler`帮助实现这些。 在你下载发布服务器war文件的时候,它会把用户名和密码匹配的用户成功验证,这对测试很有用。

除了CAS服务器,其他关键角色当然是你企业发布的其他安全web应用。 这些web应用被叫做"services"。 这儿有两种服务:标准服务和代理服务。 代理服务可以代表用户,从其他服务中请求资源。 后面会进行更详细的介绍。

19.3. 配置CAS客户端

CAS的web应用端通过Spring Security使用起来很简单。 我们假设你已经知道Spring Security的基本用法,所以下面都没有涉及这些。 我们会假设使用基于命名空间配置的方法,并且添加了CAS需要的bean。

你需要添加一个 `ServiceProperties` bean,到你的application context里。 这表现你的服务:

```
  <bean id="serviceProperties"
class="org.springframework.security.ui.cas.ServiceProperties">
    <property name="service" value="https://localhost:8443/cas-
sample/j_spring_cas_security_check"/>
    <property name="sendRenew" value="false"/>
  </bean>
```

这里的`service`必须是一个由`CasProcessingFilter`监控的URL。 这个`sendRenew`默认是false,但如果你的程序特别敏感就应该设置成true。 这个参数作用是,告诉CAS登录服务,一个单点登录没有到达。 否则,用户需要重新输入他们的用户名和密码,来获得访问服务的权限。

下面配置的bean就是展开CAS认证的过程:

```
<security:authentication-manager alias="authenticationManager"/>

<bean id="casProcessingFilter"
class="org.springframework.security.ui.cas.CasProcessingFilter">
  <security:custom-filter after="CAS_PROCESSING_FILTER"/>
  <property name="authenticationManager" ref="authenticationManager"/>
  <property name="authenticationFailureUrl" value="/casfailed.jsp"/>
```

```
  <property name="defaultTargetUrl" value="/"/>
</bean>

<bean id="casProcessingFilterEntryPoint"
    class="org.springframework.security.ui.cas.CasProcessingFilterEntryPoint">
  <property name="loginUrl" value="https://localhost:9443/cas/login"/>
  <property name="serviceProperties" ref="serviceProperties"/>
</bean>
```

应该使用 entry-point-ref选择驱动认证的CasProcessingFilterEntryPoint类。

CasProcessingFilter的属性与 AuthenticationProcessingFilter非常相似(在基于表单登录时用到)。 每个属性从字面上都很好理解。 注意我们也使用命名空间语法,为认证管理器设置一个别名,这样CasProcessingFilter需要它的一个引用。

为了CAS的操作,ExceptionTranslationFilter必须有它的authenticationEntryPoint,这里设置成CasProcessingFilterEntryPoint bean。

CasProcessingFilterEntryPoint 必须指向 ServiceProperties bean (上面讨论过了),它为企业CAS登录服务器提供URL。 这是用户浏览器将被重定向的位置。

下一步,你需要添加一个CasAuthenticationProvider 和它的合作伙伴:

```
  <bean id="casAuthenticationProvider"
class="org.springframework.security.providers.cas.CasAuthenticationProvider">
    <security:custom-authentication-provider />
    <property name="userDetailsService" ref="userService"/>
    <property name="serviceProperties" ref="serviceProperties" />
    <property name="ticketValidator">
      <bean class="org.jasig.cas.client.validation.Cas20ServiceTicketValidator">
        <constructor-arg index="0" value="https://localhost:9443/cas" />
        </bean>
    </property>
    <property name="key" value="an_id_for_this_auth_provider_only"/>
  </bean>

  <security:user-service id="userService">
    <security:user name="joe" password="joe" authorities="ROLE_USER" />
    ...
  </security:user-service>
```

一旦通过了CAS的认证,CasAuthenticationProvider 使用一个 UserDetailsService实例,来加载用户的认证信息。 这里我们展示一个简单的基于内存的设置。

如果你翻回头看一下"How CAS Works"那节,这些beans都是从名字就可以看懂的。



替换验证身份

20.1. 概述

AbstractSecurityInterceptor可以在安全对象回调期间,暂时替换SecurityContext和SecurityContextHolder里的Authentication对象。 只有在原始Authentication对象被AuthenticationManager和AccessDecisionManager成功处理之后,才有可能发生这种情况。 如果有需要,RunAsManager会显示替换的Authentication对象,这应该通过SecurityInterceptorCallback调用。

通过在安全对象回调过程中临时替换 Authentication 对象,安全调用可以调用其他需要不同认证授权证书的对象。 这也可以为特定的GrantedAuthority对象执行内部安全检验。 因为Spring Security提供不少帮助类,能够基于SecurityContextHolder的内容自动配置远程协议,这些运行身份替换在远程web服务调用的时候特别有用。

20.2. 配置

一个Spring Security提供的 RunAsManager 接口::

```
  Authentication buildRunAs(Authentication authentication, Object object,
ConfigAttributeDefinition config);
  boolean supports(ConfigAttribute attribute);
  boolean supports(Class clazz);
```

第一个方法返回 Authentication对象,在方法的调用期间替换以前的 Authentication 对象。 如果方法返回null,意味着不需要进行替换。 第二个方法用在AbstractSecurityInterceptor中,作为它启动时校验配置属性的一部分。 supports(Class)方法会被安全拦截器的实现调用,确保配置的RunAsManager支持安全拦截器即将执行的安全对象类型。

Spring Security提供了一个 RunAsManager的具体实现。 如果任何一个ConfigAttribute是以RUN_AS_开头的,RunAsManagerImpl类返回一个替换的RunAsUserToken。 如果找到了任何这样的ConfigAttribute,替换的RunAsUserToken会通过一个新的GrantedAuthorityImpl,为每一个RUN_AS_ ConfigAttribute包含同样的主体,证书,赋予的权限,就像原来的Authentication对象一样。 每个新 GrantedAuthorityImpl 会以ROLE_开头,对应RUN_AS ConfigAttribute。 比如,一个替代RunAsUserToken,对于RUN_AS_SERVER的结果是包含一个ROLE_RUN_AS_SERVER赋予的权限。

替代的 RunAsUserToken 就像其他 Authentication 对象一样。 它可能需要通过代理合适的AuthenticationProvider被AuthenticationManager验证。 这个RunAsImplAuthenticationProvider执行这样的认证,它直接获得任何一个有效的RunAsUserToken。

为了保证恶意代码不会创建一个RunAsUserToken,由RunAsImplAuthenticationProvider保障获得一个key的散列值被保存在所有生成的标记里。 RunAsManagerImpl和RunAsImplAuthenticationProvider在bean上下文里,创建使用同样的key:

```
<bean id="runAsManager" class="org.springframework.security.runas.RunAsManagerImpl">
  <property name="key" value="my_run_as_password"/>
</bean>

<bean id="runAsAuthenticationProvider"
    class="org.springframework.security.runas.RunAsImplAuthenticationProvider">
  <property name="key" value="my_run_as_password"/>
</bean>
```

通过使用相同的key,每个RunAsUserToken可以被它验证,并使用对应的RunAsManagerImpl创建。 出于安全原因,这个RunAsUserToken创建后就不能改变。

Prev | Home | Next
Chapter 19. CAS认证 | Sponsored by SpringSource | Chapter 21. 容器适配器认证



容器适配器认证

21.1. 概述

非常早期版本的Spring Security使用容器适配器进行最终用户的认证。 虽然它运行良好,但是需要很多时间来支持不同的容器版本,对于开发者来说配置时间也太长了。 因为这个原因,HTTP表单认证和HTTP基础认证方法才被开发出来,直到今天,被推荐用在几乎所有的程序中。

容器适配器让Spring Security可以将主机的最终用户程序与容器直接集成。 这种集合意味着,程序可以继续使用容易本身的认证和授权能力(比如`isUserInRole()`和基于表单或基本认证),虽然Spring Security提供加强的安全拦截能力(应该注意到,Spring Security也允许是用`ContextHolderAwareRequestWrapper`发送`isUserInRole()`,和简单的servlet规范兼容的方法)。

一般通过适配器进行容器和Spring Security集成。 适配器提供一个容器兼容的用户认证提供器,需要返回容器兼容的用户对象。

适配器由容器实例化,并定义在容器特定的配置文件里。 适配器将读取定义了普通的认证管理器设置的Spring的application context,就像可以用来认证请求的认证提供器一样。 application context通常叫做`acegisecurity.xml`,要把它放在容器指定的位置。

Spring Security现在支持Jetty, Catalina (Tomcat), JBoss 和 Resin。 其他容器的适配器也很容易编写。

21.2. 适配器认证提供器

始终是这种情况,容器适配器生成的`Authentication`对象还是需要在`AbstractSecurityInterceptor`需要处理请求的时候被`AuthenticationManager`认证。 `AuthenticationManager`需要确认适配器提供的`Authentication`对象是有效的,也确实被一个可信任的适配器认证过了。

适配器创建了 `Authentication` 对象,它是不变的,实现了`AuthByAdapter`接口。 这些对象保存了由适配器定义的key的散列码,。 这允许`Authentication`对象被`AuthByAdapterProvider`验证。 这个认证提供器定义如下:

```
<bean id="authByAdapterProvider"
        class="org.springframework.security.adapters.AuthByAdapterProvider">
<property name="key"><value>my_password</value></property>
</bean>
```

这里的key必须与定义在容器特定配置文件,用来启动适配器的key相同。 这个`AuthByAdapterProvider`自动获得任何有效的`AuthByAdapter`实现,然后返回期待的key的散列值。

重申一下,这意味着适配器会在初始化认证的时候使用供应器,比如`DaoAuthenticationProvider`,返回一个包含key的散列值的`AuthByAdapter`实例,。 然后,当应用程序调用安全拦截器管理的资源时,`AuthByAdapter`实例,放在`SecurityContextHolder`里的`SecurityContext`,会被程序的`AuthByAdapterProvider`测试到。 这里不需要附加认证供应器,比如放到应用程序特定的application context里的`DaoAuthenticationProvider`,这是唯一的`Authentication`对象类型,会被从容器适配器里的程序用到的。

Classloader问题常常困扰着容器,以后会描述容器适配器的用法。 每个容器要求一个特别指定的配置。 安装教程提供在下面。 一旦安装好,请花点儿时间尝试一下例子,让你对容器适配器的配置有更多的了解。

在使用容器适配器和`DaoAuthenticationProvider`的时候,要确保将`forcePrincipalAsString`属性设置成`true`。

21.3. Jetty

以下代码,在Jetty 4.2.18下通过测试。

`$JETTY_HOME`代表你Jetty安装的根目录。

编辑 `$JETTY_HOME/etc/jetty.xml` 文件,在 `<Configure class>` 部分添加一个新的 `addRealm` 调用:

```
<Call name="addRealm">
<Arg>
  <New
class="org.springframework.security.adapters.jetty.JettySpringSecurityUserRealm">
    <Arg>Spring Powered Realm</Arg>
    <Arg>my_password</Arg>
    <Arg>etc/acegisecurity.xml</Arg>
  </New>
</Arg>
</Call>
```

把 acegisecurity.xml 复制到 $JETTY_HOME/etc目录下。

把下面的文件复制到 $JETTY_HOME/ext目录下:

- aopalliance.jar
- commons-logging.jar
- spring.jar
- acegi-security-jetty-XX.jar
- commons-codec.jar
- burlap.jar
- hessian.jar

上面那些JAR文件(或者是acegi-security-XX.jar)都不能放在你程序的WEB-INF/lib目录下。 你的web.xml里设置的realm名字是与Jetty对应的。 web.xml必须使用与你的jetty.xml里相同的<realm-name>(像上面的例子里,"Spring Powered Realm")。

21.4. JBoss

下面的代码在JBoss 3.2.6下通过了测试。

$JBOSS_HOME代表你JBoss安装的根目录。

有两个不同方法,把spring环境集成到Jboss里。

第一种方法是,修改你的 $JBOSS_HOME/server/your_config/conf/login-config.xml 文件,这样它就在<Policy>部分包含了新的入口:

```
<application-policy name = "SpringPoweredRealm">
<authentication>
  <login-module code =
"org.springframework.security.adapters.jboss.JbossSpringSecurityLoginModule"
        flag = "required">
    <module-option name = "appContextLocation">acegisecurity.xml</module-option>
    <module-option name = "key">my_password</module-option>
 </login-module>
</authentication>
</application-policy>
```

把 acegisecurity.xml 复制到 $JBOSS_HOME/server/your_config/conf目录下。

在配置文件 acegisecurity.xml里包含了spring环境的定义,包含了所有认证管理的bean。 我们必须注意,SecurityContext会在每次请求时被创建销毁,所以登录操作会造成很大的资源消耗。 可选的第二个方法通过org.springframework.beans.factory.access.SingletonBeanFactoryLocator使用Spring单例。 需要的配置方法如下:

```
<application-policy name = "SpringPoweredRealm">
<authentication>
```

```
  <login-module code =
"org.springframework.security.adapters.jboss.JbossSpringSecurityLoginModule"
        flag = "required">
    <module-option name = "singletonId">springRealm</module-option>
    <module-option name = "key">my_password</module-option>
    <module-option name = "authenticationManager">authenticationManager</module-
option>
 </login-module>
</authentication>
</application-policy>
```

上面的代码片段， authenticationManager是一个助手属性,定义成期望的AuthenticationManager的名字,在IoC容器里有多个定义的时候可以用到。 这个singletonId属性引用了定义在beanRefFactory.xml中的一个bean。 这个文件需要放在JBoss的classpath中,包括$JBOSS_HOME/server/your_config/conf。 这个beanRefFactory.xml包含了下面的声明:

```
<beans>
<bean id="springRealm" singleton="true" lazy-init="true"
class="org.springframework.context.support.ClassPathXmlApplicationContext">
<constructor-arg>
  <list>
    <value>acegisecurity.xml</value>
  </list>
</constructor-arg>
</bean>
</beans>
```

最后,无论使用了哪种配置方法,你需要把下面的文件复制到 $JBOSS_HOME/server/your_config/lib目录下:

- aopalliance.jar
- spring.jar
- acegi-security-jboss-XX.jar
- commons-codec.jar
- burlap.jar
- hessian.jar

上面那些JAR文件(或者是acegi-security-XX.jar)都不能放在你程序的WEB-INF/lib目录下。 你的web.xml里设置的realm名字是与JBoss对应的。 然而,你web应用的WEB-INF/jboss-web.xml里的<security-domain>应该与你的login-config.xml中的内容一样。 比如,你的jboss-web.xml看起来应该是这样:

21.5. Resin

下面的代码在Resin 3.0.6下通过测试。

$RESIN_HOME代表你Resin安装的根目录。

Resin提供了好几种方法实现容器适配器。 在下面的教程里,我们使用了尽量与其他容器适配器一致的配置。 这会让Resin用户更容易发布同样的程序,确保配置正确。 开发者对Resin感到舒服,自然可以使用它的方法,打包JAR同web程序自己,和/或支持单点登录。

把下面的文件复制到 $RESIN_HOME/lib目录下:

- aopalliance.jar
- commons-logging.jar
- spring.jar
- acegi-security-resin-XX.jar
- commons-codec.jar

- burlap.jar
- hessian.jar

与其他容器适配器使用的,容器范围的 acegisecurity.xml文件不同,每个Resin web程序会包含自己的WEB-INF/resin-acegisecurity.xml文件。 每个web应用会包含一个resin-web.xml文件,Resin用它来启动容器适配器:

```
<web-app>
<authenticator>
<type>org.springframework.security.adapters.resin.ResinAcegiAuthenticator</type>
<init>
  <app-context-location>WEB-INF/resin-acegisecurity.xml</app-context-location>
  <key>my_password</key>
</init>
</authenticator>
</web-app>
```

像上面提供的基本配置那样,上面那些JAR文件(或者是acegi-security-XX.jar)都不能放在你程序的WEB-INF/lib目录下。 你的web.xml里设置的realm名字是与Resin无关的,认证的类根据<authenticator>设置的。

21.6. Tomcat

下面的代码在Jakarta Tomcat 4.1.30 和 5.0.19通过的了测试。

$CATALINA_HOME代表你的 Catalina (Tomcat) 安装根目录。

编辑你的 $CATALINA_HOME/conf/server.xml 文件,让 <Engine> 部分只包含一个活动的 <Realm> 入口。一个realm入口的例子如下:

```
      <Realm

className="org.springframework.security.adapters.catalina.CatalinaSpringSecurityUserRealm'

        appContextLocation="conf/acegisecurity.xml"
         key="my_password" />
```

确认从你的 <Engine>部分删除了其他的 <Realm> 入口。

把 acegisecurity.xml 复制到$CATALINA_HOME/conf目录下。

把 spring-security-catalina-XX.jar 复制到$CATALINA_HOME/server/lib 目录下。

吧下面的文件复制到 $CATALINA_HOME/common/lib目录下:

- aopalliance.jar
- spring.jar
- commons-codec.jar
- burlap.jar
- hessian.jar

上面那些JAR文件(或者是acegi-security-XX.jar)都不能放在你程序的WEB-INF/lib目录下。 你的web.xml里设置的realm名字是与Catalina无关的。

我们收到了在Mac OS X下使用这个容器适配器的问题报告。需要使用下面的脚本:

```
#!/bin/sh
export CATALINA_HOME="/Library/Tomcat"
export JAVA_HOME="/Library/Java/Home"
cd /
$CATALINA_HOME/bin/startup.sh
```

最后,重启tomcat。

Prev | Home | Next
Chapter 20. 替换验证身份 | Sponsored by SpringSource | Appendix A. 安全数据库表结构



安全数据库表结构

可以为框架采用不同的数据库结构,这个附录为所有功能提供了一种参考形式。 你只要为需要的功能部分提供对应的表结构。

这些DDL语句都是对应于HSQLDB数据库的。 你可以把它们当作一个指南,参照它,在你使用的数据库中定义表结构。

A.1. User表

`UserDetailsService`的标准JDBC实现,需要从这些表里读取用户的密码,帐号信息(可用或禁用)和权限(角色)列表。

```
  create table users(
      username varchar_ignorecase(50) not null primary key,
      password varchar_ignorecase(50) not null,
      enabled boolean not null);

  create table authorities (
      username varchar_ignorecase(50) not null,
      authority varchar_ignorecase(50) not null,
      constraint fk_authorities_users foreign key(username) references
users(username));
      create unique index ix_auth_username on authorities (username,authority);;
```

A.1.1. 组权限

Spring Security 2.0支持了权限分组

```
create table groups (
  id bigint generated by default as identity(start with 0) primary key,
  group_name varchar_ignorecase(50) not null);

create table group_authorities (
  group_id bigint not null,
  authority varchar(50) not null,
  constraint fk_group_authorities_group foreign key(group_id) references
groups(id));

create table group_members (
  id bigint generated by default as identity(start with 0) primary key,
  username varchar(50) not null,
  group_id bigint not null,
  constraint fk_group_members_group foreign key(group_id) references groups(id));
```

A.2. 持久登陆(Remember-Me)表

这个表用来保存安全性更高的持久登陆remember-me实现所需要的数据。 如果你直接或通过命名空间使用了`JdbcTokenRepositoryImpl`,你就会需要这些表结构。

```
create table persistent_logins (
  username varchar(64) not null,
  series varchar(64) primary key,
  token varchar(64) not null,
  last_used timestamp not null);
```

A.3. ACL表

这些表对应Spring Security的ACL实现。

```
create table acl_sid (
  id bigint generated by default as identity(start with 100) not null primary key,
  principal boolean not null,
  sid varchar_ignorecase(100) not null,
  constraint unique_uk_1 unique(sid,principal) );

create table acl_class (
  id bigint generated by default as identity(start with 100) not null primary key,
  class varchar_ignorecase(100) not null,
  constraint unique_uk_2 unique(class) );

create table acl_object_identity (
  id bigint generated by default as identity(start with 100) not null primary key,
  object_id_class bigint not null,
  object_id_identity bigint not null,
  parent_object bigint,
  owner_sid bigint,
  entries_inheriting boolean not null,
  constraint unique_uk_3 unique(object_id_class,object_id_identity),
  constraint foreign_fk_1 foreign key(parent_object)references
acl_object_identity(id),
  constraint foreign_fk_2 foreign key(object_id_class)references acl_class(id),
  constraint foreign_fk_3 foreign key(owner_sid)references acl_sid(id) );

create table acl_entry (
  id bigint generated by default as identity(start with 100) not null primary key,
  acl_object_identity bigint not null,ace_order int not null,sid bigint not null,
  mask integer not null,granting boolean not null,audit_success boolean not null,
  audit_failure boolean not null,constraint unique_uk_4
unique(acl_object_identity,ace_order),
  constraint foreign_fk_4 foreign key(acl_object_identity) references
acl_object_identity(id),
  constraint foreign_fk_5 foreign key(sid) references acl_sid(id) );
```



Part IV. 授权

Spring Security的高级授权能力,为它的受欢迎成都提供了一个更可信服的原因。 无论你选择如何认证-是否使用Spring Security提供的机制和供应器,或整合容器或非Spring Security认证权限-你可以找到授权服务,可以在你的程序用通过统一和简单的方式使用。

在这部分,我们将探索不同的 `AbstractSecurityInterceptor` 实现,他们在第一部分都介绍过了。 我们再继续研究如何使用授权,通过使用领域访问对象列表。

Prev | Home | Next
Appendix A. 安全数据库表结构 | Sponsored by SpringSource | Chapter 22. 通用授权概念



通用授权概念

22.1. 授权

在认证部分简略提过了,所有的Authentication实现需要保存在一个GrantedAuthority对象数组中。 这就是赋予给主体的权限。 GrantedAuthority对象通过AuthenticationManager保存到 Authentication对象里,然后从AccessDecisionManager读出来,进行授权判断。

GrantedAuthority是一个只有一个方法接口:

```
 String getAuthority();
```

这个方法允许 AccessDecisionManager获得一个精确的 String 来表示 GrantedAuthority。 通过返回的String,一个GrantedAuthority可以简单的用大多数AccessDecisionManager“读取”。 如果GrantedAuthority不能表示为一个String,GrantedAuthority会被看作是“复杂的”,然后返回null。

一个“复杂的” GrantedAuthority 的例子会是保存了操作列表和授权信息并应用在不同的客户帐号数值的一个实现。如果要显示这种复杂的GrantedAuthority,把转换成String是非常复杂的,getAuthority()方法的结果应该返回null。这会展示给任何AccessDecisionManager,它需要特别支持GrantedAuthority的实现,来了解它的内容。

Spring Security包含一个具体的 GrantedAuthority 实现,GrantedAuthorityImpl。 这允许任何用户指定的String转换成GrantedAuthority。 所有AuthenticationProvider包含安全结构,它使用GrantedAuthorityImpl组装Authentication对象。

22.2. 处理预调用

像我们在技术概述一章看到的那样,Spring Security提供了一些拦截器,来控制对安全对象的访问权限,例如方法调用或web请求。 一个是否允许执行调用的预调用决定,是由AccessDecisionManager实现的。

22.2.1. AccessDecisionManager

这个 AccessDecisionManager 被AbstractSecurityInterceptor调用,它用来作最终访问控制的决定。 这个AccessDecisionManager接口包含三个方法:

```
 void decide(Authentication authentication, Object secureObject,
ConfigAttributeDefinition config) throws AccessDeniedException;
 boolean supports(ConfigAttribute attribute);
 boolean supports(Class clazz);
```

从第一个方法可以看出来,AccessDecisionManager使用方法参数传递所有信息,这好像在认证评估时进行决定。 特别是,在真实的安全方法期望调用的时候,传递安全Object启用那些参数。 比如,让我们假设安全对象是一个MethodInvocation。 很容易为任何Customer参数查询MethodInvocation,,然后在AccessDecisionManager里实现一些有序的安全逻辑,来确认主体是否允许在那个客户上操作。 如果访问被拒绝,实现将抛出一个AccessDeniedException异常。

这个 supports(ConfigAttribute) 方法在启动的时候被AbstractSecurityInterceptor调用,来决定AccessDecisionManager是否可以执行传递ConfigAttribute。 supports(Class)方法被安全拦截器实现调用,包含安全拦截器将显示的AccessDecisionManager支持安全对象的类型。

22.2.1.1. 基于投票的AccessDecisionManager实现

虽然用户可以实现它自己的AccessDecisionManager来控制所有授权的方面,Spring Security包括很多基于投票 AccessDecisionManager Figure 22.1, “ ”

的 实现。 投票决议管理器 显示有关的类。

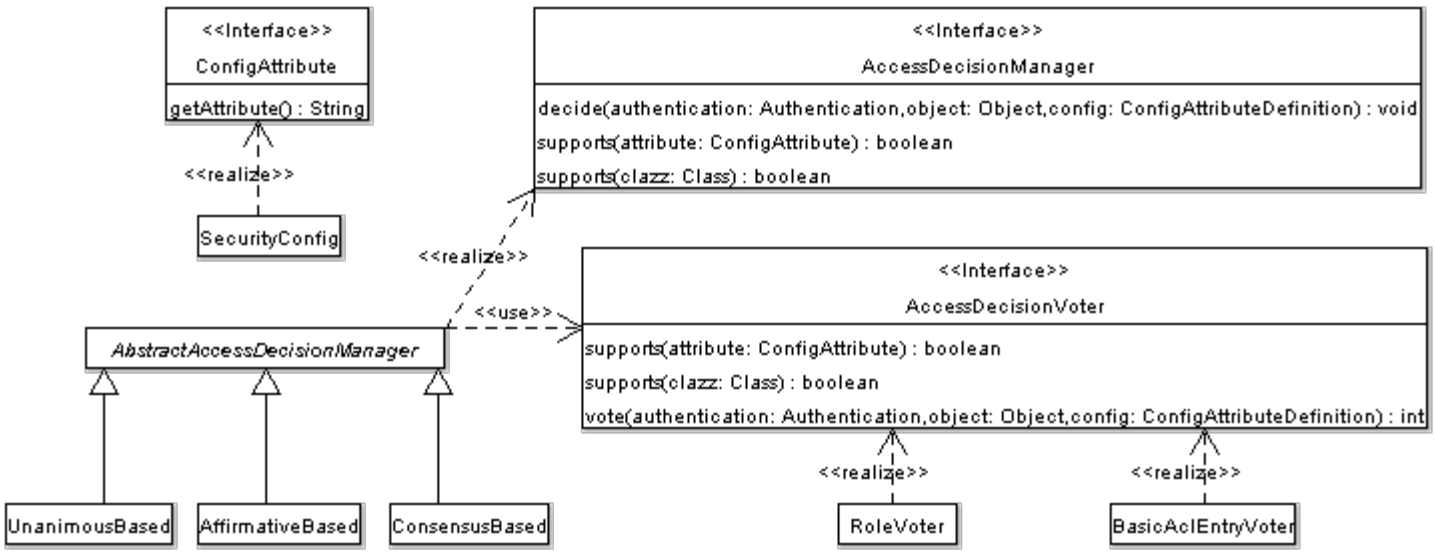


Figure 22.1. 投票决议管理器

使用这种方法,一系列的 AccessDecisionVoter 实现为授权做决定。 这个AccessDecisionManager会决定是否基于它的投票评估抛出AccessDeniedException异常。

AccessDecisionVoter 接口有三个方法:

```
int vote(Authentication authentication, Object object, ConfigAttributeDefinition
config);
boolean supports(ConfigAttribute attribute);
boolean supports(Class clazz);
```

具体实现返回一个int,使用可能的反映的AccessDecisionVoter静态属性ACCESS_ABSTAIN,ACCESS_DENIED和ACCESS_GRANTED。 如果一个投票实现没有选择授权决定,会返回ACCESS_ABSTAIN。 如果它进行过抉择,它必须返回ACCESS_DENIED或ACCESS_GRANTED。

这儿有三个由Spring Security提供的具体 AccessDecisionManager,可以进行投票。 ConsensusBased实现会授权,或拒绝访问,基于没有放弃的那些投票的共识。 那些属性在平等投票时间上被提供来控制行为,或如果所有投票都是弃权了。 AffirmativeBased实现会授予访问权限,如果收到一个或多个ACCESS_GRANTED投票(比如,一个反对投票会被忽略,如果这里至少有一个赞成票)。 像ConsensusBased实现一样,这里有一个参数控制如果所有投票都弃权的行为。 UnanimousBased提供器希望一致的ACCESS_GRANTED,来允许访问,忽略弃权。 如果这里有任何一个ACCESS_DENIED投票,它会拒绝访问。 像其他实现一样,有一个参数控制如果所有投票都弃权的行为。

有可能实现一个自定义的AccessDecisionManager进行不同的投票统计。 比如,投票一个特定的AccessDecisionVoter可能获得更多的权重,这样一个拒绝票对特定的投票者可能有否决权的效果。

22.2.1.1.1. RoleVoter

Spring Security中最常用到的AccessDecisionVoter实现是简单的RoleVoter,它把简单的角色名称作为配置属性,如果用户分配了某个角色就被允许访问。

如果有任何一个配置属性是以ROLE_开头的,就可以进行投票。 如果GrantedAuthority返回的String内容(通过getAuthority()方法),与一个或多个以ROLE_开头的ConfigAttributes完全相同的话,就表示允许访问。 如果没有匹配任何一个以ROLE_开头的ConfigAttribute,RoleVoter就会拒绝访问。 如果没有以ROLE_开头的ConfigAttribute,投票者就会弃权。 RoleVoter在匹配的时候是大小写敏感的,这也包括对 ROLE_这个前缀。

22.2.1.1.2. Custom Voters

也可能实现一个自定义的AccessDecisionVoter。 Spring Security的单元测试提供了很多例子,包括ContactSecurityVoter和DenyVoter。 如果CONTACT_OWNED_BY_CURRENT_USER的ConfigAttribute没有找到,ContactSecurityVoter在投票决定的时候就会弃权。 如果投票,它通过MethodInvocation来确认Contact对象的主体,这是方法调用的主体。 如果Contact主体匹配Authentication对象中表现的主体,它就会投赞成票。 它可能对Contact主体有一个简单的比较,使用一些Authentication表现的GrantedAuthority。 所有这些对应不多几行代码,演示授权模型的灵活性。

22.3. 处理后决定

虽然 AccessDecisionManager 被AbstractSecurityInterceptor在执行安全方法调用之前调用,一些程序需要一个方法来修改安全对象调用返回的对象。 虽然你可以简单实现自己的AOP涉及实现,Spring Security提供有许多具体实现方面调用,集成它的ACL功能。

Figure 22.2, "后决定实现" 展示Spring Security的 AfterInvocationManager 和它的具体实现。

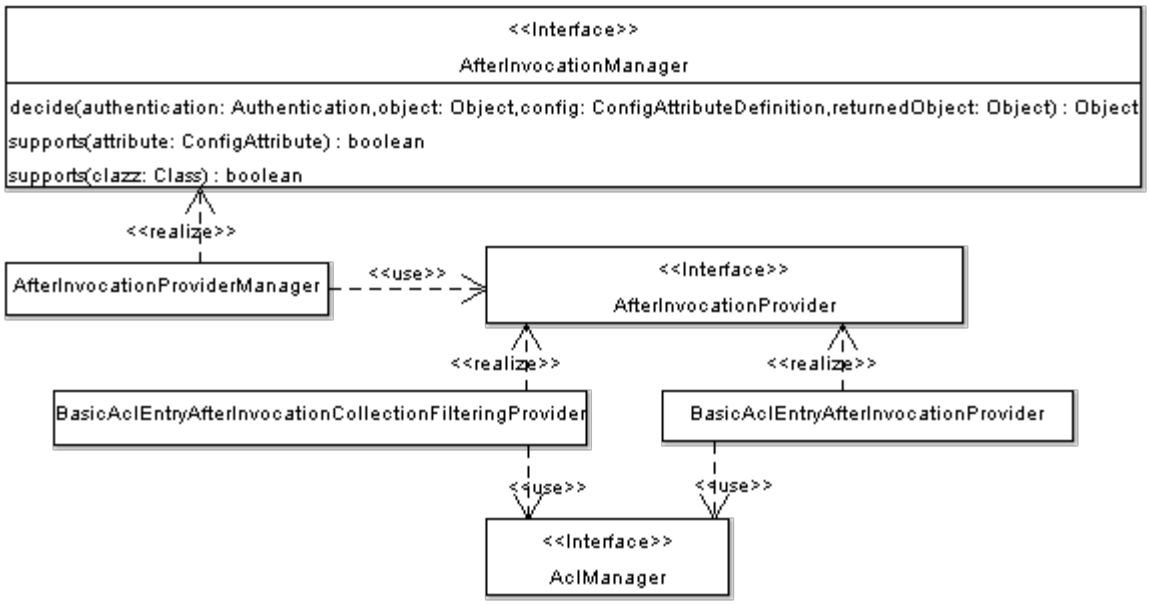


Figure 22.2. 后决定实现

就像Spring Security的其他很多部分一样,AfterInvocationManager有一个单独的具体实现,AfterInvocationProviderManager有AfterInvocationProvider列表。 每个AfterInvocationProvider被允许修改返回对象,或抛出AccessDeniedException异常。 确实多个提供器可以修改对象,作为之前提供器的结果传递给队列的下一个。 让我们现在考虑我们的AfterInvocationProvider的ACL提醒实现。

请注意,如果你使用 AfterInvocationManager,你将依然需要配置属性,让MethodSecurityInterceptor的AccessDecisionManager允许一个操作。 如果你使用典型的Spring Security包含AccessDecisionManager实现,没有在特定的安全方法调用上配置属性定义,会导致AccessDecisionVoter弃权。 这里,如果AccessDecisionManager的"allowIfAllAbstainDecisions"属性是false,会抛出一个AccessDeniedException异常。 你可以避免这个潜在的问题,使用(i)把"allowIfAllAbstainDecisions" 设置为true(虽然通常不建议这么做),或者(ii)直接确保这里至少有一个配置属性,这样AccessDecisionVoter会投赞成票。后者(推荐)方法通常使用ROLE_USER或ROLE_AUTHENTICATED配置属性。

22.3.1. ACL提醒后决议提供器

请注意:Acegi Security 1.0.3包含一个新ACL模型预览。 新ACL模型特别重写了以前的ACl模块。 新模块可以在org.springframework.security.acls包里找到,老ACL模块放在org.springframework.security.acl。 我们推荐

用户考虑测试一下新ACL模块,使用它建立程序。 老ACL模型应该考虑废弃了,可能在未来版本中删除。 下面的信息是关于新ACL包的,因此也是推荐的。

我们经常在服务层写的方法就像这样:

```
public Contact getById(Integer id);
```

常常,只有主体允许读取Contact,就应该允许获得它。 在这情况,AccessDecisionManager方法由AbstractSecurityInterceptor提供将不够了。 这是因为Contact的身份,在调用安全对象之前是完全有效地。AclAfterInvocationProvider提供了一个解决方法,像下面这样配置:

```
<bean id="afterAclRead"

class="org.springframework.security.afterinvocation.AclEntryAfterInvocationProvider">

  <constructor-arg ref="aclService"/>
  <constructor-arg>
    <list>
      <ref
local="org.springframework.security.acls.domain.BasePermission.ADMINISTRATION"/>
      <ref local="org.springframework.security.acls.domain.BasePermission.READ"/>
    </list>
  </constructor-arg>
</bean>
```

在上面的例子里,Contact需要重新审核,并传递给AclEntryAfterInvocationProvider。 提供器会抛出AccessDeniedException异常,如果requirePermission列表的其中一个没有放在Authentication里。AclEntryAfterInvocationProvider查询Acl 服务,为Authentication对应的领域对象决定应用ACL。

一个简单的 AclEntryAfterInvocationProvider 是 AclEntryAfterInvocationCollectionFilteringProvider。 它用来从主体不允许访问的情况删除Collection或元素数据,。 它永远不会抛出AccessDeniedException - 直接安静的删除不允许的元素。 提供器如下配置:

```
<bean id="afterAclCollectionRead"

class="org.springframework.security.afterinvocation.AclEntryAfterInvocationCollectionFilte

  <constructor-arg ref="aclService"/>
  <constructor-arg>
    <list>
      <ref
local="org.springframework.security.acls.domain.BasePermission.ADMINISTRATION"/>
      <ref local="org.springframework.security.acls.domain.BasePermission.READ"/>
    </list>
  </constructor-arg>
</bean>
```

像你可以想像的,返回的Object必须是让提供器可以操作的Collection或数组。 如果AclManager分析Authentication没有获得一个队列中的requirePermission,它就会删除元素。

Contacts实例程序演示这两个 AfterInvocationProvider。

22.3.2. ACL-Aware AfterInvocationProviders (老 ACL 模块)

请注意:Acegi Security 1.0.3包含一个新ACL模型预览。 新ACL模型显著的重写了以前的ACl模块。 新模块可以在org.springframework.security.acls包里找到,老ACL模块放在org.springframework.security.acl。 我们推荐用户考虑测试一下新ACL模块,使用它建立程序。 老ACL模型应该考虑废弃了,可能在未来版本中删除。

常用服务层方法,我们所有写到一个状态或另一个像这样:

```
public Contact getById(Integer id);
```

常常,只有主体允许读取Contact,应该允许获得它。 在这情况,AccessDecisionManager方法由AbstractSecurityInterceptor提供将不够了。 这是因为Contact的身份,在调用安全对象之前是完全有效地。AclAfterInvocationProvider提供了一个解决方法,它像下面这样配置:

```
public Contact getById(Integer id);
```

常常,只有主体允许读取Contact,应该允许获得它。 在这情况,AccessDecisionManager方法由AbstractSecurityInterceptor提供将不够了。 这是因为Contact的身份,在调用安全对象之前是完全有效地。AclAfterInvocationProvider提供了一个解决方法,它像下面这样配置:

```
<bean id="afterAclRead"

class="org.springframework.security.afterinvocation.BasicAclEntryAfterInvocationProvider">

  <property name="aclManager" ref="aclManager"/>
  <property name="requirePermission">
    <list>
      <ref
local="org.springframework.security.acl.basic.SimpleAclEntry.ADMINISTRATION"/>
      <ref local="org.springframework.security.acl.basic.SimpleAclEntry.READ"/>
    </list>
  </property>
</bean>
```

在上面的例子里,Contact需要重新审核,并传递给AclEntryAfterInvocationProvider。 提供器会抛出AccessDeniedException异常,如果requirePermission列表的其中一个没有放在Authentication里。AclEntryAfterInvocationProvider查询Acl 服务,为Authentication对应的领域对象决定应用ACL。

一个简单的 AclEntryAfterInvocationProvider 是 AclEntryAfterInvocationCollectionFilteringProvider。 它用来从主体不允许访问的情况删除Collection或元素数据。 它永远不会抛出AccessDeniedException - 直接安静的删除不允许的元素。 提供器如下配置:

```
<bean id="afterAclCollectionRead"

class="org.springframework.security.afterinvocation.BasicAclEntryAfterInvocationCollection

  <property name="aclManager" ref="aclManager"/>
  <property name="requirePermission">
    <list>
      <ref
local="org.springframework.security.acl.basic.SimpleAclEntry.ADMINISTRATION"/>
      <ref local="org.springframework.security.acl.basic.SimpleAclEntry.READ"/>
    </list>
  </property>
</bean>
```

像你可以想像的,返回的Object必须是Collection或数组,让提供器可以操作。 它会删除元素,如果AclManager分析Authentication没有获得一个队列中的requirePermission。

Contacts实例程序演示这两个 AfterInvocationProvider。

22.4. 授权标签库

AuthorizeTag用来包括内容,如果当前主体拥有确定的GrantedAuthority。

下面的JSP片段展示如何使用 AuthorizeTag:

```
<security:authorize ifAllGranted="ROLE_SUPERVISOR">
<td>
  <a href="del.htm?id=<c:out value="${contact.id}"/>">Del</a>
</td>
```

```
</security:authorize>
```

如果主体拥有ROLE_SUPERVISOR权限,这个标签就会显示标签的内容A。

security:authorize 标签声明下面的属性:

- ifAllGranted:这个标签列出的所有角色必须授权,才能输出标签的内容。
- ifAnyGranted:任何一个这个标签列出的角色必须授权,才能输出标签的内容。
- ifNotGranted:这个标签列出的角色没有一个是授权的,才能输出标签的内容。

你会注意到,每个属性你可以使用多个角色。 简单使用逗号来分隔角色。 这个authorize标签忽略属性之间的空格。

标签库逻辑和所有它的参数一起。 这意味着如果你结合两个或多个属性,这个标签所有属性必须是true,才会输出内容。 不要在ifNotGranted="ROLE_SUPERVISOR"后面添加 ifAllGranted="ROLE_SUPERVISOR",否则你会奇怪为什么总也看不到标签的内容。

如果想要所有的属性都返回true,授权标签允许你创建更复杂的授权场景。 比如,你可以声明ifAllGranted="ROLE_SUPERVISOR",并在同一个标签里使用ifNotGranted="ROLE_NEWBIE_SUPERVISOR",可以防止新的超级用户看到标签内容。 然而你无疑可以直接使用ifAllGranted="ROLE_EXPERIENCED_SUPERVISOR",而不是在你的设计里添加NOT条件。

最后一件事:标签验证授权有特定的顺序:首先是ifNotGranted,然后是ifAllGranted,最后是if AnyGranted。

AccessControlListTag用来包含内容,判断当前的主体是否有一个ACL来显示领域对象。

下面的JSP片段,演示如何使用 AccessControlListTag:

```
<security:accesscontrollist domainObject="${contact}" hasPermission="8,16">
<td><a href="<c:url value="del.htm"><c:param name="contactId"
value="${contact.id}"/></c:url>">Del</a></td>
</security:accesscontrollist>
```

这标签会导致标签内容显示出来,如果主体拥有对于"contact"领域对象授权16或者授权1。 这个数值其实是证书,使用BasePermission位掩码。 请参考参考指南的ACL部分,了解更多Spring Security的ACL知识。

AclTag 是老ACL模型的一部分,应该考虑废弃。因为一些历史问题,它与AccessControlListTag的作用相同。



安全对象实现

23.1. AOP联盟 (MethodInvocation) 安全拦截器

在Spring Security 2.0之前,安全MethodInvocation需要很多锅炉板配置. 现在推荐的方式,为方法安全,使用命名空间配置. 这个方式,方法安全基础设置bean,自动配置为你,这样你不需要真的需要知道实现类。 我们只需要提供一个快速概述对这些类,在这里提到了。

方法安全提升,使用 MethodSecurityInterceptor,它会保障 MethodInvocation。 依靠配置方法,一个拦截器,可能指定一个单独的bean,或在多个bean之间共享。 拦截器使用MethodDefinitionSource实例获得配置属性,应用特定的方法调用。 MapBasedMethodDefinitionSource用来保存配置属性,关键字,通过方法名(也可以是通配符),会被用在内部,在属性定义在application context里,使用<intercept-methods> 或<protect-point>元素。 其他实现会用来处理基于注解的配置。

23.1.1. 精确的 MethodSecurityIterceptor 配置

你可以,当然,配置一个 MethodSecurityIterceptor 直接在你的application context里,为使用一个Spring AOP代理机制:

```
<bean id="bankManagerSecurity"

class="org.springframework.security.intercept.method.aopalliance.MethodSecurityInterceptor

  <property name="authenticationManager" ref="authenticationManager"/>
  <property name="accessDecisionManager" ref="accessDecisionManager"/>
  <property name="afterInvocationManager" ref="afterInvocationManager"/>
  <property name="objectDefinitionSource">
    <value>
      org.springframework.security.context.BankManager.delete*=ROLE_SUPERVISOR

org.springframework.security.context.BankManager.getBalance=ROLE_TELLER,ROLE_SUPERVISOR

    </value>
  </property>
</bean>
```

23.2. AspectJ (JoinPoint) 安全拦截器

AspectJ 安全拦截器相对于,上面讨论的AOP联盟安全拦截器,就非常简单了。 事实上,我们这节只讨论不同的部分。

AspectJ 拦截器的名字是 AspectJSecurityInterceptor。 与AOP联盟安全拦截器不同,在Spring的application context中的安全拦截器通过代理织入,AspectJSecurityInterceptor是通过AspectJ编译器织入。 在一个系统里使用两种类型的安全拦截器也是常见的,使用AspectJSecurityInterceptor处理领域对象实例的安全,AOP联盟MethodSecurityInterceptor用来处理服务层安全。

让我们首先考虑如何把AspectJSecurityInterceptor配置到spring的application context里:

```
<bean id="bankManagerSecurity"

class="org.springframework.security.intercept.method.aspectj.AspectJSecurityInterceptor">

  <property name="authenticationManager" ref="authenticationManager"/>
  <property name="accessDecisionManager" ref="accessDecisionManager"/>
  <property name="afterInvocationManager" ref="afterInvocationManager"/>
  <property name="objectDefinitionSource">
    <value>
```

```
        org.springframework.security.context.BankManager.delete*=ROLE_SUPERVISOR

org.springframework.security.context.BankManager.getBalance=ROLE_TELLER,ROLE_SUPERVISOR

    </value>
</property>
</bean>
```

像你看到的,类名的部分,AspectJSecurityInterceptor其实与AOP联盟安全拦截器一样。 实际上,两个拦截器共享同样的objectDefinitionSource,ObjectDefinitionSource运行的时候使用java.lang.reflect.Method而不是AOP库特定的类。 当然,你的访问表决,相对AOP库指定的对象(比如MethodInvocation 或 JoinPoint),好像可以考虑更精确范围,在使用防伪决议(比如方法参数)。

下一步,你需要定义一个 AspectJ 切面。比如:

```
package org.springframework.security.samples.aspectj;

import
org.springframework.security.intercept.method.aspectj.AspectJSecurityInterceptor;
import org.springframework.security.intercept.method.aspectj.AspectJCallback;
import org.springframework.beans.factory.InitializingBean;

public aspect DomainObjectInstanceSecurityAspect implements InitializingBean {

private AspectJSecurityInterceptor securityInterceptor;

pointcut domainObjectInstanceExecution(): target(PersistableEntity)
      && execution(public * *(..)) && !within(DomainObjectInstanceSecurityAspect);

Object around(): domainObjectInstanceExecution() {
  if (this.securityInterceptor == null) {
    return proceed();
  }

  AspectJCallback callback = new AspectJCallback() {
      public Object proceedWithObject() {
        return proceed();
      }
  };

  return this.securityInterceptor.invoke(thisJoinPoint, callback);
}

public AspectJSecurityInterceptor getSecurityInterceptor() {
  return securityInterceptor;
}

public void setSecurityInterceptor(AspectJSecurityInterceptor securityInterceptor) {
  this.securityInterceptor = securityInterceptor;
}

public void afterPropertiesSet() throws Exception {
  if (this.securityInterceptor == null)
    throw new IllegalArgumentException("securityInterceptor required");
  }
}
```

在上面例子里,安全拦截器会作用在每一个PersistableEntity实例上,这是没提到过的一个抽象类(你可以使用任何其他的类或你喜欢的切点)。 对于那些情况AspectJCallback需要,因为proceed();语句,有特定的含义,只有在around()内容中。 AspectJSecurityInterceptor调用这个匿名AspectJCallback类,在它想继续目标对象时。

你会需要配置Spring读取切面,织入到AspectJSecurityInterceptor中。 下面的声明会处理这个:

```
<bean id="domainObjectInstanceSecurityAspect"

class="org.springframework.security.samples.aspectj.DomainObjectInstanceSecurityAspect"
```

```
       factory-method="aspectOf">
<property name="securityInterceptor"><ref
bean="aspectJSecurityInterceptor"/></property>
</bean>
```

就是这个了!现在你可以在你的系统里任何地方创建bean了,无论用什么意思你想到的(比如new Person();),他们会被安全拦截器应用。

23.3. FilterInvocation 安全拦截器

为了保护 FilterInvocation,开发者需要添加 FilterSecurityInterceptor 到它们的过滤器链。 典型配置例子如下:

在 application context 你需要配置三个bean:

```
<bean id="exceptionTranslationFilter"
       class="org.springframework.security.ui.ExceptionTranslationFilter">
  <property name="authenticationEntryPoint" ref="authenticationEntryPoint"/>
</bean>

<bean id="authenticationEntryPoint"

class="org.springframework.security.ui.webapp.AuthenticationProcessingFilterEntryPoint">

  <property name="loginFormUrl" value="/acegilogin.jsp"/>
  <property name="forceHttps" value="false"/>
</bean>

<bean id="filterSecurityInterceptor"
       class="org.springframework.security.intercept.web.FilterSecurityInterceptor">
  <property name="authenticationManager" ref="authenticationManager"/>
  <property name="accessDecisionManager" ref="accessDecisionManager"/>
  <property name="objectDefinitionSource">
    <security:filter-invocation-definition-source>
      <security:intercept-url pattern="/secure/super/**"
access="ROLE_WE_DONT_HAVE"/>
      <security:intercept-url pattern="/secure/**"
access="ROLE_SUPERVISOR,ROLE_TELLER"/>
    </security:filter-invocation-definition-source>
  </property>
</bean>
```

ExceptionTranslationFilter 提供了Java异常和HTTP响应之间的桥梁。 他们与维护的用户接口之间是完全独立的。 过滤器没有执行任何真实的安全提升。 如果一个 AuthenticationException 被检测到,过滤器会调用 AuthenticationEntryPoint 扎展开认证过程(比如一个用户登录)。

AuthenticationEntryPoint 会被调用,如果用户请求受保护的HTTP请求,但是他们没有认证。 类处理表现对应响应到用户,这样验证可以开始。 三个具体实现由Spring Security提供:AuthenticationProcessingFilterEntryPoint 为展开表单认证, BasicProcessingFilterEntryPoint 为展开HTTP基本认证过程,CasProcessingFilterEntryPoint 为展开JA-SIG中心认证服务(CAS)登录。 AuthenticationProcessingFilterEntryPoint 和 CasProcessingFilterEntryPoint有可选项,可以强制使用HTTPS,所以如果需要它,请参考JavaDocs。

FilterSecurityInterceptor 用来处理HTTP资源的安全。 像其他安全拦截器,它需要引用AuthenticationManager 和 AccessDecisionManager,它们都会在下面的单独章节里进行讨论。 FilterSecurityInterceptor也使用配置属性进行配置,应用不同的HTTP URL请求。 一个对配置属性的完全讨论,提供在这份文档的高级设计章节中。

FilterSecurityInterceptor 可以使用两种方式配置配置属性。 第一种,在上面演示了,使用<filter-invocation-definition-source>命名元素。 它直接使用 <filter-chain-map>配置一个FilterChainProxy,但是<intercept-url>子元素,只使用pattern和access属性。 第二种,写你自己的ObjectDefinitionSource,虽然这个超越了文档的范围。 不论使用哪种方法,ObjectDefinitionSource用来返回ConfigAttributeDefinition对象,包含所有配置属性,分配给单独的受保护HTTP URL。

应该注意 FilterSecurityInterceptor.setObjectDefinitionSource()方法其实期望一个FilterInvocationDefinitionSource实例。 这是一个继承了ObjectDefinitionSource的标记接口。 它直接提供ObjectDefinitionSource,理解FilterInvocation。 感兴趣简单,我们继续参考FilterInvocationDefinitionSource,像一个ObjectDefinitionSource,大多数用户的FilterSecurityInterceptor的区别都有很小的相关性。

在使用命名空间选项配置拦截器时,的哦好用来分割不同的配置属性,应用在每个HTTP URL。 每个配置属性分配到它自己的SecurityConfig对象里。 SecurityConfig对象在高级设计章中讨论。 ObjectDefinitionSource被属性编辑器创建,FilterInvocationDefinitionSource,匹配配置属性FilterInvocations,基于请求URL的表达式计算。 两个标准表达式语法被支持了。 默认的是处理所有表达式,用Apache Ant路径和正则表达式,也支持ore复杂情况。 path-type属性用来指定使用的模式类型。 不可能在一个定义里使用组合表达式语法。 比如,上一个配置使用正则表达式,代替Ant路径,会写成下面这样:

```
<bean id="filterInvocationInterceptor"
        class="org.springframework.security.intercept.web.FilterSecurityInterceptor">
  <property name="authenticationManager" ref="authenticationManager"/>
  <property name="accessDecisionManager" ref="accessDecisionManager"/>
  <property name="runAsManager" ref="runAsManager"/>
  <property name="objectDefinitionSource">
    <security:filter-invocation-definition-source path-type="regex">
      <security:intercept-url pattern="\A/secure/super/.*\Z"
access="ROLE_WE_DONT_HAVE"/>
      <security:intercept-url pattern="\A/secure/.*\"
access="ROLE_SUPERVISOR,ROLE_TELLER"/>
    </security:filter-invocation-definition-source>
  </property>
</bean>
```

无论使用什么表达式语法,表达式通常执行,根据它们的定义。 因此重要的,更特定的表达式要放在不特定的表达式,在列表的高处。 在我们上面的例子里有提及,更特殊的 /secure/super/ 模式放在,较少特殊 /secure/模型,高的地方。如果它们换了位置,/secure/会一直匹配,/secure/super/永远不会执行的。

像使用其他安全拦截器,validateConfigAttributes属性被观察。 设置成true的时候(默认),在启动的时候FilterSecurityInterceptor 会执行,如果提供的配置属性是有效的。 它检测每个可以执行的配置属性,通过AccessDecisionManager 或 RunAsManager。 如果没有可以执行的配置属性,会抛出一个异常。



领域对象安全

24.1. 概述

请注意:在2.0.0之前,Spring Security还叫Acegi Security。 在老的Acegi Security发布里提供了一个ACL模块,放在 org.[acegisecurity/springsecurity].acl下。 旧包现在被废弃了,会在Spring Security未来发布里删除。 这章提到的新ACL模块,是由Spring Security 2.0.0正式推荐的,可以在org.springframework.security.acls包找到。

复杂程序常常需要定义访问权限,不是简单的web请求或方法调用级别。而是,安全决议需要包括谁(认证),哪里(MethodInvocation)和什么(一些领域对象)。 换而言之,验证决议也需要考虑真实的领域对象实例,方法调用的主体。

想像我们为宠物店设计一个程序。 在你的基于Spring程序里有两个主要的用户组:宠物商店的工作人员和宠物商店的顾客。 工作人员可以访问所有数据,而你的顾客只能看到他自己的数据。 让它更有趣一点儿,你的客户可以允许其他用户看他自己的数据,比如他们"学龄前小狗"教练,或他们本地"小马俱乐部"的负责人。 以Spring Security为基础,我们可以使用很多方法:

- 编写你的业务方法来提升安全。 你可以使用一个集合,包含 Customer领域对象实例,来决定哪个用户可以访问。通过SecurityContextHolder.getContext().getAuthentication(), 你可以得到Authentication对象。
- 编写一个 AccessDecisionVoter 提升安全性,通过保存在Authentication对象里的GrantedAuthority[]。 这意味着你的AuthenticationManager需要使用自定义GrantedAuthority[]组装这个Authentication,处理每个主体访问的Customer领域对象实例。
- 编写一个 AccessDecisionVoter 提升安全性,直接打开目标Customer领域对象。 这意味着你的投票者需要访问一个DAO,允许它重审Customer对象。 它会访问用户提交的Customer对象的集合,然后执行合适的决议。

每个方法都是完全可用的。 然而,你的第一种认证会涉及你的业务代码。 它的主要问题是单元测试困难,也很难在其他地方重用Customer的授权逻辑。 从Authentication获得GrantedAuthority[]也还好,但是不适合大规模数量的Customer。 如果用户可以访问五万个Customer(不是在这个例子里,但是想像一下,如果它是一个大型的小马俱乐部),这么大的内存消耗,和时间消耗,建造Authentication是不可取的。 最后一个方法,直接从外部代码打开Customer,可能是三个中最好的了。 它分离了概念,没有滥用内存或CPU周期,但它还是没什么效率,在AccessDecisionVoter和最终业务方法里,它自己会执行一个DAO响应,来重申 Customer对象。 每个方法调用,都要评估两次,非常不可取。 另外,每个方法列出了你需要,从头写自己访问控制列表(ACL)持久化和业务逻辑。

幸运的是,这里有另一个选择,我们在下面讨论。

24.2. 关键概念

Spring Security的ACL服务放在spring-security-acl-xxx.jar中。 你需要把这个JAR添加到你的classpath下,来使用Spring Security的领域对象实例安全能力。

Spring Security的领域对象实例安全能力其实是一个访问控制列表(ACL)的概念。 在你的系统中每个领域对象实例都有它自己的ACL,然后这个ACL数据信息,谁可以,谁不可以和领域对象工作。 在这种思想下,Spring Security在你的系统提供三个主要的ACL相关能力:

- 一个有效的方法,为所有你的领域对象(修改那些ACL)检索ACL条目。
- 一个方法,在方法调用之前确认给定的主体有权限同你的对象工作。
- 一个方法,在方法调用之后确认给定的主体有权限同你的对象工作(或它返回的什么东西)。

像第一点所示,Spring Security的ACL模块的一个主要能力,是提供高性能的检索ACL。 这个ACL资源能力特别重要,因为在你的系统中每个领域对象实例,可能有多个访问控制条目,每个ACL可能继承其他ACL,像一个树形结构(这是Spring Security支持的,非常常用)。 Spring Security的ACL能力仔细定义来支持高性能检索ACL,可插拔缓存,最小死锁数据库更新,不依赖ORM(我们直接使用的JDBC),适当封装,数据库透明更新。

给定的数据库是ACL模块操作的中心,让我们来看看默认实现使用的四个主要表。 下面介绍的这些表,为了Spring Security ACL的部署,使用的表在最后列出:

- ACL_SID让我们定义系统中唯一主体或授权(“SID”意思是“安全标识”)。 它包含的列有ID,一个文本类型的SID,一个标志,用来表示是否使用文本显示引用的主体名或一个GrantedAuthority。 因此,对每个唯一的主体或GrantedAuthority都有单独一行。 在使用获得授权的环境下,一个SID通常叫做"recipient"授予者。
- ACL_CLASS 让我们在系统中确定唯一的领域对象类。包含的列有ID和java类名。 因此,对每个我们希望保存ACL权限的类都有单独一行。
- ACL_OBJECT_IDENTITY 为系统中每个唯一的领域对象实例保存信息。 列包括ID,指向ACL_CLASS的外键,唯一标识,所以我们知道为哪个ACL_CLASS实例提供信息,parent,一个外键指向ACL_SID表,展示领域对象实例的拥有者,我们是否允许ACL条目从任何父亲ACL继承。 我们对每个领域对象实例有一个单独的行,来保存ACL权限。
- 最后,ACL_ENTRY保存分配给每个授予者单独的权限。 列包括一个ACL_OBJECT_IDENTITY的外键,recipient(比如一个ACL_SID外键),我们是否通过审核,和一个整数位掩码,表示真实的权限被授权或被拒绝。 我们对于每个授予者都有单独一行,与领域对象工作获得一个权限。

像上一段提到的,ACL系统使用整数位掩码。 不要担心,你不需要知道使用ACL系统位转换的好处,但我们有充足的32位可以转换。 每个位表示一个权限,默认授权是可读(位0),写(位1),创建(位2),删除(位3)和管理(位4)。 如果你希望使用其他权限,很容易实现自己的Permission实例,其他的ACL框架部分不了解你的扩展,依然可以运行。

了解你的系统中领域对象的数量很重要,完全用不害怕我们选择使用整数位掩码的事实。 虽然我们有32位可用来作权限,你可能有几亿领域对象实例(意味着在ACL_OBJECT_IDENTITY表中有几亿行,ACL_ENTRY也很可能是这样)。我们说出这点,因为我们有时发现人们犯错误,决定他们为每个潜在的领域对象提供一位,情况并非如此。

现在我们提供了ACL系统可以做的基本概述,它看起来像一个表结构,现在让我们探讨关键接口。 关键接口是:

- Acl: 每个领域对象有一个,并只有一个Acl对象,它的内部保存着AccessControlEntry,记住这是Acl的所有者。一个Acl不直接引用领域对象,但是作为替代的是使用一个ObjectIdentity。 这个Acl保存在ACL_OBJECT_IDENTITY表里。
- AccessControlEntry: 一个 Acl 里有多个AccessControlEntry,在框架里常常略写成ACE。 每个ACE引用特别的Permission,Sid和Acl。 一个ACE可以授权或不授权,包含审核设置。 ACE保存在ACL_ENTRY表里。
- Permission: 一个 permission 表示特殊不变的位掩码,为位掩码和输出信息提供方便的功能。 上面的基本权限(位0到4)保存在BasePermission类里。
- Sid: 这个 ACL 模块需要引用主体和GrantedAuthority[]。 间接的等级由Sid接口提供,简写成“安全标识”。通常类包含PrincipalSid(表示主体在Authentication里)和GrantedAuthoritySid。 安全标识信息保存在ACL_SID表里。
- ObjectIdentity: 每个领域对象放在ACl模型的内部,使用ObjectIdentity。 默认实现叫做ObjectIdentityImpl。
- AclService: 重审Acl对应的ObjectIdentity。 包含的实现(JdbcAclService),重审操作代理LookupStrategy。 这个LookupStrategy为检索ACL信息提供高优化策略,使用批量检索(BasicLookupStrategy)然后支持自定义实现,和杠杆物化视图,继承查询和类似的表现中心,非ANSI的SQL能力。
- MutableAclService: 允许修改了的 Acl放到持久化中。 如果你不愿意,可以不使用这个接口。

请注意,我们的AclService和对应的数据库类都使用ANSI SQL。 这应该可以在所有的主流数据库上工作。 在写作的时候,系统成功在Hypersonic SQL, PostgreSQL, Microsoft SQL Server 和 Oracle上测试通过。

Spring Security的两个实例演示了ACL模块。 第一个是Contacts实例,另一个是文档管理系统(DMS)实例。 我们建议大家看一看这些例子。

24.3. 开始

为了开始使用Spring Security的ACL功能,你会需要在一些地方保存你的ACL信息。 有必要使用Spring的DataSource实例。 DataSource注入到JdbcMutableAclService和BasicLookupStrategy实例中。 后一个提供了高性能ACL检索能力,前一个提供变异能力。 参考例子之一,使用Spring Security,为一个例子配置。 你也需要使用四个ACL指定的表建立数据库,这写在最后一章(参考ACL实例,查看对应的SQL语句)。

一旦你创建了需要的结构,和JdbcMutableAclService的实例,你下一个需求是确认你的领域模型支持Spring Security ACL包的互操作。 希望的ObjectIdentityImpl会证明足够,它提供可以使用的大量方法。 大部分人会使用领域对象,包含public Serializable getId()方法。 如果返回类型是long或与long兼容(比如int),你会发现你不需要

为ObjectIdentity进行更多考虑。 ACL模块的许多部分对应long标识符。 如果你没有使用long(或int, byte等等),你需要重新实现很多类。 我们不倾向在Spring Security ACL模块中支持非long标识符,因为所有数据库序列都支持,最常用的数据类型标识,也可以容纳所有常用场景的足够长度。

下面的代码片段,显示如何创建一个Acl,或修改一个存在的 Acl:

```
// Prepare the information we'd like in our access control entry (ACE)
ObjectIdentity oi = new ObjectIdentityImpl(Foo.class, new Long(44));
Sid sid = new PrincipalSid("Samantha");
Permission p = BasePermission.ADMINISTRATION;

// Create or update the relevant ACL
MutableAcl acl = null;
try {
  acl = (MutableAcl) aclService.readAclById(oi);
} catch (NotFoundException nfe) {
  acl = aclService.createAcl(oi);
}

// Now grant some permissions via an access control entry (ACE)
acl.insertAce(acl.getEntries().length, p, sid, true);
aclService.updateAcl(acl);
```

在上面的例子里,我们检索ACl,分配给"Foo"领域对象,使用数字44作标识。 我们添加一个ACE,这样名叫"Samantha"的主体可以“管理”这个对象。 代码片段是自解释的,除了insertAce方法。 insertAce方法的第一个参数是Acl里新条目被插入的决定位置。 在上面的的例子里,我们只把新ACE放到以存在的ACE的尾部。 最后一个参数是一个布尔值,显示是否ACE授权或拒绝。 大多数时间,是授权(true),如果它是拒绝(false),权限就会被冻结。

Spring Security 没有提供任何特定整合,自动创建,更新,或删除ACL,作为你的DAO的一部分或资源操作。 作为替代的,你会需要像上面一样为你的单独领域对象写代码。 值得考虑在你的服务层使用AOP,来自动继承ACL信息,使用你的服务层操作。 我们发现以前这是一个非常有效的方式。

一旦,你使用上面的技术,在数据库里保存一些ACL信息,下一步是使用ACL信息,作为授权决议逻辑的一部分。 这里你有一大堆选择。 你可以写你自己的AccessDecisionVoter或AfterInvocationProvider,期待在方法调用之前或之后触发。 这些类使用AclService来检索对应的ACL,然后调用Acl.isGranted(Permission[] permission, Sid[] sids, boolean administrativeMode),决定权限是授予还是拒绝。 可选的,你可能使用我们的AclEntryVoter,AclEntryAfterInvocationProvider或AclEntryAfterInvocationCollectionFilteringProvider类。 所有这些类提供一个基于声明的方法,在运行阶段来执行ACL信息,释放你从需要写任何代码。 请参考例子程序,学习更多如何使用这些类。



acegi到spring security的转换方式

作者 Chris.Baker,发表于 2008/04/22 - 9:44am.

http://java.dzone.com/tips/pathway-acegi-spring-security-

以前它叫做spring的acegi安全框架,现在重新标识为spring security 2.0,它实现了简易配置的承诺,提高了开发者的生产力。 它已经是java平台上应用最广的安全框架了,在sourceforge上拥有250,000的下载量,Spring Security 2.0又提供了一系列的新功能。

本文主要介绍了如果把之前建立在acegi基础上的spring应用转换到spring security 2.0上。

25.1. Spring Security是什么

Spring Security是目前用于替换acegi的框架,它提供了一系列新的功能。

- 大为简化了配置
- 继承OpenID,标准单点登录
- 支持windows NTLM,在windows合作网络上实现单点登录
- 支持JSR 250("EJB 3")的安全注解
- 支持AspectJ切点表达式语言
- 全面支持REST Web请求授权
- 长期要求的支持组,层级角色和用户管理API
- 提升了功能,使用后台数据库的remember-me实现
- 通过spring webflow 2.0对web状态和流转授权进行新的支持
- 通过Spring Web Services 1.5加强对WSS(原来的WS-Security)的支持
- 整个更多的.....

25.2. 目标

目前,我工作在一个spring的web应用上,使用acegi控制对资源的访问权限。 用户信息保存在数据库中,我们配置acegi使用了基于JDBC的UserDetails服务。 同样的,我们所有的web资源都保存在数据库里,acegi配置成使用自定义的AbstractFilterInvocationDefinitionSource,对每个请求检测授权细节。

随着Spring Security 2.0的发布,我想看看是不是可以替换acegi,但还要保持当前的功能,使用数据库作为我们验证和授权的数据源,而不是xml配置文件(大多数演示程序里使用的都是xml)。

这里是我采取的步骤.....

25.3. 步骤

- 第一步(也是最重要的)是下载新的Spring Security 2.0框架,并确保jar文件放到正确的位置(/WEB-INF/lib/)。

  Spring Security 2.0下载包里包含22个jar文件。我不需要把它们全用上(尤其是那些sources包)。在这次练习中我仅仅包含了以下几个:

  - spring-security-acl-2.0.0.jar
  - spring-security-core-2.0.0.jar
  - spring-security-core-tiger-2.0.0.jar
  - spring-security-taglibs-2.0.0.jar

- 在web.xml文件里配置一个DelegatingFilterProxy

```
<filter>
    <filter-name>springSecurityFilterChain</filter-name>
    <filter-class>org.springframework.web.filter.DelegatingFilterProxy</filter-
class>
</filter>
<filter-mapping>
    <filter-name>springSecurityFilterChain</filter-name>
    <url-pattern>/*</url-pattern>
</filter-mapping>
```

- Spring Security 2.0的配置比acegi简单太多了,所以我没有在以前acegi配置文件的基础上进行修改,我发现从一个空白文件开始更简单。如果你想修改你以前的配置文件,我确定你删除的行数比添加的行数还要多。

  配置文件的第一部分是指定安全资源过滤器的细节,这让安全资源可以通过数据库读取,而不是在配置文件里保存信息。这里是一个你将在大多数例子中看到的代码。

```
<http auto-config="true" access-denied-page="/403.jsp">
    <intercept-url pattern="/index.jsp" access="ROLE_ADMINISTRATOR,ROLE_USER"/>
    <intercept-url pattern="/securePage.jsp" access="ROLE_ADMINISTRATOR"/>
    <intercept-url pattern="/**" access="ROLE_ANONYMOUS" />
</http>
```

  使用这些内容进行替换:

```
<authentication-manager alias="authenticationManager"/>

<beans:bean id="accessDecisionManager"
class="org.springframework.security.vote.AffirmativeBased">
    <beans:property name="allowIfAllAbstainDecisions" value="false"/>
    <beans:property name="decisionVoters">
        <beans:list>
            <beans:bean class="org.springframework.security.vote.RoleVoter"/>
            <beans:bean
class="org.springframework.security.vote.AuthenticatedVoter"/>
        </beans:list>
    </beans:property>
</beans:bean>

<beans:bean id="filterInvocationInterceptor"
class="org.springframework.security.intercept.web.FilterSecurityInterceptor">
<beans:property name="authenticationManager" ref="authenticationManager"/>
    <beans:property name="accessDecisionManager" ref="accessDecisionManager"/>
    <beans:property name="objectDefinitionSource" ref="secureResourceFilter" />
</beans:bean>

<beans:bean id="secureResourceFilter"
class="org.security.SecureFilter.MySecureResourceFilter" />

<http auto-config="true" access-denied-page="/403.jsp">
    <concurrent-session-control max-sessions="1" exception-if-maximum-
exceeded="true" />
    <form-login login-page="/login.jsp" authentication-failure-url="/login.jsp"
default-target-url="/index.jsp" />
    <logout logout-success-url="/login.jsp"/>
</http>
```

  这段配置的主要部分secureResourceFilter,这是一个实现了FilterInvocationDefinitionSource的类,它在Spring Security需要对请求页面检测权限的时候调用。这里是MySecureResouceFilter的代码:

```
```

```
package org.security.SecureFilter;

import java.util.Collection;
import java.util.List;

import org.springframework.security.ConfigAttributeDefinition;
import org.springframework.security.ConfigAttributeEditor;
import org.springframework.security.intercept.web.FilterInvocation;
import
org.springframework.security.intercept.web.FilterInvocationDefinitionSource;


public class MySecureResourceFilter implements FilterInvocationDefinitionSource
{

    public ConfigAttributeDefinition getAttributes(Object filter) throws
IllegalArgumentException {

        FilterInvocation filterInvocation = (FilterInvocation) filter;

        String url = filterInvocation.getRequestUrl();

        // create a resource object that represents this Url object
        Resource resource = new Resource(url);

        if (resource == null) return null;
        else{
            ConfigAttributeEditor configAttrEditor = new
ConfigAttributeEditor();
            // get the Roles that can access this Url
            List<Role> roles = resource.getRoles();
            StringBuffer rolesList = new StringBuffer();
            for (Role role : roles){
                rolesList.append(role.getName());
                rolesList.append(",");
            }
            // don't want to end with a "," so remove the last ","
            if (rolesList.length() > 0)
                rolesList.replace(rolesList.length()-1, rolesList.length()+1,
"");
            configAttrEditor.setAsText(rolesList.toString());
            return (ConfigAttributeDefinition) configAttrEditor.getValue();
        }
    }

    public Collection getConfigAttributeDefinitions() {
        return null;
    }

    public boolean supports(Class arg0) {
        return true;
    }

}
```

getAttributes()方法返回权限的名称(我称之为角色),它们控制当前url的访问权限。

- 好了,现在我们需要安装信息数据库,下一步是让Spring Security从数据库中读取用户信息。这个Spring Security 2.0的例子告诉你如何从下面这样的配置文件里获得用户和权限的列表:

```
<authentication-provider>
    <user-service>
    <user name="rod" password="password" authorities="ROLE_SUPERVISOR,
ROLE_USER" />
    <user name="dianne" password="password" authorities="ROLE_USER,ROLE_TELLER"
/>
    <user name="scott" password="password" authorities="ROLE_USER" />
```

```
    <user name="peter" password="password" authorities="ROLE_USER" />
    </user-service>
</authentication-provider>
```

你可以把这些例子的配置替换掉,这样你可以像这样从数据库中直接读取用户信息:

```
<authentication-provider>
    <jdbc-user-service data-source-ref="dataSource" />
</authentication-provider>
```

这里有一种非常快速容易的方法来配置安全数据库,意思是你需要使用默认的数据库表结构。默认情况下,<jdbc-user-service>需要下面的几个表:
user,authorities,groups,group_members和group_authorities。

我的情况下,我的安全数据库表无法这样工作,它和<jdbc-user-service>要求的不同,所以我需要修改<authentication-provider>:

```
<authentication-provider>
    <jdbc-user-service data-source-ref="dataSource"
    users-by-username-query="SELECT U.username, U.password, U.accountEnabled AS
'enabled' FROM User U where U.username=?"
    authorities-by-username-query="SELECT U.username, R.name as 'authority'
FROM User U JOIN Authority A ON u.id = A.userId JOIN Role R ON R.id = A.roleId
WHERE U.username=?"/>
</authentication-provider>
```

通过添加users-by-username-query和authorities-by-username-query属性,你可以使用你自己的SQL覆盖默认的SQL语句。就像在acegi中一样,你必须确保你的SQL语句返回的列与Spring Security所期待的一样。这里有另一个group-authorities-by-username-query属性,我在这里没有用到,所以也没有出现在这里例子中,不过它的用法与其他两个SQl语句的方法完全一致。

<jdbc-user-service>的这些功能大概是在上个月才加入的,在Spring Security之前版本中是无法使用的。幸运的是它已经被加入到Spring Security中了,这让我们的工作更加简单。你可以通过 这里和这里获取信息。

dataSource bean中指示的是链接数据库的信息,它没有包含在我的配置文件中,因为它并不只用在安全中。这里是一个dataSource的例子,如果谁不熟悉可以参考一下:

```
<bean id="dataSource"
class="org.springframework.jdbc.datasource.DriverManagerDataSource">
    <property name="driverClassName" value="com.mysql.jdbc.Driver"/>
    <property name="url" value="jdbc:mysql://localhost/db_name?
useUnicode=true&characterEncoding=utf-8"/>
    <property name="username" value="root"/>
    <property name="password" value="pwd"/>
</bean>
```

- 这就是Spring Security的所有配置文件。我最后一项任务是修改以前的登陆页面。在acegi中你可以创建自己的登陆<form>,向正确的URL发送正确命名的HTML输入元素。现在你也可以在Spring Security 2.0里这样做,只是一些名称发生了改变。你可以像以前一样使用用户名j_username和密码j_password。

```
<input type="text" name="j_username" id="j_username"/>
<input type="password" name="j_password" id="j_password"/>
```

但是你必须把<form>中的action指向j_spring_security_check而不是j_acegi_security_check。

```
<form method="post" id="loginForm" action="<c:url
value='j_spring_security_check'/>"
```

在你的应用中有一些地方,用户可以进行注销,这是一个链接把注销请求发送给安全框架,这样它就可以进行相应的处理。需要把它从j_acegi_logout改成j_spring_security_logout。

```
<a href='<c:url value="j_spring_security_logout"/>'>Logout</a>
```

25.4. 总结

这个简短的指南,包含了如何配置Spring Security 2.0使用数据库中的资源,它并没有演示Spring Security 2.0中的新特性,然而我想它可以演示一些非常常用的框架功能,我希望你们觉得它有用。

Spring Security 2.0与acegi相比的好处之一是配置文件非常简单,这在我比较老acegi配置文件(172行)和新配置文件(42行)的时候,清楚的显示出路爱。

这里是我完整的securityContext.xml文件:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans:beans xmlns="http://www.springframework.org/schema/security"
xmlns:beans="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans,http://www.springframework
beans-
2.0.xsd,http://www.springframework.org/schema/security,http://www.springframework.org/sche
security-2.0.xsd">

        <authentication-manager alias="authenticationManager"/>

        <beans:bean id="accessDecisionManager"
class="org.springframework.security.vote.AffirmativeBased">
                <beans:property name="allowIfAllAbstainDecisions" value="false"/>
                <beans:property name="decisionVoters">
                        <beans:list>
                                <beans:bean
class="org.springframework.security.vote.RoleVoter"/>
                                <beans:bean
class="org.springframework.security.vote.AuthenticatedVoter"/>
                        </beans:list>
                </beans:property>
        </beans:bean>

        <beans:bean id="filterInvocationInterceptor"
class="org.springframework.security.intercept.web.FilterSecurityInterceptor">
                <beans:property name="authenticationManager"
ref="authenticationManager"/>
                <beans:property name="accessDecisionManager"
ref="accessDecisionManager"/>
                <beans:property name="objectDefinitionSource"
ref="secureResourceFilter" />
        </beans:bean>

        <beans:bean id="secureResourceFilter"
class="org.security.SecureFilter.MySecureResourceFilter" />

        <http auto-config="true" access-denied-page="/403.jsp">
                <concurrent-session-control max-sessions="1" exception-if-maximum-
exceeded="true" />
                <form-login login-page="/login.jsp" authentication-failure-
url="/login.jsp" default-target-url="/index.jsp" />
                <logout logout-success-url="/login.jsp"/>
        </http>

        <beans:bean id="loggerListener"
class="org.springframework.security.event.authentication.LoggerListener"/>
```

```
        <authentication-provider>
                <jdbc-user-service data-source-ref="dataSource" users-by-username-
query="SELECT U.username, U.password, U.accountEnabled AS 'enabled' FROM User U
where U.username=?" authorities-by-username-query="SELECT U.username, R.name as
'authority' FROM User U JOIN Authority A ON u.id = A.userId JOIN Role R ON R.id =
A.roleId WHERE U.username=?" />
        </authentication-provider>
</beans:beans>
```

就像在第一步时提到到,下载Spring Security是最重要的步骤。从那里开始就可以一帆风顺了。

Prev | Home

Chapter 24. 领域对象安全 | Sponsored by SpringSource